



Momento 2

Desenvolvimento de Aplicações Informáticas

PL4 - Grupo 42



Índice

Equipa	8
Introdução	9
1. <i>Iteration Plan</i>	10
1.1 Introdução	10
1.2 <i>Work Item List</i>	10
1.2.1 <i>Work Item List M3</i>	19
1.3 Lista de Problemas	19
1.4 Avaliação da Iteração	20
1.5 Medições do Projeto/ Avaliação do Estado	21
2. <i>Review Record</i>	24
3. <i>Work Order</i>	25
3.1 Ações de mitigação	27
4. <i>Vision</i>	29
4.1 Introdução	29
4.2 Posicionamento	29
4.2.1 <i>Problem Statement</i>	29
4.2.2 <i>Product Position Statement</i>	30
4.3 Descrição das partes interessadas (<i>stakeholders</i>)	31
4.3.1 Resumo das partes interessadas (<i>stakeholders</i>)	31

4.3.2 Ambiente do Utilizador	33
4.4 Visão Geral do Produto	35
4.4.1 Necessidades e Funcionalidades	35
4.5 Outros Requisitos do Produto.....	37
5. <i>System-Wide Requirements</i>	39
5.1 Introdução.....	39
5.2 Qualidades do Sistema	39
5.2.1 Usabilidade	39
5.2.2 Confiabilidade	40
5.2.3 Performance.....	40
5.2.4 Suportabilidade	41
5.4 Interfaces do Sistema.....	41
5.4.1 Interfaces do Utilizador.....	41
5.4.2 Interfaces para Sistemas Externos ou Dispositivos.....	42
5.5 Regras de Negócio.....	44
5.5.1 Privacidade.....	44
5.5.2 Restrições.....	44
5.5.3 Disponibilidade.....	44
5.6 Restrições do Sistema.....	45
5.7 Conformidade do Sistema.....	45
5.7.1 Requisitos de Licenciamento	45



5.7.2 Legalidades, Direitos de Autor e Outros Avisos	46
5.7.3 Leis Aplicáveis.....	46
5.8 Documentação do Sistema.....	47
6. <i>Project Defined Process</i>	48
6.1 Introdução.....	48
6.1.1 Propósito	48
6.2 Visão Geral do caso de Desenvolvimento.....	48
6.2.1 <i>Project LifeCycle</i>	48
6.2.2 Planos de Iteração de Amostra.....	49
6.3 <i>Work Products</i>	55
7. <i>Tools</i>	58
8. <i>Glossary</i>	61
9. <i>Architecture Notebook</i>	67
9.1 Propósito.....	67
9.2 Objetivos e Filosofia da Arquitetura.....	67
9.3 Pressupostos e Dependências	69
9.3.1 Pressupostos.....	70
9.3.2 Dependências	71
9.4 Requisitos Significativos da Arquitetura	72
9.5 Decisões, Restrições e Justificações	74
9.5.1 Decisões	75

9.5.2 Restrições.....	76
9.6 Mecanismos Arquitetónicos.....	76
9.6.1 Mecanismo de Simulação e Aquisição de Dados	77
9.6.3 Mecanismo de Comunicação e Integração de Sistemas	78
9.6.4 Mecanismo de Gestão dos Gémeos Digitais	78
9.6.5 Mecanismo da Interface de visualização do Gémeo Digital.....	79
9.6.6 Mecanismo Gestão de Eventos e Alertas Automáticos.....	79
9.7 Abstrações Chave	80
9.8 Camadas Arquitetónicas.....	81
9.8.1 Camadas e Componentes:.....	81
9.8.2 Consistência e Uniformidade	83
9.8.3 Benefícios	84
9.9 Visão Arquitetónica	85
9.9.1 Diagrama de mecanismos e camadas	85
9.9.2 Diagrama dos diferentes componentes do sistema	86
10. Modelo de Casos de Uso	87
10.1 Introdução	87
10.2 Catálogo de Atores	87
11. Especificação dos Casos de Uso.....	100
12. Casos de Teste	132
13. Build.....	139



13.1-Introdução.....	139
13.2-Model	139
13.2.1-Bus.java	140
13.2.2-BusLocation.java	141
13.2.3-BusRecord.java	141
13.3-Controller	143
13.3.1-BusController.java	143
13.3.2-BusDataController.java	145
13.3.3-BusSimulatorController.java.....	146
13.4-Repository	147
13.4.1-BusLocationRepository.java	148
13.4.2-BusRecordRepository.java.....	148
13.4.3-BusRepository.java	149
13.5-Service.....	149
13.5.1-BusDataService.java	149
13.5.2-BusSimulatorService.java.....	151
13.6-Config	152
13.6.1- WebConfig.java	153
13.7-BusMonitoringApplication.java.....	153
13.8- Interface.....	154



13.9 – Modelo Base de Dados	155
14. Software Development Plan	156
14.1 Organização do Projeto	156
14.2 Milestones e Objetivos.....	158
14.3 Lições aprendidas.....	161
15. <i>Risk List</i>	163
16. Conclusão	175
16.1 Ferramentas de IA.....	176

Equipa



Ana Alexandrina Rodrigues
A108713
A108713@uminho.pt



Ana Raquel Carvalho
A108732
A108732@uminho.pt



Beatriz Eduarda Martins
A108718
A108718@uminho.pt



Francisco Oliveira Ribeiro
A10
A10@uminho.pt



Marco António Mota
A108277
A108277@uminho.pt



Maria Sampaio e Mourão
A107406
A107406@uminho.pt



Rui Miguel Mendes
A108720
A108720@uminho.pt



Sandro Ferreira Mendes
A108450
A108277@uminho.pt



Yesika Alvarez Gomes
A108447
A108447@uminho.pt

Introdução

No âmbito da unidade curricular de Desenvolvimento de Aplicações Informáticas (DAI), este momento tem como finalidade apresentar o primeiro protótipo da nossa solução. Durante esta fase, alinhados com o OpenUP, estruturamos o nosso um plano para o nosso projeto que inclui o desenvolvimento da visão geral do produto bem como a lista de requisitos funcionais do sistema, ferramentas utilizadas, a arquitetura a ser implementada no sistema, modelos de caso de uso e diagramas.

Este projeto visa a criação de um software para os transportes urbanos de Braga com o propósito de monitorização em tempo real dos veículos utilizados no sistema Bus Rapid Transit (BRT), utilizando gémeos digitais dos autocarros, paragens e vias, permitindo uma gestão eficiente e intuitiva do mesmo. Esta interação tem como principal foco a recolha de requisitos e a criação de um plano alinhado com as expectativas e necessidade do cliente em causa.

Ao longo deste trabalho serão identificadas e detalhadas as características que devem ser incluídas no sistema em causa tais como os atores que serão usados no modelo de caso de uso, a arquitetura do software, e os primeiros testes ao protótipo desenvolvido. Através disto temos como objetivo que a solução produzida esteja conforme as exigências do cliente e proporcionando uma experiência intuitiva para os seus utilizadores.

1. Iteration Plan

1.1 Introdução

O Plano de Iteração detalha os objetivos da iteração, tarefas a realizar, recursos necessários, cronograma e prazos, além dos critérios de aceitação para avaliar o sucesso. Sendo um documento dinâmico, é revisto e atualizado regularmente para garantir que a equipa mantém o alinhamento com os seus objetivos.

1.2 Work Item List

Nome/ descrição dos artefactos	Prioridade	Estado	Iteração	Atribuído a	Horas de trabalho	Estimativa de horas restantes
Momento 1						
Plano do projeto (Project Plan)	Alta	Concluído	1	Gestor do Projeto e Analistas	8h	0h
- Introdução e Organização do projeto	Alta	Concluído	1	Raquel (Analista)	2h	0h
-Desenvolvimento e implementação das Práticas do projeto a adotar	Alta	Concluído	1	Beatriz (Gestor do Projeto)	3h	0h

-Desenvolver as <i>Milestones</i> e Objetivos	Alta	Concluído	1	Yesika (Analista)	2h	0h
-Refletir e desenvolver o <i>Deployment</i> (a implementação a ser feita)	Alta	Concluído	1	Beatriz (Gestor do Projeto)	1h	0h
Lista de Riscos (<i>Risk List</i>)	Alta	Concluído	1	Sandro e Marco (Programadores)	6h	0h
Plano de iteração (<i>Iteration Plan</i>)	Alta	Concluído	1	Programadores, Arquiteto e Testador	14h	0h
-Desenvolver a introdução para o <i>Iteration Plan</i> (Plano de Iteração)	Alta	Concluído	1	Rui (Programador)	1h	0h
-Desenvolver as <i>Key Milestones</i> (<i>Milestones-Chave</i>)	Alta	Concluído	1	Francisco (Programador)	1h	0h

-Desenvolver os <i>High-Level objectives</i> (Objetivos de Alto-Nível)	Alta	Concluído	1	Francisco (Programador)	2h	0h
-Desenvolver a <i>Work Item List</i> (Atribuição de Itens de Trabalho)	Alta	Concluído	1	Maria (Arquiteto)	4h	0h
- Desenvolver os <i>Issues</i> (Problemas)	Alta	Concluído	1	Rui (Programador)	2h	0h
-Desenvolver os <i>Evaluation Criteria</i> e os <i>assessments</i> (critérios de avaliação)	Alta	Concluído	1	Ana (Testador)	3h	0h
Momento 2						
Desenvolver/continuar plano de iteração	Alta	Concluído	2	Gestor do Projeto	8.5h	8h

Registo de revisões (Review Record)	Média	Concluído	2	Gestor do Projeto	4h	4h
Desenvolver a Ordem de trabalho (Work Order)	Alta	Concluído	2	Gestor do Projeto e Arquiteto	3h	4h
Visão (Vision)	Alta	Concluído	2	Beatriz(PM)e Maria (Arq)	7h	6h
Desenvolver Visão técnica	Alta	Concluído	2	Beatriz(PM)e Maria (Arq)	7h	6h
Identificar e Refinar Requisitos	Alta	Concluído	2	Analistas	0h	12h
Detalhar os Requisitos do sistema e desenvolver <i>System-Wide Requirements</i>	Alta	Concluído	2	Rui (Prog)	5h	10h
Desenvolver Casos de Uso e Elaborar	Alta	Concluído	2	Raquel (A) e Yesika (A)	28h	24h

Detalhar cenários de casos de uso	Alta	Concluído	2	Raquel (A) e Yesika (A)	20h	1h
Elaborar diagramas de sequência	Alta	Concluído	2	Maria (Arq) e Ana (T)	4h	6h
Desenvolver <i>Test Cases</i>	Alta	Concluído	2	Ana (T)	14h	12h
Processo Definido pelo Projeto (<i>Project Defined Process</i>)	Alta	Concluído	2	Arquiteto e Programadores	0h	8h
Desenvolver o <i>Project Defined Process</i>	Alta	Concluído	2	Sandro (Prog) e Francisco (Prog)	6h	4h
Adaptar o processo	Alta	Concluído	2	Arquiteto e Programadores	3h	3h
Implementar o processo	Alta	Concluído	2	Arquiteto e Programadores	6h	6h
Ferramentas (<i>Tools</i>)	Alta	Concluído	2	Programadores e Tester	0h	4h

Configurar e estudar as ferramentas	Alta	Concluído	2	Programadores e Tester	3h	3h
Verificar a Configuração e Instalação das Ferramentas	Alta	Concluído	2	Programadores e Tester	1h	1h
Glossário (<i>Glossary</i>)	Média	Concluído	2	Marco (Prog)	2h	3h
Desenvolver a Arquitetura	Alta	Concluído	2	Arquiteto	14h	20h
Desenvolver o <i>Architecture Notebook</i>	Alta	Concluído	2	Maria(Arq)	10h	10h
Visualizar e Refinar a Arquitetura	Alta	Concluído	2	Arquiteto e Programadores	2h	2h
Construção (<i>Build</i>)	Alta	Concluído	2	Programadores	0h	40h

Criar a Base da estrutura de Dados	Alta	Concluído	2	Rui (Prog)	1h	3h
Desenvolver o protótipo inicial da Digital Twin	Alta	Concluído	2	Rui (Prog)	15h	10h
Implementação da camada Model	Alta	Concluído	2	Programadores	11h(classes e camada)	12h
Criar classes, respetivas camadas e database	Alta	Concluído	2	Programadores	11h(classes e camada)	12h
Rever os artefactos do m1	Média	Concluído	2	Todos	3h	3h
Rever relatório	Média	Concluído	2	Todos	2h	2h
Formatar o Relatório	Média	Concluído	2	Raquel (A)	8h	7h
Momento 3						

Plano de iteração (Iteration Plan)	Alta	Por concluir	3	Gestor do Projeto	0h	6h
Registo de revisões (Review Record)	Média	Por concluir	3	Analistas	0h	3h
Ordem de trabalho (Work Order)	Alta	Por concluir	3	Gestor do Projeto e Arquiteto	0h	4h
Design	Alta	Por concluir	3	Programadores	0h	25h
Teste de Programador (Developer Test)	Média	Por concluir	3	Programadores	0h	14h
Test Script	Alta	Por concluir	3	Testador	0h	10h
Registo de Teste (Test Log)	Média	Por concluir	3	Programadores e Testador	0h	10h
Construção (Build)	Alta	Por concluir	3	Programadores	0h	60h
Rever artefactos do m1/m2	Média	Por concluir	3	Todos	0h	4h

Momento 5						
Avaliação da Iteração <i>(Iteration Assessment)</i>	Alta	Por concluir	5	Gestor do Projeto	0h	6h
Registo de Revisões <i>(Review Record)</i>	Média	Por concluir	5	Analistas	0h	6h
Implementação <i>(Implementation)</i>	Alta	Por concluir	5	Programadores e Testador	0h	22h
Construção <i>(Build)</i> - protótipo final	Alta	Por concluir	5	Arquiteto e Programadores	0h	60h
Documentação de Suporte <i>(Support Documentation)</i>	Média	Por concluir	5	Analistas	0h	6h
Rever artefactos do m1/m2/m3	Média	Por concluir	5	Todos	0h	6h

1.2.1 Work Item List M3

Momento	Tarefas	Nome	Prioridade	Estado	Data limite	Tr	Notas	Horas previstas
M3	Iteração 3 (M3)			Não iniciado	yyyy/mm/dd		Notas	
	Desenvolver/continuar plano de iteração	Beatriz (PM)	P0	Não iniciado	2025/03/28		Notas	11 h
	Issues List	Beatriz (PM)	P0	Não iniciado	2025/03/28		Notas	1 h
	Iteration Assessment	Beatriz (PM)	P3	Não iniciado	2025/05/02		Notas	1 h
	Project Measurements/Status Assessment	Beatriz (PM)	P3	Não iniciado	2025/05/02		Notas	2 h
	Review Record	Beatriz (PM) e Analistas	P3	Não iniciado	2025/05/02		Notas	3 h
	Work Order	Beatriz (PM)	P0	Não iniciado	2025/03/28		Notas	4 h
	Identificar e Refinar requisitos	Analistas e Arquiteto	P1	Não iniciado	2025/03/28		Notas	16 h
	Detalhar cenários de casos de uso	Analistas e Arquiteto	P1	Não iniciado	2025/03/28		Notas	8 h
	Detalhar System-Wide Requirements	Analistas	P1	Não iniciado	2025/03/28		Notas	8 h
	Desenvolver Incremento de Solução		P1	Não iniciado	2025/03/28		Notas	152 h
	Refinar a Arquitetura	Arquiteto	P1	Não iniciado	2025/04/04		Notas	10 h
	Design the solution	Programadores	P1	Não iniciado	2025/04/04		Notas	25 h
	Desenvolver o Design	Programadores	P1	Não iniciado	2025/04/04		Notas	6 h
	Implementar testes de desenvolvimento	Programadores	P2	Não iniciado	2025/04/11		Notas	14 h
	Desenvolver o artefacto Developer Test	Programadores	P2	Não iniciado	2025/04/11		Notas	3 h
	Implementar solução	Programadores	P2	Não iniciado	2025/04/11		Notas	20 h
	Executar testes de desenvolvimento	Programadores	P2	Não iniciado	2025/04/18		Notas	4 h
	Desenvolver o Test Log	Tester	P2	Não iniciado	2025/04/18		Notas	10 h
	Integrar e Criar Build	Programadores	P2	Não iniciado	2025/04/25		Notas	60 h
	Testar a solução		P3	Não iniciado	2025/04/25		Notas	19 h
	Implementar testes de desenvolvimento	Tester	P2	Não iniciado	2025/04/18		Notas	5 h
	Desenvolver o Test Script	Tester	P2	Não iniciado	2025/04/11		Notas	10 h
	Executar testes	Tester	P2	Não iniciado	2025/04/18		Notas	4 h
	Revisões do M1 e M2 (se necessário)	Beatriz (PM)	P3	Não iniciado	2025/04/25		Notas	2 h
	Rever relatório	Todos	P3	Não iniciado	2025/05/02		Notas	2 h
	Formatar relatório	Raquel e Beatriz	P3	Não iniciado	2025/05/02		Notas	4 h

1.3 Lista de Problemas

Problemas	Estado	Notas	Conclusão
Atraso na Elaboração dos Casos de Uso	Resolvido	Devido à complexidade na definição dos casos de uso e dos requisitos funcionais necessários	Após as reuniões e discussões de ideias, foram definidos os requisitos funcionais, distribuídas tarefas o que permitiu um melhor avanço na elaboração dos Casos de Uso
Desafios na preparação do ambiente	Resolvido	Dificuldade na instalação de algumas ferramentas, e falta de documentação clara sobre o uso das mesmas.	Após o estudo de cada uma das ferramentas a serem utilizadas, este problema foi ultrapassado.

Desafios na Arquitetura do sistema	Resolvido	Dificuldade em visualizar e validar a arquitetura, necessidade de ajustar a estrutura inicial.	Realizar revisões para validar a estrutura e definir padrões de integração desde o início.
Desafios na implementação e criação dos Gêmeos Digitais.	Resolvido	Dificuldade na implementação de código, integração com os gêmeos digitais e comunicação entre módulos.	A comunicação constante entre programadores, arquitetos e analistas, além do estudo e pesquisa das melhores abordagens para as implementações, foi essencial para o sucesso na criação e integração dos gêmeos digitais.
Gestão da carga de trabalho ao longo do projeto	Resolvido	A distribuição das tarefas exigiu alguns ajustes para evitar sobrecarga em determinadas fases da Iteração.	Gestão do tempo na organização da Iteração, planeando horas para a realização de cada tarefa.

1.4 Avaliação da Iteração

Foram identificadas falhas na execução de alguns outputs, o que atrasou o início de tarefas dependentes e exigiu um esforço adicional da equipa para corrigir os problemas e cumprir os prazos, nomeadamente na realização e elaboração dos casos de uso.

Apesar desses desafios, a equipa conseguiu superar obstáculos e garantir entregas de qualidade, satisfazendo os requisitos do cliente. Para evitar a repetição desses problemas em futuras iterações, será essencial identificar as causas, revisar processos de execução de outputs, reforçar a capacitação da equipa em áreas específicas e melhorar a comunicação e coordenação interna.

Para avaliar o desempenho da iteração e apoiar melhorias futuras, foram definidos critérios específicos que permitiram analisar os principais aspetos do desenvolvimento, conforme apresentado na tabela seguinte.

Critérios	Avaliação planeada
Cumprimento de prazos e planeamento	Verificar se as tarefas foram concluídas dentro dos prazos estabelecidos e se o planeamento foi eficaz
Qualidades dos Outputs	Avaliar se os outputs e a documentação técnica desenvolvida estão completos, bem definidos e adequados para suportar as próximas fases do projeto.
Qualidade da implementação	Avaliar se o código e a implementação seguem boas práticas, estão corretamente estruturados e funcionam conforme o esperado.

1.5 Medições do Projeto/ Avaliação do Estado

	M1		M2		
Nome	semana 1	semana 2	semana 3	semana 4	semana 5
Beatriz Martins	11 horas	10 horas	5 horas	5,5 horas	9 horas
Raquel Carvalho	9 horas	10 horas	5 horas	5 horas	18 horas
Yesika Gomes	6 horas	10 horas	5 horas	5 horas	11 horas
Maria Mourão	7 horas	7 horas	11 horas	4 horas	9 horas
Ana Rodrigues	7 horas	4 horas	1 hora	2 horas	14,5 horas
Rui Mendes	7 horas	4 horas	5 horas	9 horas	12 horas
Sandro Mendes	6 horas	3 horas	3 horas	7 horas	6 horas
Francisco Ribeiro	6 horas	3 horas	3 horas	7 horas	2 horas
Marco Mota	6 horas	5 horas	1 hora	4 horas	7 horas
TOTAL	65 horas	56 horas	39 horas	43,5 horas	88,5 horas
Horas Previstas		49 horas	40 horas	45 horas	80 horas

Durante a Iteração, foram feitos planeamentos e conduzidas avaliações semanais para acompanhar o progresso do projeto, analisando os indicadores de requisitos, qualidade e tempo/esforço.

1ª Semana

Tarefas		
Planeadas	24	
Concluídas	22	92%
Aprovadas	21	88%
Reprovadas	3	13%

Tempo/Esforço	
Tempo Planeado	49 horas
Tempo Real	56 horas

2ª Semana

Outputs		
Planeados	18	
Concluídos	13	72%
Aprovados	16	89%
Reprovados	2	11%

Tempo/Esforço	
Tempo Planeado	40 horas
Tempo Real	39 horas

3ª Semana

Outputs		
Planeados	27	
Concluídos	25	93%
Aprovados	25	93%
Reprovados	2	7%

Tempo/Esforço	
Tempo Planeado	45 horas
Tempo Real	43.5 horas

4ª Semana

Outputs		
Planeados	21	
Concluídos	22	100%
Aprovados	22	100%
Reprovados	0	100%

Tempo/Esforço	
Tempo Planeado	50 horas
Tempo Real	88,5 horas

2. Review Record

Todas as tarefas e artefactos foram avaliados com base em métricas e critérios definidos, seguindo um processo de revisão estruturado. Sempre que uma tarefa era concluída, o *project manager*, *tester* ou um analista revia os outputs, aprovando-os ou solicitando ajustes. Caso necessário, eram feitas refinações antes da aprovação final.

Tr Artefacto	Responsável pela revisão	Estado do artefacto	Estado da revisão	Tr Notas
Plano de Iteração	Todos	Em curso	Revisto	Rever o plano de iteração para o M3
Project Defined Process	Beatriz	Concluído	Revisto	Acrescentar os work products e âmbito
Architecture Notebook	Analistas e Programadores	Concluído	Revisto	Notas
Test Case	Beatriz, Raquel e Yesika	Em curso	Revisto	Notas
Tools	Beatriz	Concluído	Revisto	Acrescentar as ferramentas de desenvolvim
Build	Rui	Em curso	Revisto	Corrigir as classes
Vision	Beatriz	Concluído	Revisto	Notas
System-Wide-Requirements	Beatriz	Concluído	Revisto	Especificar mais para o nosso sistema
Casos de Uso	Maria e Ana	Concluído	Revisto	Notas
Risk List	Todos	Em curso	Revisto	Rever quais os riscos que ainda estão ativos

3. Work Order

Para o planeamento das quatro semanas da Iteração 2 (M2), foram organizadas e detalhadas as tarefas a serem realizadas, os responsáveis e as horas previstas para cada uma delas. Serviu como um guia para o acompanhamento do progresso, garantindo que todas as atividades fossem concluídas dentro do prazo estabelecido. As tarefas foram organizadas por prioridade e distribuídas entre os membros da equipa.

Semana 1

planeamento semana 2							
Tarefa	Prioridade	Nome	Estado	Data limite	Horas	Horas previstas	
Desenvolver/continuar plano de iteração	P0	Beatriz	Concluído	2025/02/23	3 horas	3 horas	
Desenvolver a Ordem de trabalho	P0	Beatriz	Concluído	2025/02/23	1 hora	1 hora	
Definir a ideia que vamos desenvolver	P0	Todos	Concluído	2025/02/23	2 horas	1 hora	
Pesquisar sobre o tema aplicações, estudar os seus papeis, e ferramentas	P0	Todos	Concluído	2025/02/24			
Identificar e Refinar Requisitos (System-Wide Requirements, Use Case Model, Test Case)	P1	Analistas e Arquiteta (R)	Em curso	2025/02/27			
System-Wide Requirements	P1	Rui Mendes	Concluído	2025/02/27	14 horas no total	12 horas total	
Use Case Model (iniciar pelo menos)	P1	Raquel e Yesika	Concluído	2025/02/27	R- 5 horas; Y-5 horas	8 horas	
Identificar os requisitos do sistema (funcionais e não funcionais)	P1	Analistas e Arquiteta (R)	Em curso	2025/02/27			
Criar e detalhar os casos de uso (Definir os atores e as suas interações com o sistema)	P1	Analistas (Raquel)	Concluído	2025/02/27	R- 4 horas	3 horas	
Criar uma visão geral dos casos de uso	P1	Analistas (Raquel) Yesika	Concluído	2025/02/27			
Preparação do ambiente (Project Defined Process e Tools)	P0	Programadores	Concluído	2025/02/27	4 horas total	4 horas total	
Desenvolver o Project Defined Process(Introdução, objetivo, modelo de ciclo de vida)	P1	Sandro	Concluído	2025/02/27	2 horas		
Desenvolver Project Defined Process(Workflow, Work products, Papéis)	P1	Francisco	Concluído	2025/02/27	2 horas		
Planear reunião com o cliente	P2	Todos, Yesika	Concluído	2025/02/27	Y- 4 horas	2 horas	
Desenvolver Tools	P1	Ana Rodrigues	Concluído	2025/02/27	2 horas	2 horas	
Configurar as ferramentas (escolher, instalar, preparar e estudar as ferramentas que serão usadas)	P0	Programadores	Concluído	2025/02/27			
Análise e configuração de ferramentas e início da criação de casos de teste	P2	Ana Rodrigues	Concluído	2025/02/27	1 hora	1 hora	
Desenvolvimento da arquitetura (identificar os principais componentes, começar Architecture Notebook)	P1	Maria	Concluído	2025/02/27	2 horas	2 horas	
Desenvolver a visão técnica inicial (Vision, Introdução, Posicionamento, Problem statement, Product Position Statement)	P1	Beatriz Martins	Concluído	2025/02/27	2 horas	2 horas	
Desenvolver vision(Stakeholder Description, Product overview, Other Product Requirements)	P1	Maria	Concluído	2025/02/27	4 horas	3 horas	
Reunião semanal	P1	Todos	Concluído	2025/02/26	1 hora	1 hora	
Desenvolver relatório semanal	P1	Beatriz Martins	Concluído	2025/02/27	1 hora	1 hora	

Semana 2

Tarefas_do_projeto1						
Tr	Tarefa	Prioridade	Nome	Estado	Data limite	Horas prevista
	Desenvolver/Continuar o plano de Iteração	P0	Beatriz	Concluído	2025/02/28	2 horas
	Finalizar os System-Wide Requirements	P0	Rui Mendes	Concluído	2025/03/03	3 horas
	Detalhar e completar os casos de Uso	P0	Raquel e Yesika	Em curso	2025/03/03	10 horas total
	Construção dos casos de uso 1, 5, e 6	P0	Raquel	Em curso	2025/03/03	3 horas
	Construção dos casos de uso 2, 3 e 4	P0	Yesika	Em curso	2025/03/03	3 horas
	Definir cenários completos de interação dos atores com o sistema	P0	Raquel e Yesika	Em curso	2025/03/03	
	Completar a visão geral dos casos de uso	P0	Raquel e Yesika	Em curso	2025/03/03	
	Finalizar architecture notebook	P0	Maria	Concluído	2025/03/05	10 horas total
	Finalização do desenvolvimento da Arquitetura	P0	Maria	Concluído	2025/03/05	9 horas total
	Finaliza esboço inicial da arquitetura(definir os principais módulos e a sua interação)	P0	Maria	Concluído	2025/03/05	
	Identificar as camadas do sistema	P0	Maria	Concluído	2025/03/05	
	Especificar os componentes essenciais	P0	Maria	Concluído	2025/03/05	
	Avançar no modelo da base de dados	P1	Maria	Concluído	2025/03/07	1 hora
	Avançar na implementação da Base de dados	P1	Programadores (Rui Mendes)	Concluído	2025/03/07	3 horas
	Definir Entidades, atributos e relações detalhadamente	P1	Rui	Concluído	2025/03/07	1 hora
	Diagramas de sequência ou classes	P1	Maria e Ana Rodrigues	Em curso	2025/03/07	
	Refinar o Project Defined Process	P2	Sandro e Kiko	Concluído	2025/03/07	1 hora
	Reunião semanal	P1	Todos	Concluído	2025/03/05	1 hora
	Desenvolver relatório semanal	P2	Beatriz Martins	Concluído	2025/03/07	1 hora

Semana 3

Tarefas_do_projeto2						
Tr	Tarefa	Prioridade	Tr Nome	Estado	Data limite	Horas previstas
	Desenvolver/Continuar plano de iteração	P0	Beatriz	Concluído	2025/03/14	2 horas
	Rever e atualizar planeamento do projeto	P1	Beatriz	Concluído	2025/03/14	1 hora
	Rever artefactos desenvolvidos até agora	P1	Beatriz	Concluído	2025/03/14	2 horas
	Finalização do desenvolvimento dos casos de uso	P0	Yesika e Raquel	Concluído	2025/03/12	10 horas (5 para cada)
	Finalização da descrição detalhada dos casos de uso	P0	Yesika e Raquel	Em curso	2025/03/12	8 horas(4 para cada)
	Escolher os diagramas de sequencia e de atividades	P0	Maria e Ana	Concluído	2025/03/12	1 hora
	Refinar architecture notebook	P0	Maria	Concluído	2025/03/14	2 horas
	Build	P1	Programadores	Em curso	2025/03/14	
	Fazer o prototipo inicial da digital twin	P1	Rui	Concluído	2025/03/14	4 horas
	Corrigir o Process Defined Process	P1	Sandro e Francisco	Concluído	2025/03/14	2 horas
	Implementação da camada Model	P1	Programadores	Concluído	2025/03/14	
	Criar as classes e das respetivas camadas:	P1	Programadores	Concluído	2025/03/14	
	Autocarro.java	P1	Rui	Concluído	2025/03/14	2 horas(para as classes e car
	RegistoAutocarro.java	P1	Marco	Concluído	2025/03/14	1 hora
	LocalizacaoAutocarro.java	P1	Rui	Concluído	2025/03/14	1 hora
	Via.java	P1	Sandro	Concluído	2025/03/14	3 horas (para as 3 classes e c
	Paradem.iava	P1	Sandro	Concluído	2025/03/14	4 horas (para as 3)
	Rota.java	P1	Sandro	Concluído	2025/03/14	
	DadoTrafego.java	P1	Marco	Concluído	2025/03/14	3 horas(para as 3 classes e c
	Alerta.java	P1	Marco	Concluído	2025/03/14	1 hora
	HistoricoManutencao.java	P1	Francisco	Concluído	2025/03/14	3 horas(para as 3 classes e c
	Sensor.java	P1	Francisco	Concluído	2025/03/14	4 horas(para as 3)
	RegistoSensor.java	P1	Francisco	Concluído	2025/03/14	
	Database	P1	Progamadores (cada um faz a da cl	Concluído	2025/03/14	
	Implementação da main	P1	Rui	Concluído	2025/03/14	1 hora
	Reunião Semanal	P1	Todos	Concluído	2025/03/12	1 hora

Semana 4

	A	B	C	D	E	F	G	H
	Tarefas_do_projeto3							
1	Tr Tarefa	Prioridade	Tr Nome	Estado	Data de fim	Horas previstas	Horas	Tr Notas
2	Desenvolver/ continuar plano de iteração	P1	Beatriz	Concluído	2025/03/20	2 horas	2 horas	Notas
3	Rever e atualizar plano do projeto	P1	Beatriz	Concluído	2025/03/20	1 hora	1 hora	Notas
4	Rever M1 e registar possíveis mudanças	P1	Beatriz	Concluído	2025/03/20	1 hora	1 hora	Notas
5	Rever todos os artefactos do M2	P2	Todos	Concluído	2025/03/20	1 hora		Notas
6	diagramas de sequência	P0	Maria	Concluído	2025/03/19	6 horas	4 horas	Notas
7	Terminar o detalhamento dos casos de uso	P0	Raquel e Yesika	Concluído	2025/03/16	8 horas (4 cada uma)	20 total(10 a ca	Notas
8	Fazer os test cases	P0	Ana	Concluído	2025/03/19	14 horas	12 horas	Notas
9	Refinar a Arquitetura	P0	Maria	Concluído	2025/03/20	4 horas	4 horas	Notas
10	Terminar o glossário	P1	Marco	Concluído	2025/03/20	1 hora	1 hora	Notas
11	Fazer a conclusão	P2	Beatriz	Concluído	2025/03/20	1 hora	1 hora	Notas
12	Terminar as tools	P0	Ana	Concluído	2025/03/20	1 hora	1,5 horas	Notas
13	Implementação	P1	Programadores	Concluído	2025/03/20			Notas
14	Corrigir classes	P1	Marco e Sandro	Concluído	2025/03/20	4 horas (2 horas para	4 horas	Notas
15	Continuar Implementação da digital twin	P1	Rui	Concluído	2025/03/20	2 horas		Notas
16	Implementação aplicação spring-boot Autoc	P1	Rui	Concluído	2025/03/20	11 horas	11 horas	Notas
17	Terminar o artefacto build	P0	Francisco, Marco e Sandr	Concluído	2025/03/20	3 horas	3 horas(1h para	Notas
18	Testar implementação do digital twin(API)	P2	Rui	Concluído	2025/03/20	1 hora		Notas
19	Rever e atualizar lista de riscos	P0	Sandro e Marco	Concluído	2025/03/20	2 horas	2 horas	Notas
20	Planejar M3	P1	Beatriz	Concluído	2025/03/20	3 horas	3 horas	(momento 3 e tod
21	Planejar semana 6	P1	Beatriz	Concluído	2025/03/20			Notas
22	Formatar Relatório	P2	Raquel	Concluído	2025/03/20	8 horas	7 horas	Notas
23	Reunião semanal	P1	Todos	Concluído	2025/03/19	1 hora	1 hora	Notas

3.1 Ações de mitigação

As estratégias adotadas até agora foram fundamentais para minimizar os atrasos, garantindo que o projeto avance dentro dos limites definidos.

- **Revisão do Cronograma:** Avaliação e ajustes nos prazos das tarefas que apresentaram atrasos, com o objetivo de assegurar que o progresso do projeto siga conforme os objetivos gerais, apesar dos imprevistos.
- **Realocação de Recursos:** Distribuição de mais membros da equipa para as tarefas que estavam em atraso, visando acelerar a execução e garantir que as tarefas mais críticas sejam concluídas dentro dos novos prazos.

- **Intensificação da Comunicação e Monitoramento:** Aumento da interação entre os membros da equipa, por meio de reuniões regulares, com o intuito de acompanhar o andamento das tarefas e resolver problemas rapidamente.
- **Gestão de Riscos:** Identificação de potenciais riscos que possam surgir durante o andamento do projeto e elaboração de estratégias de contingência para mitigar esses riscos.

4. Vision

4.1 Introdução

Este artefacto tem como principal objetivo capturar a visão das partes interessadas, garantindo que as necessidades, expectativas e requisitos do cliente sejam atendidos. Nele, são definidos os requisitos essenciais que consideramos fundamentais para o funcionamento eficiente do sistema, proporcionando uma visão geral dos processos a serem desenvolvidos e uma descrição clara das suas funcionalidades.

Este passo é fundamental, uma vez que permite obter uma análise preliminar do projeto, facilitando a comunicação entre a equipa e as partes interessadas. Dessa forma, todos os envolvidos mantêm um entendimento alinhado sobre o produto a ser desenvolvido, promovendo um melhor acompanhamento do seu progresso.

4.2 Posicionamento

4.2.1 Problem Statement

Problema	A ausência de um sistema de monitorização em tempo real do estado dos autocarros, impede o acompanhamento de fatores essenciais, como temperatura interna, falhas mecânicas e localização.
Afeta	Passageiros da TUB, TUB, departamento de manutenção e gestor de frota.
Impacto	Avárias frequentes que resultam em atrasos, aumento do tempo de inatividade dos autocarros, interrupções no serviço e inconveniência para os passageiros.
Solução	Um sistema <i>Digital Twin</i> que permita a monitorização contínua dos autocarros e das infraestruturas, deteção de falhas e otimização da operação, garantindo um serviço mais eficiente e seguro.

4.2.2 Product Position Statement

Para	Cliente: TUB Utilizadores finais: Departamento de manutenção, Gestor de frota da TUB e departamento de Sistemas de Informação.
Que	Monitorizar em tempo real os autocarros, analisar os dados recebidos fornecendo soluções para os problemas detetados.
Produto	TwinTUB
Descrição	Através da monitorização em tempo real, permite detetar falhas rapidamente, reduzir custos de manutenção, aumentar a segurança operacional e melhorar a eficiência do transporte, garantindo uma resposta ágil e uma preparação antecipada para eventuais problemas que afetam os veículos e passageiros.
Competitividade	<i>World Sat</i> Aplicação de gestão de frotas de autocarros com monitorização em tempo real, otimizando a operação e segurança dos veículos. Permite rastrear a localização, verificar velocidade, tensão da bateria e outros parâmetros essenciais. Inclui funcionalidades como cercas eletrônicas e relatórios detalhados de viagens, facilitando a manutenção preventiva e a gestão eficiente da frota.
O nosso Produto	A nossa solução integra Gémeos Digitais para prever falhas mecânicas, otimizar a manutenção e reduzir custos e inatividade. Além da monitorização em tempo real, permite a deteção antecipada de anomalias, integração com sensores e infraestrutura rodoviária, e análises avançadas para melhorar a eficiência operacional. Isto garante uma gestão inteligente e preventiva da frota, melhorando a segurança, a confiabilidade e a experiência do utilizador.

4.3 Descrição das partes interessadas (*stakeholders*)

4.3.1 Resumo das partes interessadas (*stakeholders*)

Nome	Descrição	Responsabilidades
Equipa de desenvolvimento	Equipa constituída por 9 pessoas responsáveis por implementar a aplicação a ser desenvolvida.	A equipa é responsável por desenvolver o projeto de modo a satisfazer os objetivos e requisitos propostos pelo cliente, bem como em garantir a qualidade técnica do sistema. Cada elemento da equipa desempenha uma função: gestor do projeto (1), arquiteto (1), analista (2), programador (4) e testador (1).
TUB	Empresa responsável pela operação de autocarros que pretende implementar o sistema BRT na cidade de Braga	As responsabilidades da TUB passam por: definir claramente os requisitos, trabalhar na adaptação interna do produto, implementar tecnologias para melhorar a gestão da frota e fornecer suporte no desenvolvimento da solução.

Passageiros da TUB	Utilizadores do serviço de transporte público que dependem dos autocarros para deslocações diárias.	Beneficiam da redução dos tempos de espera e melhoria da pontualidade dos autocarros, graças à otimização da alocação de veículos com base na contagem de passageiros e padrões de utilização. A segurança e a confiabilidade do serviço também serão aumentadas devido à monitorização em tempo real do estado dos autocarros.
Departamento de Manutenção	Departamento encarregado de realizar as manutenções necessárias nos autocarros.	Responsável por garantir a operacionalidade, segurança e eficiência da frota. Assim, a plataforma seria usada para aceder a informações relativas às necessidades de manutenção das viaturas.
Departamento de Sistemas de Informação	Unidade responsável pela gestão da infraestrutura tecnológica da TUB e pela integração de novos sistemas.	Responsável por garantir a integração do TwinTUB com os sistemas internos da TUB, assegurar a gestão e segurança dos dados, dar suporte técnico à plataforma, garantir a disponibilidade e escalabilidade do sistema, e apoiar na extração e análise de dados para otimizar a operação da frota.

Gestor de Frota	Profissional encarregado da coordenação e gestão operacional da frota de autocarros da TUB.	Responsável por monitorizar a utilização dos autocarros, gerir a distribuição da frota conforme a procura dos passageiros, otimizar a alocação de veículos com base nos dados do TwinTUB, e tomar decisões estratégicas para melhorar a eficiência e reduzir custos operacionais. Utilizará o sistema para obter informações em tempo real sobre localização, estado dos veículos e padrões de utilização dos passageiros.
------------------------	---	--

4.3.2 Ambiente do Utilizador

A plataforma TwinTUB será utilizada por diferentes intervenientes, cada um com um papel específico dentro do ecossistema da TUB. Entre os utilizadores mais relevantes destacam-se o Gestor de Frota, o Departamento de Manutenção e o Departamento de Sistemas de Informação (DSI), que irão interagir com o sistema de formas distintas consoante as suas necessidades operacionais.

O fluxo de operação do TwinTUB ajusta-se ao tipo de utilizador e às funcionalidades disponíveis. O Gestor de Frota recorrerá à plataforma para monitorizar a utilização dos autocarros, analisar padrões de deslocação dos passageiros e otimizar a distribuição da frota. Com base nas informações geradas em tempo real, será possível ajustar rotas e horários para melhorar a eficiência do serviço, reduzir tempos de espera e responder rapidamente a imprevistos.

O Departamento de Manutenção terá um papel essencial na supervisão do estado mecânico dos autocarros e na deteção de falhas, recorrendo à análise contínua de dados recolhidos pelos sensores e pelas câmaras equipadas com inteligência artificial. O sistema fornecerá alertas, permitindo uma intervenção antecipada e garantindo que a frota se mantém

operacional com o mínimo de tempo de inatividade. Sensores serão responsáveis por monitorizar parâmetros como temperatura interna, estado do motor e funcionamento de componentes críticos, prevenindo avarias inesperadas e reduzindo custos operacionais.

O Departamento de Sistemas de Informação será responsável pela integração do TwinTUB com os sistemas internos da TUB, assegurando que os dados recolhidos são processados, armazenados e utilizados de forma eficiente e segura. Além disso, terá a missão de garantir a compatibilidade com outras plataformas de gestão de transporte, assegurando que a solução se mantém compatível com os processos operacionais já existentes na empresa.

A recolha e transmissão de dados do TwinTUB ocorrerá de forma contínua, assegurando a monitorização em tempo real da localização e do estado dos veículos. Para isso, a plataforma utilizará GPS/GNSS instalados nos autocarros para localização precisa da frota. A comunicação dos dados será realizada através de Wi-Fi nas estações e redes móveis 4G/5G durante os percursos, garantindo uma transmissão fiável. Esta infraestrutura permitirá um rastreio detalhado dos autocarros, possibilitando uma gestão mais eficiente da frota, o cumprimento rigoroso dos horários e a otimização das rotas conforme a procura e as condições de tráfego.

Além da vertente operacional, o TwinTUB também desempenha um papel crucial na contagem de passageiros, utilizando câmaras com inteligência artificial para analisar padrões de utilização e ajustar a oferta de veículos conforme a procura. Esta funcionalidade permitirá à TUB otimizar a gestão da frota, reduzindo tempos de espera e melhorando a experiência dos passageiros.

Por fim, a plataforma será desenvolvida com uma interface acessível, garantindo que os diferentes utilizadores possam explorar todas as funcionalidades do sistema de forma intuitiva.

O objetivo final do sistema é potenciar um serviço de transporte público mais eficiente, seguro e inteligente, de forma a reduzir tempos de espera, a aumentar a fiabilidade da frota e a promover uma mobilidade urbana mais sustentável para a cidade de Braga.

4.4 Visão Geral do Produto

4.4.1 Necessidades e Funcionalidades

Necessidade	Prioridade	Funcionalidades
Monitorização em tempo real das viaturas	5	- Criação de Gémeos Digitais das viaturas para representar cada autocarro em tempo real.. - Recolha de dados dos sensores (temperatura interna, estado do motor, eficiência mecânica). - Detecção antecipada de falhas mecânicas. - Emissão de alertas para manutenção preventiva, de forma a evitar falhas operacionais.
Monitorização em tempo real das vias	5	- Criação do Gémeo Digital das Vias, refletindo as condições das estradas. - Monitorização contínua das vias para identificar obstáculos. - Utilização de câmaras com IA e sensores para análise das condições da estrada e previsão de problemas. - Geração de alertas automáticos para os operadores ajustarem as rotas conforme necessário.
Localização da viatura e	5	- Acompanhamento da localização em tempo real via GPS. - Transmissão contínua de dados via Wi-Fi nas

cumprimento de horários		estações e 4G/5G durante os percursos. - Comparação da posição com os horários previstos para otimização da gestão da frota.
Contagem em tempo real dos passageiros no MetroBus	4	- Câmaras de inteligência artificial para contagem automática de passageiros. - Identificação de padrões de utilização para ajuste dinâmico da oferta de veículos.
Deteção de obstáculos nas vias	4	- Identificação de obstruções, acidentes e congestionamentos em tempo real. - Geração de alertas automáticos sobre problemas na via para que os operadores reajam rapidamente.
Integração com sistemas internos da TUB	4	- Compatibilidade do TwinTUB com os sistemas de gestão de transporte já utilizados pela TUB. - Gestão e armazenamento seguro dos dados pelo Departamento de Sistemas de Informação (DSI).
Interface para operadores e gestores de frota	3	- Painel de monitorização em tempo real dos gêmeos digitais das viaturas, vias e paragens. - Relatórios analíticos e simulações, permitindo a tomada de decisões. - Sistema de gestão de alertas, exibindo notificações críticas para os operadores agirem rapidamente.

--	--	--

4.5 Outros Requisitos do Produto

Tipo	Requisito	Prioridade
RU	Visualização clara e organizada dos dados da frota e passageiros	5
RU	Modelos claros das estruturas das viaturas, das paragens e das vias.	4
RC	Tolerância a falhas e rápida recuperação de serviços em caso de falha do sistema	5
RC	Exibição sucinta e relevante de informações e alertas	3
RC	Sistema de armazenamento seguro	4
RD	Atualização em tempo real da localização dos veículos	5
RD	Atualização em tempo real da ocupação das viaturas (contagem dos passageiros)	5
RD	Capacidade de processar grande volume de dados sem perda de desempenho	5

RS	Suporte a atualizações	3
RI	Interface com cores intuitivas	2
RI	Interface de fácil compreensão	3

5. System-Wide Requirements

5.1 Introdução

Os requisitos de alto nível do sistema têm como objetivo descrever restrições e qualidades essenciais que se aplicam ao sistema como um todo, completamente os requisitos funcionais específicos. Este artefacto assegura uma visão abrangente do produto, garantindo que todas as partes do sistema operam em conformidade com as expectativas das partes interessadas e respeitam as diretrizes estabelecidas. Além disso, facilita a validação e alinhamento dos requisitos, permitindo um desenvolvimento coerente e estruturado.

5.2 Qualidades do Sistema

5.2.1 Usabilidade

Os requisitos de usabilidade avaliam a interface com o utilizador e incluem fatores como prevenção de erros, estética e design, ajuda, documentação e consistência.

Facilidade de aprendizagem: O utilizador deve ser capaz de aprender a utilizar o sistema rapidamente.

Eficiência das tarefas: As tarefas devem ser concluídas de forma rápida e eficiente.

Apresentação clara e concisa de informações e Alertas: A apresentação dos dados recolhidos e dos alertas deve ser clara, objetiva e concisa, garantindo a informação essencial sobre o estado dos autocarros sem comprometer a sua compreensão.

Compreensibilidade: O sistema deve ser intuitivo e fácil de perceber.

Satisfação do utilizador: O sistema deve proporcionar uma experiência agradável e eficaz.

5.2.2 Confiabilidade

Os requisitos de confiabilidade garantem a integridade e robustez do sistema, assegurando a sua estabilidade mesmo em condições adversas.

Precisão: O sistema deve apresentar uma elevada precisão no tratamento de dados captados, garantida manutenção preventiva e assegurando que os cálculos outputs correspondam à realidade.

Disponibilidade: O sistema deve estar acessível durante longos períodos, com paragens apenas para manutenção.

Recuperação: Em caso de falha, o sistema deve ser restaurado rapidamente através de backups e planos de contingência.

Resistência e gestão de falhas: O sistema deve ser robusto e fiável, garantindo estabilidade mesmo perante falhas ocasionais, sejam humanas ou técnicas. Os erros serão categorizados segundo a sua severidade (menor, significativo ou crítico).

5.2.3 Performance

Os requisitos de desempenho garantem uma operação eficiente e rápida.

Tempos de resposta: O tempo entre a ação do utilizador e a resposta do sistema deve ser reduzido.

Taxa de transferência: O sistema deve atualizar os dados periodicamente de forma ágil, garantindo a sua performance mesmo com grandes fluxos de informação,

Capacidade: O sistema deve suportar a monitorização de toda a frota da TUB e estar preparado para um aumento da procura.

5.2.4 Suportabilidade

A suportabilidade inclui compatibilidade, escalabilidade e capacidade de manutenção do sistema.

Suporte a atualizações: O sistema deve ter suporte integrado, adaptando-se a mudanças tecnológicas, aumento da frota e novas necessidades.

Compatibilidade: A aplicação será compatível com Android e iOS.

Instalação: O software será instalado nos dispositivos da TUB.

Expansibilidade: O sistema será escalável para suportar um aumento de utilizadores.

5.4 Interfaces do Sistema

Descrição do conjunto de requisitos relacionados às interfaces de utilizador que serão implementadas no software do TwinTUB Digital Twin.

5.4.1 Interfaces do Utilizador

5.4.1.1 *Look & Feel*

A interface gráfica será desenvolvida para proporcionar alta usabilidade, permitindo acesso rápido às informações do sistema de transporte público. O design será simples e intuitivo, utilizando as cores e identidade visual dos Transportes Urbanos de Braga (TUB). O sistema será otimizado para mínimo número de interações, garantindo um fluxo ágil e acessível para diferentes utilizadores.

Além disso, será suportado em múltiplos dispositivos, incluindo computadores, tablets e estações de trabalho dos departamentos responsáveis.

5.4.1.2 Requisitos de Layout e Navegação

O sistema terá informação útil para diferentes perfis de utilizadores, garantindo acesso a toda a informação, mas organizada conforme as necessidades de cada função. Os dados serão agrupados para otimizar a experiência:

- Gestor de Frota;
- Departamento de Manutenção;
- Departamento de Sistemas de Informação (DSI).

Cada tipo de utilizador terá acesso às funcionalidades agrupadas conforme as suas necessidades. No entanto, a estrutura de navegação será padronizada para facilitar a experiência do utilizador.

5.4.1.3 Consistência

As interfaces irão apresentar menus claros e intuitivos, com títulos descritivos para facilitar a navegação. Recursos interativos, como visualizações de mapas, gráficos de desempenho e notificações em tempo real, serão incorporados para fornecer informações relevantes de forma acessível.

Os dados históricos e relatórios serão armazenados na nuvem, garantindo acesso contínuo sem sobrecarregar os dispositivos locais.

5.4.2 Interfaces para Sistemas Externos ou Dispositivos

5.4.2.1 Interfaces de Software

O Digital Twin interagirá com diversos sistemas, incluindo:

- Sistemas internos da TUB para sincronização de dados operacionais e gestão da frota.

- Sensores *IoT* e câmaras de inteligência artificial instalados nos autocarros e infraestruturas rodoviárias para recolha e transmissão de dados.
- Base de dados central para armazenamento e gestão de informações sobre desempenho, tráfego e incidentes.

5.4.2.2 Interfaces de Hardware

Para um funcionamento eficiente, o sistema necessitará de:

- Conexão contínua à Internet para sincronização de dados em tempo real.
- Servidores *cloud* para armazenamento e processamento de informações dos autocarros e vias.
- Sensores inteligentes nos autocarros, responsáveis por monitorizar o estado do veículo, lotação, consumo de combustível, entre outros.
- Câmaras de inteligência artificial, responsáveis por fazer a contagem dos passageiros e detetar possíveis anomalias nas vias.

5.4.2.3 Interfaces de Comunicação e Notificações

O sistema integrará funcionalidades de notificação para diferentes utilizadores:

Gestor de Frota: Relatórios automáticos sobre desempenho da frota, eficiência operacional e ajustes necessários.

Departamento de Manutenção: Alertas sobre necessidade de manutenção, falhas mecânicas e desempenho dos veículos.

Departamento de Sistemas de Informação (DSI): Monitorização da infraestrutura digital, armazenamento seguro e integração com plataformas externas.

5.4.2.4 Monitorização e Otimização

O sistema incluirá ferramentas para otimizar a operação da TUB, permitindo:

- Análise preditiva de manutenção para reduzir o tempo de inatividade da frota.
- Ajuste dinâmico da frota com base na procura e tráfego em tempo real.
- Armazenamento seguro e escalável para lidar com grandes volumes de dados operacionais.

5.5 Regras de Negócio

5.5.1 Privacidade

As informações dos utilizadores e os dados operacionais da empresa serão protegidos contra acessos não autorizados. Nenhuma informação será compartilhada com terceiros sem autorização explícita, garantindo conformidade com as normas de proteção de dados.

5.5.2 Restrições

O acesso ao sistema será limitado a utilizadores com credenciais verificadas, incluindo coordenadores, administradores e gestores da frota. Medidas adicionais de segurança, como autenticação de dois fatores, poderão ser implementadas para reforçar a proteção dos dados.

5.5.3 Disponibilidade

A plataforma TwinTUB estará operacional 24 horas por dia, 365 dias por ano, garantindo acesso contínuo e ininterrupto às informações essenciais para a gestão da frota e manutenção dos veículos.

5.6 Restrições do Sistema

O Digital Twin integrará sistemas internos da TUB, sensores IoT e uma base de dados central para monitorização e gestão da frota em tempo real.

O sistema será compatível com Linux, Windows, x86 e ARM, garantindo operação offline com sincronização periódica para atualização contínua.

Para maior eficiência, funcionará mesmo em condições de conectividade limitada, utilizando cache e compressão de dados.

A interface será intuitiva e acessível, eliminando a necessidade de recursos externos.

O servidor central permanecerá sempre online, assegurando a integridade e estabilidade do sistema.

Além disso, o sistema permitirá a monitorização via dispositivos móveis (Android/iOS) e o hardware *IoT*, este será compacto, energeticamente eficiente e resistente a temperaturas extremas.

5.7 Conformidade do Sistema

5.7.1 Requisitos de Licenciamento

Os dados pessoais dos utilizadores armazenados pela aplicação serão utilizados exclusivamente para as funcionalidades previstas, garantindo conformidade com as normas de privacidade e proteção de dados. O acesso às funcionalidades da aplicação será gerido por um administrador autorizado da TUB.

5.7.2 Legalidades, Direitos de Autor e Outros Avisos

A aplicação será desenvolvida no âmbito da unidade curricular Desenvolvimento de Aplicações Informáticas da Universidade do Minho. Os direitos de autor pertencem à equipa de desenvolvimento do projeto.

A aplicação cumprirá o Regulamento Geral sobre a Proteção de Dados (GDPR), assegurando que todas as informações dos utilizadores estão protegidas e utilizadas conforme a legislação em vigor.

A aplicação não utilizará tecnologias patenteadas de terceiros sem autorização, e os logotipos e marcas serão aplicados apenas dentro das diretrizes e licenças permitidas.

5.7.3 Leis Aplicáveis

A aplicação estará sujeita às seguintes leis e regulamentações:

5.7.3.1 Lei de Proteção de Dados Pessoais

A aplicação seguirá as disposições estabelecidas pelo Regulamento Geral sobre a Proteção de Dados (GDPR) na União Europeia, garantindo a segurança e privacidade dos dados pessoais dos utilizadores.

5.7.3.2 Legislação de Propriedade Intelectual

A equipa assegurará o cumprimento das leis de propriedade intelectual, respeitando os direitos autorais associados ao desenvolvimento da aplicação. Isso inclui o uso adequado de bibliotecas de código aberto e o respeito por patentes e direitos autorais de terceiros.

5.7.3.3 Normativas do Setor de Transporte Público

A aplicação obedecerá às normativas e regulamentos específicos do setor de transporte público, garantindo conformidade com regras de segurança, padrões de acessibilidade e diretrizes das autoridades competentes.

5.8 Documentação do Sistema

A documentação do sistema incluirá todos os documentos necessários para a instalação, configuração e operação da aplicação, garantindo que os utilizadores compreendem o funcionamento e utilização do TwinTUB Digital Twin.

Manual de Instalação: Instruções detalhadas para a correta instalação do sistema, minimizando erros que possam comprometer a sua funcionalidade.

Guias de Utilização: Documentação específica para cada perfil de utilizador, descrevendo as principais funcionalidades e fluxos de trabalho:

- **Login:** O Gestor de Frota, Departamento de Manutenção e DSI fornecerão os seus dados ao administrador para criação de conta e atribuição de credenciais de acesso.
- **Gestor de Frota:** Acesso às rotas/vias, horários e paragens, avaliar e analisar relatórios sobre a eficiência e o desempenho dos veículos e gestão operacional em tempo real.
- **Departamento de Manutenção:** Monitorização do estado mecânico dos veículos e sensores, gestão de alertas e planeamento de manutenção preventiva.
- **Departamento de Sistemas de Informação (DSI):** Gestão da infraestrutura digital, segurança dos dados, efetuar diagnósticos, solucionar questões técnicas que afetem o desempenho do sistema e integração com outros sistemas.
- **Gémeos digitais para Monitorização de Frota:** Monitorização em tempo real do desempenho dos veículos, exibindo dados como localização, combustível, temperatura do motor, pressão dos pneus entre outros. Estado e condições das vias e para as paragens, informação de lotação, atrasos e tempos de chegada dos autocarros. Gera alertas para parâmetros críticos, como necessidade de manutenção ou anomalias.

6. *Project Defined Process*

6.1 Introdução

6.1.1 Propósito

O *Project Defined Process* constitui uma abordagem estruturada que descreve, de forma detalhada, as etapas e procedimentos a seguir durante o desenvolvimento de um projeto. Esta metodologia permite definir e organizar as atividades, identificando os produtos de trabalho e os marcos essenciais, desde a conceção inicial até à entrega final. Ao estabelecer um plano claro e consistente, promove-se uma comunicação eficaz entre os diversos intervenientes, garantindo que os objetivos e os requisitos do sistema são devidamente alinhados com as expectativas dos clientes.

6.2 Visão Geral do caso de Desenvolvimento

6.2.1 *Project LifeCycle*

No modelo que adotamos, o OpenUP define o ciclo de vida do projeto por meio de um processo iterativo dividido em quatro fases principais:

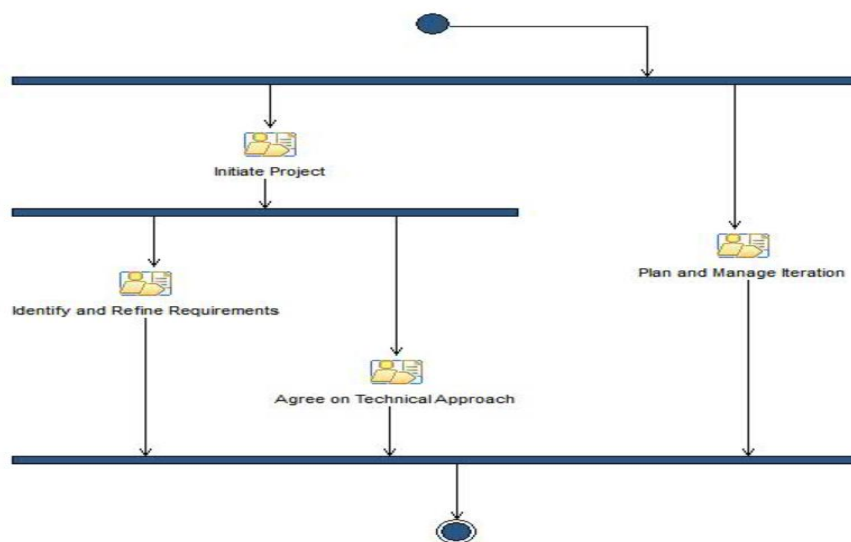
- **Inception**
- **Elaboration**
- **Construction**
- **Transition**

Cada uma dessas fases é composta por iterações, onde cada ciclo representa um marco no desenvolvimento e contribui para o alcance dos objetivos propostos.

6.2.2 Planos de Iteração de Amostra

6.2.2.1 Fase da *Inception*

Na fase da Inception definem-se as atividades e os produtos de trabalho realizados durante uma iteração típica da fase de Iniciação. Esta etapa abrange tarefas que visam obter o consenso dos clientes e o planeamento do projeto. Elabora-se um plano que contempla a identificação dos riscos, as estimativas e os prazos, com o intuito de definir prioridades e identificar os requisitos do sistema.



Podemos descrever as diferentes atividades da fase da Inception da seguinte forma:

- **Initiate Project**
 - Desenvolver a Visão Técnica;
 - Planear o Projeto.

- **Plan and Manage Iteration**
 - Planear a iteração;
 - Preparar o ambiente;
 - Adaptar o processo;
 - Configurar ferramentas;

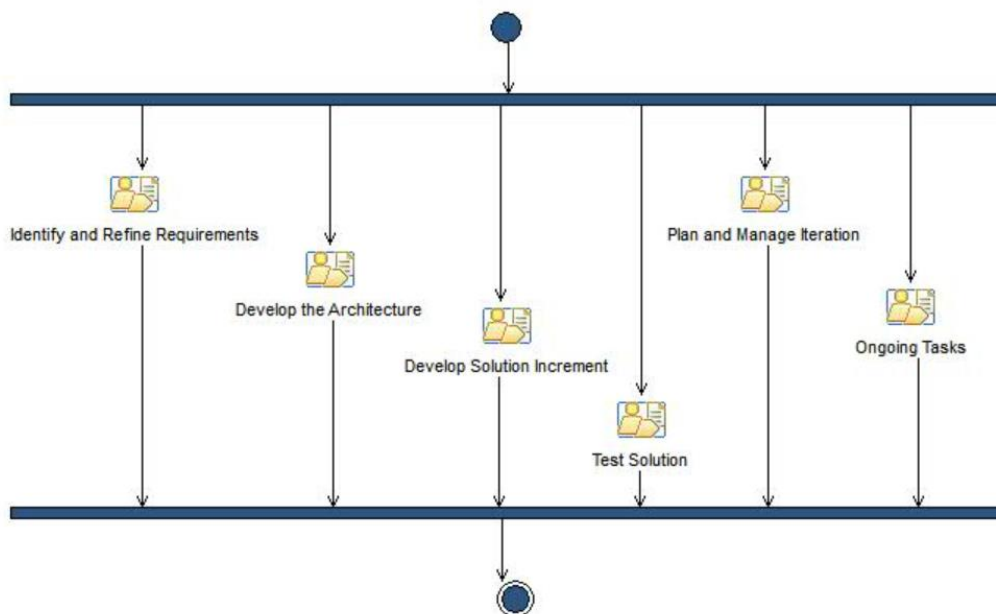
- Verificar configuração e instalação das ferramentas;
- Implementar o processo.

- **Identify and Refine Requirements**

- Identificar e detalhar os requisitos;
- Detalhar cenários de casos de uso;
- Detalhar os requisitos do sistema;
- Criar os casos de teste.

6.2.2.2 Fase da *Elaboration*

Durante a fase de Elaboração, o foco está na compreensão aprofundada e na melhoria dos requisitos levantados na fase anterior. Para isso, realizam-se análises minuciosas que visam confirmar a viabilidade do projeto, bem como a definição das primeiras arquiteturas e conceitos de design. Nesta etapa, é crucial estabelecer uma base robusta para a arquitetura do sistema, mitigando os riscos mais significativos. O principal objetivo é criar os alicerces necessários para o desenvolvimento posterior de uma solução que satisfaça as expectativas das partes interessadas.



- **Plan and Manage Iteration**

- Planear a iteração;

- Preparar o ambiente;
- Adaptar o processo;
- Configurar ferramentas;
- Verificar configuração de ferramentas e instalação;
- Implementar o processo;
- Gerir a iteração;
- Delinear o plano de implementação;
- Avaliar resultados.

- ***Identify and Refine Requirements***

- Identificar e detalhar requisitos;
- Detalhar cenários de caso de uso;
- Detalhar requisitos do sistema;
- Criar casos de teste.

- ***Develop the architecture***

- Desenvolver a solução;
- Aperfeiçoar a arquitetura.

- ***Develop Solution Increment***

- Desenhar a solução
- Implementar testes de desenvolvimento;
- Implementar a solução;
- Executar testes de desenvolvimento;
- Integrar e criar a construção.

- ***Test solution***

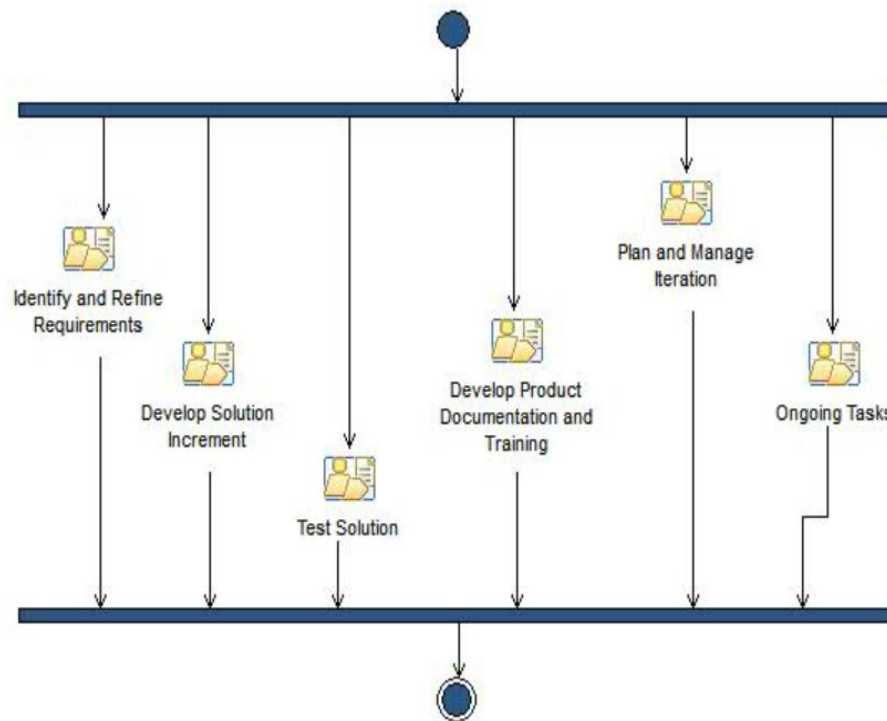
- Implementar testes;
- Executar testes.

- ***Ongoing Tasks***

- Pedir alteração.

6.2.2.3 Fase da construção

Nesta etapa, o sistema de software é efetivamente desenvolvido, concentrando-se na implementação dos requisitos previamente definidos. As atividades que caracterizam a fase de Construção podem ser descritas da seguinte forma:



- ***Plan and Manage Iteration***
 - Planear a iteração;
 - Preparar o ambiente;
 - Adaptar o processo;
 - Configurar ferramentas;
 - Verificar configuração de ferramentas e instalação;
 - Implementar o processo;
 - Gerir a iteração;
 - Avaliar resultados.
- ***Identify and Refine Requirements***
 - Identificar e detalhar requisitos;

- Detalhar casos de uso;
- Detalhar requisitos do sistema;
- Criar casos de teste.

- ***Develop Solution Increment***
 - Desenhar a solução;
 - Implementar testes de desenvolvimento;
 - Implementar a solução;
 - Executar testes de desenvolvimento;
 - Integrar e criar a construção.

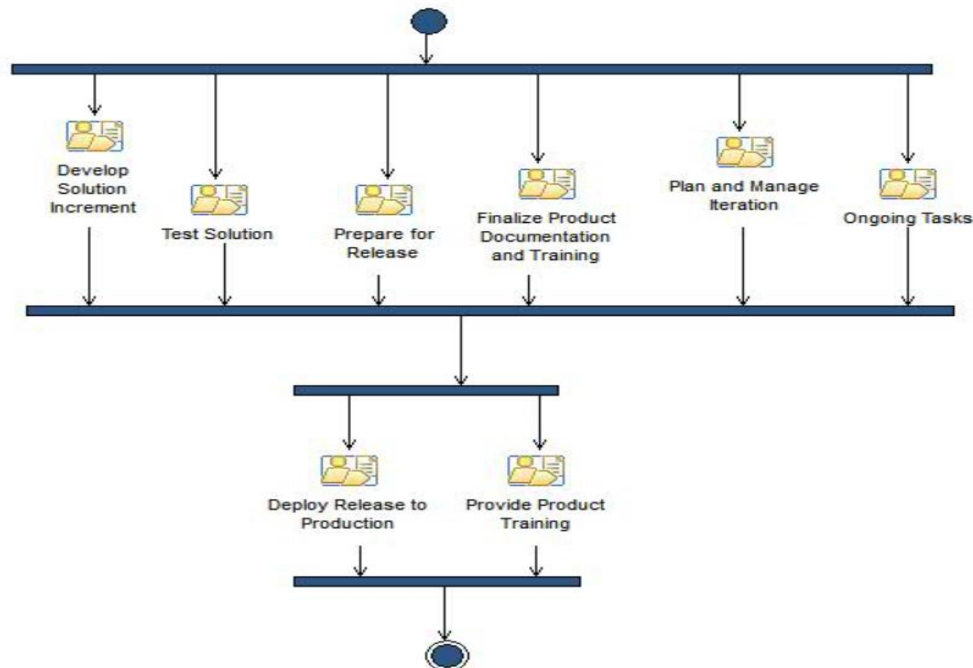
- ***Test Solution***
 - Implementar testes;
 - Executar testes.

- ***Ongoing Tasks***
 - Pedir alteração

- ***Develop Product Documentation and Training***
 - Desenvolver a documentação do produto
 - Desenvolver a documentação do utilizador
 - Desenvolver a documentação de suporte;
 - Desenvolver materiais de treino.

6.2.2.4 Fase da Transição

Nesta fase, o software é entregue oficialmente ao cliente, passando por diversas etapas de teste. O objetivo central é implementar o sistema de forma definitiva, garantindo a aprovação das partes interessadas e a conformidade com as expectativas estabelecidas.



- **Plan and Manage Iteration**

- Planear a iteração;
- Preparar o ambiente;
- Adaptar o processo;
- Configurar ferramentas;
- Verificar configuração de ferramentas e instalação;
- Implementar o processo;
- Gerir a iteração;
- Avaliar resultados.

- **Develop Solution Increment**

- Desenhar a solução;
- Implementar testes do desenvolvedor;
- Implementar a solução;

- Executar testes do desenvolvedor;
- Integrar e criar a construção.

- **Test Solutions**

- Implementar testes;
- Executar testes.

- **Finalize Product Documentation and Training**

- Desenvolver a documentação do produto;
- Desenvolver a documentação do utilizador;
- Desenvolver a documentação de suporte;
- Desenvolver materiais de treino.

- **Prepare for Release**

- Rever e entrar em conformidade com os controlos de lançamento;
- Instalar e validar infraestrutura;
- Desenvolver o plano de recuperação;
- Desenvolver comunicações de lançamento;
- Fornecer treinamento de produtos;
- Entregar treino do utilizador final;
- Entregar treino de suporte.

- **Ongoing Tasks**

- Pedir alteração.

6.3 Work Products

- **Processo de Definição do Projeto (*Project Defined Process*):** Serve de suporte para orientar os diferentes membros na estruturação e desenvolvimento do projeto, garantindo alinhamento com os objetivos definidos para a replicação virtual dos componentes rodoviários.

- **Lista de Riscos (*Risk List*):** Identifica e documenta os riscos associados ao desenvolvimento do *Digital Twin*, permitindo a mitigação de falhas e a antecipação de desafios técnicos e operacionais.
- **Ferramentas (*Tools*):** Conjunto de tecnologias e frameworks utilizados para a implementação do *Digital Twin*, incluindo Java 22 e a ferramenta NIOP da Neadvance para um dos componentes.
- **Glossário (*Glossary*):** Define os termos técnicos e específicos do projeto, assegurando uma comunicação clara e eficiente entre os *stakeholders*, a equipa de desenvolvimento e os utilizadores do sistema.
- **Visão (*Vision*):** Representa a perspetiva dos *stakeholders* sobre o projeto de *Digital Twins*, descrevendo a importância da monitorização, prevenção e segurança operacional em tempo real.
- **Requisitos Abrangentes do Sistema (*System Wide Requirements*):** Define os atributos essenciais e restrições do sistema, incluindo critérios como usabilidade, confiabilidade, desempenho e suportabilidade, garantindo um desvio máximo de 1% entre grandezas físicas e digitais.
- **Modelo de Caso de Uso (*Use Case Model*):** Fornece uma visão geral do comportamento esperado do sistema, descrevendo interações entre os diferentes componentes do *Digital Twin*, como viaturas, vias e paragens.
- **Especificação de Caso de Uso (*Use Case Specification*):** Analisa detalhadamente um cenário específico de utilização do *Digital Twin*, como a monitorização de ocorrências ou a comunicação entre elementos virtuais e físicos.
- **Caderno de Arquitetura (*Architecture Notebook*):** Documenta a arquitetura de software que orientará o desenvolvimento do sistema, incluindo decisões técnicas, padrões adotados

e integração entre componentes.

- **Casos de Teste (*Test Case*):** Conjunto de testes desenvolvidos para validar o comportamento do sistema, garantindo a conformidade com os requisitos e a robustez da implementação.
- **Comprovação do Conceito (*Proof of Concept*):** Processo que permite validar a viabilidade do *Digital Twin* e identificar as tecnologias mais adequadas para a sua implementação, assegurando que o sistema pode cumprir os requisitos operacionais e de desempenho estabelecidos.

7. Tools

As *Tools* são fundamentais para suportar o desenvolvimento de software. No planeamento de um projeto, é essencial analisar as particularidades do pedido do cliente para criar uma lista de ferramentas ajustada às necessidades específicas. Estas ferramentas permitem que as equipas construam o produto esperado de forma eficiente e organizada, proporcionando o suporte necessário para todo o processo de desenvolvimento.

Tipo de Ferramenta	Ferramenta	Descrição
Comunicação	WhatsApp	Aplicação para comunicação rápida da equipa.
Elaboração de Documentos	Google Docs	Ferramenta para criação e edição colaborativa de documentos.
Gestão de Dados e Tarefas	Google Sheets	Planilhas para organização e análise de dados.
Armazenamento	Google Drive	Plataforma para armazenar e partilhar ficheiros.
Metodologia de Desenvolvimento	OpenUP	Processo ágil utilizado na gestão do desenvolvimento do projeto.
Pesquisa e Navegação	Browsers	Ferramentas para pesquisa e navegação na web.

Desenvolvimento	VS Code	Editor de código utilizado no desenvolvimento.
Desenvolvimento	Java	Linguagem de programação utilizada no <i>back-end</i> do sistema.
Desenvolvimento	Spring Boot	Framework para desenvolvimento de aplicações <i>back-end</i> em Java.
Desenvolvimento	Maven	Ferramenta de gestão de dependências para projetos Java.
Desenvolvimento	Xampp	Painel de controlo para gerenciar servidores locais.
Base de Dados	MySQL	Permite o tratamento e gestão dos dados dos utilizadores da aplicação.
Plataforma Low-Code	NIOP	Ferramenta utilizada para desenvolvimento de aplicações <i>low-code</i> .
Repositório de Código	GitHub	Plataforma para gestão de repositórios de código.
Modelagem 3D	Blender	Software utilizado para modelagem 3D dos componentes do projeto.

Diagramação	Lucidchart	Ferramenta para criação de diagramas.
Diagramação	Draw.io	Ferramenta para criação de diagramas.
Web Development	HTML	Linguagem de marcação utilizada no desenvolvimento web.
Web Development	CSS	Linguagem de estilo utilizada para definir o <i>layout</i> e design.
Web Development	JavaScript	Linguagem de programação utilizada no <i>front-end</i> .
Inteligência Artificial	ChatGPT	Assistente de IA utilizado para suporte técnico e desenvolvimento de ideias.

8. Glossary

Este artefacto tem como objetivo identificar e clarificar as principais ideias e conceitos relacionados com o projeto em desenvolvimento. Sendo assim, com a criação deste glossário pretendemos que fique explícito para as partes interessadas o que pretendemos fazer neste trabalho e quais os termos e definições usados, facilitando a compreensão e assegurando a uniformidade do projeto.

As definições apresentadas a seguir estão ordenadas por ordem alfabética.

—A—

Analista: Responsável pela recolha e análise dos requisitos e expectativas para o projeto, garantindo que as necessidades das partes interessadas sejam compreendidas e realizadas.

API de Geolocalização: Interface que permite a obtenção de dados de localização em tempo real para monitorizar a frota de autocarros.

Application Programming Interfaces (APIs): Métodos que permitem a comunicação entre os diferentes softwares. No caso da TUB, usaremos as APIs para aceder a dados em tempo real sobre por exemplo a localização do veículo em causa.

Arquiteto: Responsável pela estrutura do software, garantindo que as decisões tomadas pela equipa serão devidamente implementadas.

Arquitetura do Software: Modelo estruturado que define a organização do sistema, incluindo o banco de dados, APIs, *front-end* e *back-end*, assegurando eficiência e usabilidade do sistema.

Artefactos: Recurso que pode ser criado ou atualizado pela equipa responsável pelo desenvolvimento do projeto, ajudando na compreensão e na descrição dos componentes necessários para o software.

Ator: Entidade externa que interage no software como por exemplo os passageiros.

Autenticação: Processo de verificação da entidade do utilizador antes de acessar o sistema como por exemplo o login por e-mail.

—B—

Back-end: Parte do sistema que é responsável pelo processamento dos dados e lógica do software, além de conectar a *front-end* com a base de dados e APIs.

Base de dados: Utilizadas para armazenar e organizar dados para facilitar o acesso e gestão dos mesmos.

Bus Rapid Transit Braga (BRT-Braga): Sistema de transporte implementado em Braga.

—C—

Camada de Processamento e Armazenamento de Dados: Estrutura responsável por armazenar e processar grandes volumes de dados operacionais do sistema TwinTUB.

Cenário de Caso de Uso: Representa um caso específico onde é descrito como o utilizador interage com o sistema em diferentes situações.

Computação em Nuvem: Tecnologia utilizada para processar, armazenar e aceder remotamente aos dados do sistema.

—D—

Dados Operacionais: Informações recolhidas em tempo real sobre o estado dos veículos, vias e ocupação dos autocarros.

Dashboards Interativos: Interfaces gráficas que apresentam visualmente os dados de monitorização do sistema.

Digital Twin: Representação virtual do sistema de transporte, permitindo a monitorização e análise de dados em tempo real.

—E—

Edge Computing: Processamento de dados próximo da fonte, reduzindo a latência da comunicação entre sensores e gémeos digitais.

—F—

Falha Crítica: Erro ou avaria que compromete a operação dos autocarros ou do sistema TwinTUB, exigindo resposta imediata.

Front-end: Parte do sistema com a qual o utilizador interage diretamente, incluindo interface gráfica e elementos visuais da aplicação.

—G—

Gestão de Frota: Processo de monitorização e otimização da operação dos autocarros para melhorar a eficiência do transporte público.

Gestor de Projeto: Responsável por planear, coordenar e supervisionar todo o desenvolvimento do projeto, garantindo que os prazos de entrega e objetivos sejam cumpridos.

—I—

Inteligência Artificial (IA): Tecnologia usada para analisar dados, prever falhas e otimizar a operação do sistema de transportes.

Interface: Elementos gráficos e interativos que permitem ao utilizador interagir com o sistema, como botões, menus e formulários.

Internet das Coisas (IoT): Rede de dispositivos conectados, como sensores e câmaras, que recolhem e transmitem dados em tempo real.

—L—

Lista de Riscos (*Risk List*): Recurso responsável por identificar os possíveis problemas no desenvolvimento do projeto.

—M—

Machine Learning: Subcampo da inteligência artificial usado para analisar padrões nos dados operacionais e prever falhas nos veículos.

Mecanismo de Alertas Automáticos: Sistema que identifica falhas e gera notificações para a equipa de manutenção e gestores de frota.

Milestones: Pontos de controlo do projeto que marcam a conclusão de fases cruciais para o projeto.

—N—

Notificação em Tempo Real: Mensagem automática enviada para operadores e gestores quando ocorre um evento crítico no sistema.

—O—

OpenUP: Metodologia de desenvolvimento usada no projeto para garantir que a equipa tenha uma perceção clara das entregas bem como a qualidade que deve possuir.

—P—

Plano de Manutenção Preventiva: Estratégia baseada na análise de dados para antecipar falhas mecânicas e reduzir tempo de inatividade dos veículos.

Programador: Responsável pela escrita e pela manutenção do código do sistema, garantindo a funcionalidade e a qualidade do mesmo.

—R—

Repositório: Local de armazenamento do código do sistema, bem como a documentação do projeto, permitindo a colaboração dos programadores.

—S—

Sensores IoT: Dispositivos instalados nos autocarros e vias que recolhem dados sobre a operação do sistema.

Sistema de Monitorização: Plataforma que centraliza os dados recolhidos pelos sensores, permitindo análise e otimização da frota.

Software: Conjunto de programas e sistemas desenvolvidos para o projeto TUB.

Stakeholder: Inclui qualquer parte interessada abrangendo clientes, equipa de desenvolvimento e utilizadores.

—T—

Tester: Responsável por testar o programa desenvolvido pelos programadores, documentando possíveis erros ou melhorias a serem efetuadas.

Tolerância a falhas: Capacidade do sistema TwinTUB de continuar a operar mesmo em caso de falhas parciais.

9. Architecture Notebook

9.1 Propósito

O Caderno de Arquitetura tem como objetivo fornecer uma diretriz clara e organizada para a implementação da plataforma TwinTUB, garantindo que as decisões arquitetónicas sejam documentadas e compreendidas por toda a equipa de desenvolvimento. Este artefacto descreve decisões técnicas, raciocínios, restrições, suposições e implementações necessárias para a construção do sistema, proporcionando uma visão estruturada da sua arquitetura.

Além disso, o documento apresenta a filosofia do sistema, os seus elementos fundamentais, design e estrutura, permitindo que a equipa compreenda as implicações das decisões tomadas e como estas influenciam a sua evolução. O Caderno de Arquitetura também funciona como uma referência para futuras manutenções e melhorias, garantindo que qualquer alteração à arquitetura respeite os princípios estabelecidos.

Outro ponto essencial é a sua função orientadora para os programadores, servindo como guia para a implementação da aplicação. O documento detalha como os componentes do sistema interagem, como os dados são tratados e quais as restrições arquitetónicas que devem ser respeitadas, assegurando um desenvolvimento alinhado com os objetivos técnicos e operacionais do projeto.

De forma simples e concisa, este artefacto visa uniformizar o conhecimento sobre a arquitetura do sistema dentro da equipa, garantindo que todos os membros compreendem a estrutura e as motivações das escolhas tecnológicas. Ao longo do desenvolvimento e da operação do TwinTUB, o Caderno de Arquitetura será um recurso essencial para garantir a coerência, escalabilidade e eficiência da solução.

9.2 Objetivos e Filosofia da Arquitetura

O objetivo principal da arquitetura do TwinTUB é fornecer uma representação digital precisa e dinâmica da frota de autocarros da TUB, da via onde circula e das paragens por onde passa,

permitindo monitorização em tempo real, análise de padrões de utilização e otimização operacional. Através da integração de Gémeos Digitais, sensores IoT e câmaras de Inteligência Artificial (IA), o sistema facilitará a gestão da frota, garantindo um serviço mais eficiente, seguro e sustentável.

Assim, para cumprir de replicação digital e interação em tempo real, a arquitetura da TwinTUB deve atender aos seguintes objetivos:

1-Digital Twin de Autocarros

- Recolha e monitorização de dados dos autocarros, incluindo localização GPS, consumo energético, estado do motor e bateria, temperatura interna, pressão dos pneus e anomalias mecânicas.
- Implementação de um histórico de manutenção com registo de inspeções e planeamento de intervenções preventivas.
- Gerar alertas em caso de falhas mecânicas ou incidentes como colisões e atrasos.
- Contagem de passageiros através de câmaras de inteligência artificial, permitindo:
 - Medir a lotação do autocarro em tempo real.
 - Enviar dados para a previsão de capacidade futura nas paragens.
 - Otimizar a distribuição da frota, evitando sobrecarga de veículos.

2- Digital Twin das Vias

- Monitorização do estado das vias em tempo real, incluindo identificação de obstruções através de câmaras de inteligência artificial.
- Relacionamento dinâmico com os trajetos percorridos pelos autocarros, otimizando a navegação e ajustando previsões de chegada.

3- Digital Twin das Paragens

- Cálculo da lotação prevista do próximo autocarro, permitindo uma melhor gestão do fluxo de passageiros.
- Exibir informação de atrasos e previsão de chegada.

- Gerar alertas em caso de problemas, como obstruções na via ou alterações inesperadas no serviço.

Quanto à filosofia da arquitetura, esta assenta nos seguintes pilares:

- **Desempenho:** O sistema tem de ser capaz de processar volumes significativos de dados, sem que haja perda de eficiência.
- **Integração:** É essencial que o sistema garanta compatibilidade com sensores e câmaras de inteligência artificial.
- **Escalabilidade:** Deve ser moldável e escalável de modo a lidar com sistemas físicos complexos e de grande escala.
- **Manutenção:** Deve ser assegurada uma estrutura resistente e adaptável para uma manutenção a longo prazo.

Problemas Críticos:

- **Dependências de hardware:** É necessário isolar dependências críticas, como sensores IoT e dispositivos de recolha de dados, de modo a evitar falhas locais que afetem o sistema global. Estratégias de redundância e buffering devem ser implementadas.
- **Funcionamento sob condições incomuns:** O sistema deve operar eficientemente mesmo sob alta carga de dados, latência de rede ou falhas parciais. Mecanismos como cache local, processamento assíncrono e otimização garantem a continuidade do desempenho.

9.3 Pressupostos e Dependências

A implementação bem-sucedida do TwinTUB envolve uma série de fatores críticos que devem ser considerados no âmbito da arquitetura do projeto. Estes fatores incluem pressupostos fundamentais para o funcionamento do sistema e dependências tecnológicas que impactam a sua fiabilidade e desempenho.

9.3.1 Pressupostos

- Todas as viaturas, vias e paragens terão sensores fiáveis e em quantidade suficiente (os sensores não são físicos, mas sim simulados inicialmente via PowerShell Script, enviando dados em JSON para o sistema) para a recolha de dados em tempo real.
- Os dados simulados representarão fielmente variáveis operacionais como localização GPS, temperatura, pressão e ocupação de veículos.
- Os sensores IoT (GPS, câmaras com IA, sensores de temperatura, pressão, entre outros) terão um comportamento consistente e previsível, garantindo um desvio máximo de 1% entre os valores físicos e os digitais.
- A infraestrutura de rede terá capacidade suficiente para a transmissão contínua de dados entre os sensores e os Gémeos Digitais, garantindo um atraso inferior a 1 segundo.
- O desenvolvimento do sistema contará com tecnologia robusta e eficiente, capaz de suportar grandes volumes de dados e assegurar um desempenho escalável até 1000 componentes rodoviários.
- A Base de Dados terá capacidade suficiente para armazenar registos históricos e estatísticas sobre a operação dos autocarros, vias e paragens, permitindo análises.
- Os modelos digitais gerados pelo sistema representarão com fidelidade o estado físico dos autocarros, vias e paragens, garantindo a precisão necessária para a tomada de decisões em tempo real.
- O sistema de simulação será capaz de gerar cenários baseados em dados reais, auxiliando na gestão da frota e otimização de trajetos.
- Os dados recolhidos serão protegidos por um sistema de segurança eficiente, garantindo a integridade e autenticidade das informações, evitando adulterações.
- O sistema terá mecanismos de recuperação de falhas, permitindo a continuidade operacional em caso de problemas técnicos ou humanos.
- A arquitetura do TwinTUB será suficientemente modular e flexível, permitindo a adição de novos sensores e ajustes ao modelo consoante mudanças nas viaturas, vias ou infraestrutura de transportes.
- O sistema será capaz de integrar-se com novas soluções tecnológicas ao longo do tempo, garantindo a evolução contínua e a compatibilidade com inovações futuras.

9.3.2 Dependências

A arquitetura do TwinTUB depende de certos elementos externos e internos que devem ser considerados para garantir a sua operação eficiente e fiável. As seguintes dependências devem ser tidas em conta:

- Execução correta e flexível dos scripts de simulação → O funcionamento do sistema depende da geração e manipulação de dados simulados via PowerShell, garantindo que os sensores virtuais enviam informações coerentes para a API/Comando (CMD). A flexibilidade dos scripts permite a simulação de diferentes cenários operacionais.
- Infraestrutura de transmissão de dados → O sistema requer uma rede estável e de baixa latência para garantir a comunicação eficiente entre os componentes do sistema (API/CMD, Java, Spring Boot, Base de Dados e NIOP).
- Serviços de mapas → A monitorização da posição dos autocarros e a otimização dos trajetos dependem da integração com APIs de mapas.
- Processamento e armazenamento de dados → O Java (VS Code) recebe, processa e armazena os dados simulados, assegurando que a Base de Dados suporta grandes volumes de informação sem comprometer a performance.
- Capacidade da infraestrutura de servidores/cloud → O armazenamento e processamento eficiente dos dados requer uma plataforma de computação em nuvem ou servidores locais que possam lidar com a grande quantidade de informações geradas em tempo real.
- Base de dados centralizada → O sistema necessita de uma base de dados segura e escalável para armazenar dados sobre autocarros, vias e paragens, garantindo a fiabilidade das análises.
- Tecnologia de Implementação → A aplicação será desenvolvida em Java 22, (backend e processamento), Spring Boot (gestão dos dados e APIs) e NIOP (componente de apoio aos gémeos digitais). Quaisquer limitações ou incompatibilidades nestas tecnologias podem afetar a integração e desempenho do sistema.
- Regulamentações e conformidade → O funcionamento do sistema deve estar em conformidade com as normas e regulamentações relacionadas com os transportes públicos, a privacidade de dados e a utilização de inteligência artificial para monitorização.

9.4 Requisitos Significativos da Arquitetura

Sincronização em Tempo Real

- O sistema TwinTUB deve ser capaz de recolher e processar os dados simulados dos sensores em tempo real, garantindo que a latência entre a geração dos dados e a sua representação digital não ultrapasse 1 segundo.
- A comunicação entre API/CMD, Java, Spring Boot e Base de Dados deve permitir uma resposta rápida às atualizações dos Digital Twins.

Processamento e Armazenamento de Dados

A arquitetura deve suportar grandes volumes de dados estruturados e não estruturados, incluindo:

- Dados de localização GPS dos autocarros;
- Informações sobre o estado mecânico das viaturas (motor, bateria, pneus, etc.);
- Dados de sensores IoT para deteção de obstáculos nas vias;
- Contagem de passageiros em tempo real através de câmaras com IA.
- Dados operacionais dos gémeos digitais, armazenados na Base de Dados e utilizados pelo Spring Boot para atualização dos modelos.
- Modelos 3D gerados no Blender, processados e exibidos na plataforma de visualização do TwinTUB.
- Integração com NIOP, permitindo uma gestão eficiente dos Digital Twins, incluindo informações sobre a lotação dos autocarros e condições das vias.

O sistema deve garantir eficiência no armazenamento e a possibilidade de análise para a otimização do serviço.

Contagem de Passageiros e Previsão de Lotação

- O sistema deve monitorizar a ocupação dos autocarros em tempo real, garantindo uma visão clara da lotação dos veículos e ajudando a prever a necessidade de reforço da frota.
- A contagem pode ser feita através de:
 - Simulação de entradas e saídas via PowerShell, permitindo cenários de teste controlados.

- Algoritmos de IA/Vision Computacional, quando utilizarmos câmaras para deteção automática de passageiros.
- Os dados da lotação devem ser armazenados na Base de Dados e exibidos na visualização do TwinTUB, permitindo:
 - Previsão da lotação futura nas paragens, otimizando a gestão dos transportes.
 - Alertas automáticos, caso a capacidade máxima dos autocarros esteja prestes a ser atingida.
 - Sugestões de redistribuição da frota para minimizar tempos de espera.

Detetar Obstáculos nas Vias

- O sistema deve ser capaz de detetar obstáculos nas vias em tempo real, utilizando:
 - Sensores simulados, que geram eventos indicando a presença de um obstáculo.
 - Câmaras com IA, que analisam imagens para identificar acidentes, obras ou bloqueios.
- A informação sobre obstruções será armazenada na Base de Dados e utilizada para:
 - Atualizar a visualização dos gémeos digitais e alertar operadores.
 - Ajustar trajetos e tempos estimados de chegada

Monitorização do Estado e Manutenção Preventiva dos Autocarros

- O sistema deve permitir a monitorização contínua dos principais componentes mecânicos dos autocarros através de sensores simulados.
- Devem ser recolhidos e processados dados sobre:
 - Estado do motor, bateria, pressão dos pneus, temperatura interna e consumo energético.
 - Histórico de manutenção com registo de inspeções e planeamento de intervenções preventivas.
 - Deteta automaticamente falhas mecânicas, e gera alertas para intervenção técnica.
- A manutenção deve ser baseada em análises de padrões históricos, permitindo prever falhas antes que ocorram.

- A informação sobre o estado dos veículos será exibida na visualização do TwinTUB, permitindo à equipa gestora atuar rapidamente em caso de avaria.

Compatibilidade com Outros Sistemas

O TwinTUB deve integrar-se com sistemas externos, tais como:

- APIs de mapas, permitindo a simulação da movimentação dos autocarros.
- Soluções de visualização 3D, garantindo a representação correta dos Digital Twins através da integração com Spring Boot e Blender.

A arquitetura deve ser modular e extensível, permitindo futuras integrações com novas tecnologias.

Segurança e Privacidade

A proteção dos dados transmitidos e armazenados deve ser garantida através de:

- Codificação dos dados para impedir acessos não autorizados durante a transmissão.
- Autenticação e controlo de acesso para garantir que apenas utilizadores autorizados podem visualizar ou modificar informações.
- Garantia de privacidade na simulação dos dados, assegurando que a geração e manipulação dos dados PowerShell seguem boas práticas de segurança, evitando exposição indevida de informações.

Com estes requisitos, a arquitetura do TwinTUB assegura desempenho, fiabilidade, segurança e escalabilidade, tornando-se um sistema robusto para a gestão eficiente do transporte público.

9.5 Decisões, Restrições e Justificações

No desenvolvimento do TwinTUB, algumas decisões, restrições e respetivas justificações, relativas à arquitetura do sistema, foram definidas para garantir um sistema eficiente, seguro e escalável.

9.5.1 Decisões

Decisão: Centralizar o processamento analítico na nuvem

Justificação: Para lidar com os grandes volumes de dados gerados pelos sensores IoT, o sistema utilizará a nuvem para processamento de dados em tempo real, armazenamento histórico e análises. Isso permite reduzir a dependência de hardware local, melhorar a eficiência do processamento e simplificar a manutenção.

Decisão: Implementação de um sistema distribuído para garantir baixa latência

Justificação: O TwinTUB exige uma resposta rápida dos gémeos digitais para refletir o estado real dos autocarros, vias e paragens. Para garantir um tempo de resposta inferior a 1 segundo, será utilizada uma arquitetura distribuída que processa dados próximos da fonte (edge computing) antes de enviá-los para a nuvem. Esta abordagem minimiza atrasos e melhora a sincronização em tempo real.

Decisão: Simulação de sensores e geração de dados via PowerShell

Justificação: Como não existem sensores físicos, os dados de localização, lotação dos autocarros e estado mecânico serão simulados através de scripts PowerShell. Isso permite ajustar dinamicamente os dados gerados e testar diferentes cenários operacionais.

Decisão: Integração com APIs de mapas.

Justificação: Para otimizar a gestão da frota e prever tempos de chegada dos autocarros, o TwinTUB será integrado com APIs externas de mapas. Isso permite uma adaptação dinâmica das rotas, melhorando a experiência dos passageiros e aumentando a eficiência do transporte público.

Decisão: Adotar padrões de segurança robustos para proteção contra ciberataques.

Justificação: O sistema, sendo complexo e lidando com a gestão de informação em tempo real, deve ter resistência a ciberataques que possam comprometer a privacidade dos dados e a estabilidade da operação.

9.5.2 Restrições

Restrição: Os gémeos digitais devem ser sincronizados em tempo real

Justificação: A precisão da monitorização dos autocarros, vias e paragens depende de atualizações contínuas e imediatas. O TwinTUB deve garantir que os dados dos sensores IoT simulados sejam processados e refletidos no sistema com um atraso máximo inferior a 1 segundo, permitindo respostas rápidas a avarias, congestionamentos e mudanças nas condições das vias.

Restrição: A interface do sistema deve ser acessível e responsiva

Justificação: O TwinTUB será utilizado por diferentes tipos de utilizadores, incluindo operadores de tráfego e gestores da frota. A interface deve ser intuitiva e responsiva, adaptando-se a diferentes dispositivos (computadores, tablets e smartphones). Isso permitirá uma rápida tomada de decisão e um acompanhamento eficiente da operação do transporte público.

Restrição: O sistema deve suportar um elevado volume de dados sem comprometer o desempenho

Justificação: A arquitetura do TwinTUB deve ser escalável, suportando a recolha e processamento de grandes volumes de dados provenientes dos autocarros, vias e paragens. O sistema deve ser capaz de lidar com até 1000 componentes rodoviários sem degradação do desempenho, garantindo um tempo de resposta eficiente para todas as operações.

9.6 Mecanismos Arquitetónicos

A conceção de uma boa arquitetura para os Gémeos Digitais requer a identificação de mecanismos arquitetónicos que suportem o funcionamento do software, assegurando a escalabilidade, a fiabilidade e a comunicação em tempo real. Estes mecanismos suportam a recolha, processamento e exibição dos dados dos gémeos digitais dos autocarros, vias e paragens.

9.6.1 Mecanismo de Simulação e Aquisição de Dados

Propósito:

Este mecanismo substitui a recolha de dados físicos por simulação via PowerShell, permitindo gerar informações sobre GPS, lotação dos autocarros, condições das vias e estado mecânico.

Entrada:

- Dados simulados via PowerShell: localização, temperatura, pressão dos pneus, energia.
- Dados gerados para representar obstáculos na via e falhas mecânicas.

Saída:

- Envio de dados para o Java (VS Code) para processamento.
- Armazenamento dos dados na Base de Dados.

Atributos:

- **Flexível** – Ajustável para testar diferentes cenários.
- **Fiável** – Gere de uma forma consistente e previsível os dados.
- **Baixa latência** – Permite atualizações em tempo real (<1s).

9.6.2 Mecanismo de Processamento e Normalização de Dados

Propósito:

Este mecanismo garante que os dados recebidos dos sensores simulados sejam processados, estruturados e armazenados corretamente para posterior análise e exibição.

Entrada:

- Dados simulados do PowerShell e API/CMD.
- Dados enviados pelo NIOP para integração com a gestão dos gémeos digitais.

Saída:

- Dados normalizados enviados para a Base de Dados.
- Informação estruturada enviada para Spring Boot para gestão dos gémeos digitais.

Atributos:

- **Escalável** – Suporta grandes volumes de dados.

- **Consistente** – Garante integridade e estrutura uniforme dos dados.
- **Segurança** – Proteção contra manipulação indevida da informação.

9.6.3 Mecanismo de Comunicação e Integração de Sistemas

Propósito:

Responsável por garantir a transmissão de dados entre os diferentes componentes, assegurando que os gémeos digitais estejam sempre atualizados.

Entrada:

- Dados do Java (VS Code) e Base de Dados.
- Dados operacionais enviados pelo NIOP.

Saída:

- Dados enviados para o Spring Boot para gestão dos modelos 3D.
- Atualização dos gémeos digitais e envio para a Interface Digital Twin.

Atributos:

- **Baixa latência** – Resposta rápida para sincronização dos gémeos digitais.
- **Interoperabilidade** – Suporte a diferentes tecnologias e protocolos.
- **Fiabilidade** – Garantia de entrega e integridade dos dados.

9.6.4 Mecanismo de Gestão dos Gémeos Digitais

Propósito:

Este mecanismo gere os gémeos digitais e modelos 3D para que representem fielmente a frota de autocarros e as vias, permitindo monitorização e simulação realista.

Entrada:

- Modelos 3D gerados pelo Blender.
- Dados operacionais do Spring Boot sobre os autocarros e vias.

Saída:

- Representação dos gémeos digitais na Interface Digital Twin.
- Atualização da representação visual conforme os dados operacionais.

Atributos:

- **Precisão** – Modelos 3D alinhados com os dados reais.
- **Escalável** – Suporte a múltiplos gémeos digitais simultaneamente.
- **Integração** – Comunicação com Blender e Spring Boot.

9.6.5 Mecanismo da Interface de visualização do Gémeo Digital

Propósito:

Este mecanismo fornece uma interface gráfica intuitiva para que os operadores consigam monitorizar o estado dos autocarros, vias e paragens. A plataforma permite visualizar dados em tempo real, analisar alertas de avarias e consultar previsões de tráfego, facilitando a gestão operacional do transporte público.

Entrada:

- Dados processados pelo backend (Spring Boot).
- Modelos 3D gerados pelo Blender.

Saída:

- Informação exibida em dashboards e mapas interativos.

Atributos:

- **Interativo** – Permite manipulação dos modelos digitais.
- **Tempo real** – Atualizações contínuas conforme os dados recebidos.

9.6.6 Mecanismo Gestão de Eventos e Alertas Automáticos

Propósito:

Este mecanismo garante que eventos críticos, como avarias mecânicas, obstáculos ou falhas nos sensores, sejam automaticamente detetados e reportados.

Entrada:

- Dados dos sensores e sistema de monitorização.
- Base de Dados com histórico de falhas e alertas.

Saída:

- Notificações enviadas para os operadores através da interface.

Atributos:

- **Automatização** – Identificação imediata de falhas.
- **Priorização** – Diferenciação entre falhas críticas e secundárias.
- **Segurança** – Garante integridade dos eventos reportados.

9.7 Abstrações Chave

Dados dos Sensores: Representação dos dados simulados gerados por fontes externas, como PowerShell, que imitam dados de sensores IoT (GPS, temperatura, pressão, contagem de passageiros, etc.) para simular as condições dos autocarros, vias e paragens.

Gémeo Digital: Criação e gestão da representação virtual dos autocarros, vias e paragens, permitindo sincronização em tempo real entre os dados e o modelo digital.

Gestão e Armazenamento de Dados: Componente responsável por armazenar grandes volumes de dados recolhidos dos autocarros, vias e paragens, garantindo acesso eficiente e seguro em tempo real.

Mecanismo de Comunicação e Integração de Sistemas: Responsável por assegurar a comunicação contínua e eficaz entre os dados simulados, os gémeos digitais, o back-end e a interface de visualização. Este mecanismo é essencial para garantir que todos os componentes do sistema funcionem de forma integrada e sincronizada.

Interface de Visualização: Interfaces de utilizador que permitem uma monitorização em tempo real, visualização de modelos e análise dos resultados.

Gestão de Eventos e Alertas: Sistema que deteta e notifica eventos como avarias nos autocarros, congestionamentos e incidentes nas vias.

9.8 Camadas Arquitetónicas

O sistema será desenvolvido com base em um framework arquitetural em camadas, estruturado para garantir a modularidade, escalabilidade e consistência. Essa abordagem organiza os componentes em camadas funcionais interdependentes, cada uma com responsabilidades claras e interfaces bem definidas.

9.8.1 Camadas e Componentes:

9.8.1.1 Camada de Aquisição de Dados

Descrição:

Esta camada é responsável por captar dados das fontes externas, como sensores simulados e dados operacionais. Ela garante que as informações iniciais sejam recebidas e enviadas ao sistema para processamento posterior.

Componentes Principais:

- NIOP: Plataforma low-code para integrar dados operacionais de sensores.
- PowerShell (Simulação): Gere dados simulados para testar e monitorizar os dados do sistema.
- API/CMD: Interface de comunicação para transmissão de dados para o back-end.

Papel no Sistema:

Captar dados de diversas fontes e prepará-los para o processamento e armazenamento.

9.8.1.2 Camada de Processamento de Dados

Descrição:

Responsável pelo processamento, normalização e armazenamento dos dados recebidos. Ele

assegura que os dados brutos sejam transformados em informações estruturadas e acessíveis para as camadas subsequentes.

Componentes Principais:

- Mecanismo de Processamento e Normalização de Dados: Organiza os dados recebidos e os converte em formato estruturado.
- Base de Dados: Armazena os dados estruturados e históricos para consultas futuras e análise de falhas.

Papel no Sistema:

Garantir que os dados sejam estruturados, normalizados e armazenados corretamente para facilitar o acesso, análise e utilização posterior.

9.8.1.3 Camada Gestão de Sistemas e Integração

Descrição:

Esta camada tem a função de integrar diferentes sistemas e garantir a comunicação entre eles, permitindo que o sistema como um todo funcione de forma fluida e coordenada.

Componentes Principais:

- Mecanismo de Comunicação e Integração de Sistemas: Facilita a comunicação entre a camada de processamento de dados, a Base de Dados e os gémeos digitais.
- Mecanismo de Gestão de Eventos e Alertas: Identifica falhas ou anomalias no sistema e emite alertas em tempo real para os operadores.

Papel no Sistema:

Assegura a integração contínua dos sistemas, garantindo que os dados dos sensores sejam comunicados, processados e refletidos nas representações digitais em tempo real.

9.8.1.4 Camada de Gestão de Gémeos Digitais

Descrição:

Esta camada é responsável pela gestão e atualização dos modelos 3D dos gémeos digitais, garantindo que as representações digitais dos autocarros, vias e paragens estejam em sincronia com os dados reais.

Componentes Principais:

- Modelos 3D (Blender): Geração dos modelos tridimensionais dos gémeos digitais.
- Spring Boot: Gere os dados operacionais e processa a interação com os modelos 3D.
- Mecanismo de Gestão dos Gémeos Digitais: Sincroniza os modelos 3D com os dados reais para garantir que as representações digitais sejam precisas.

Papel no Sistema:

Representa os dados físicos (autocarros, vias, paragens) em modelos digitais, permitindo que os operadores interajam com a versão virtual do sistema.

9.8.1.5 Camada de Visualização e Interface de Alerta

Descrição:

Esta camada permite a visualização das informações sobre o estado da frota e das vias, bem como fornece uma interface interativa para os operadores visualizarem e tomarem decisões baseadas em dados em tempo real.

Componentes Principais:

- Mecanismo de Interface de Gémeo Digital: Exibe as informações de estado operacional e falhas dos gémeos digitais através de *dashboards* e mapas interativos.
- Operadores: Usuários que interagem com a interface para tomar decisões operacionais.

Papel no Sistema:

Proporciona uma visualização intuitiva e interativa dos gémeos digitais, permitindo que os operadores realizem monitoramento, gestão e tomada de decisões em tempo real.

9.8.2 Consistência e Uniformidade

Separação de Preocupações: Cada camada do TwinTUB é independente e responsável por uma função específica, garantindo que a recolha de dados, comunicação, processamento, análise e visualização funcionem de forma modular e sem interferências diretas entre si.

Interfaces Padronizadas: A comunicação entre as camadas ocorre através de APIs bem documentadas, permitindo a integração eficiente entre sensores, bases de dados, módulos de análise e a interface de monitorização.

Escalabilidade: A arquitetura do TwinTUB foi projetada para permitir a adição de novos sensores, algoritmos de machine learning e funcionalidades sem impactar negativamente as camadas existentes, garantindo flexibilidade para evolução futura.

9.8.3 Benefícios

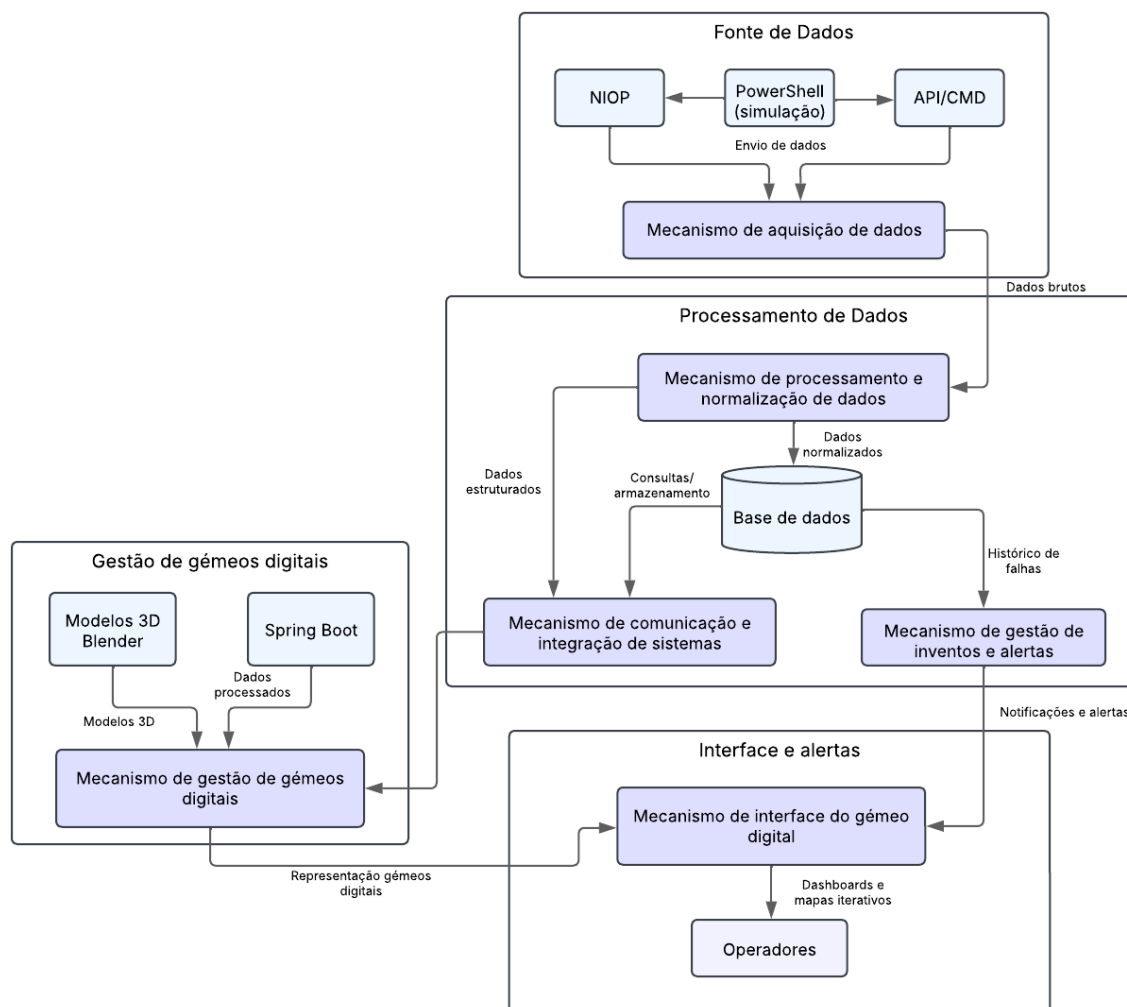
Manutenção Simplificada: Alterações em uma camada não afetam diretamente as outras, permitindo atualizações e correções sem comprometer a estabilidade do sistema.

Modularidade: Cada componente do sistema (sensores, comunicação, processamento, análise e interface) pode ser desenvolvido, testado e atualizado de forma independente, melhorando a confiabilidade.

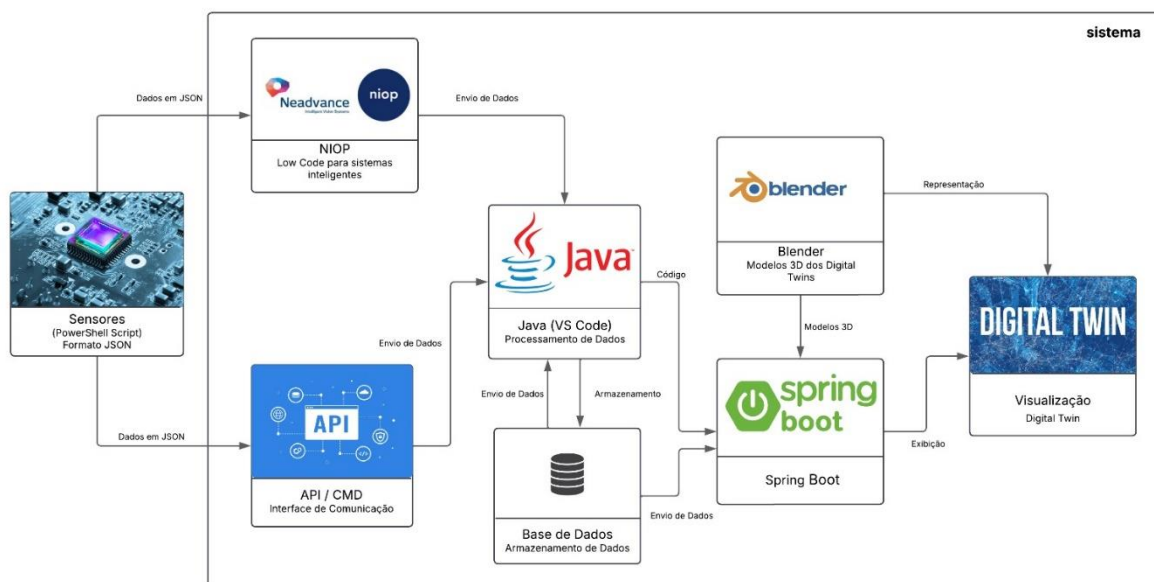
Eficiência Operacional: O processamento em tempo real dos dados recolhidos pelos sensores e as análises permitem uma resposta rápida a obstáculos na via e falhas mecânicas, otimizando a gestão da frota de transporte público.

9.9 Visão Arquitetónica

9.9.1 Diagrama de mecanismos e camadas



9.9.2 Diagrama dos diferentes componentes do sistema



10. Modelo de Casos de Uso

10.1 Introdução

Os modelos de caso de uso são representações gráficas e textuais que descrevem as interações entre os atores e um sistema, sendo fundamentais para a especificação de requisitos funcionais. Esses modelos incluem diagramas de caso de uso, que ilustram visualmente os cenários de uso, e especificações detalhadas, que descrevem o fluxo de eventos, pré-condições, pós-condições e requisitos especiais.

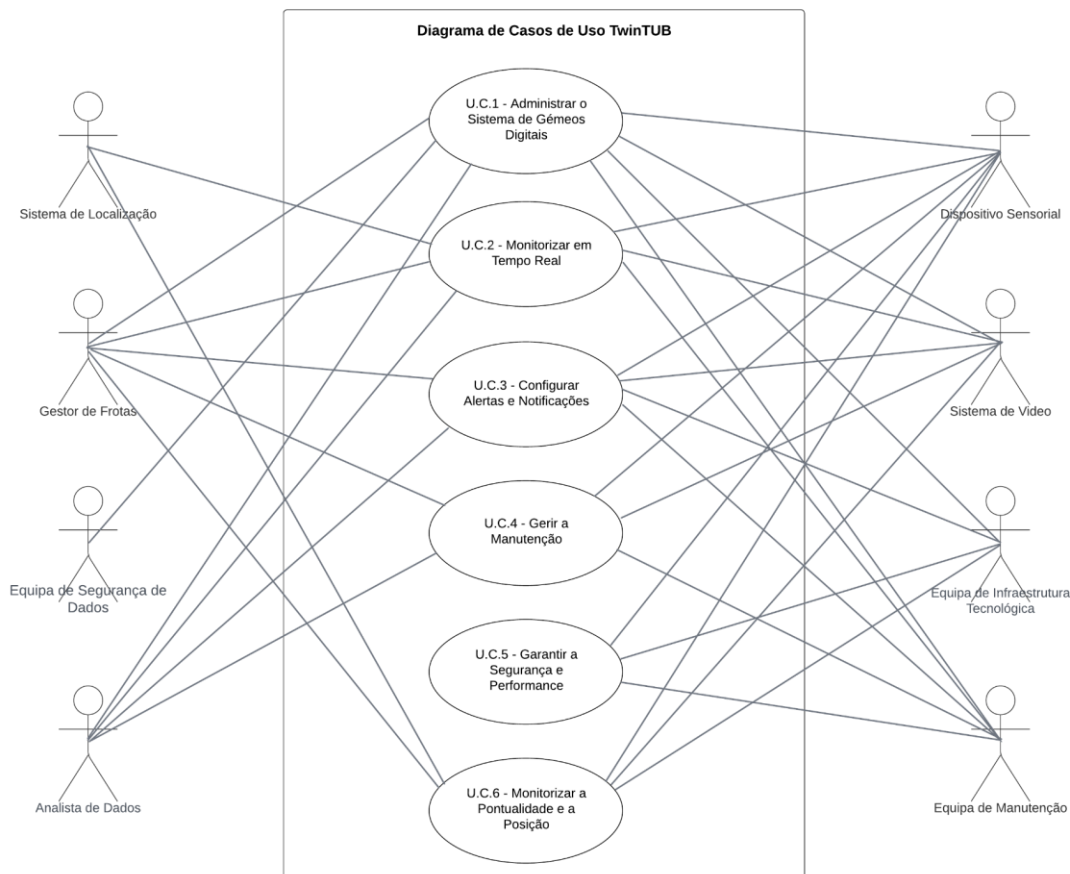
Este trabalho apresenta modelos de caso de uso acompanhados das suas respectivas especificações, visando documentar de forma clara o comportamento do sistema e funcionalidades.

10.2 Catálogo de Atores

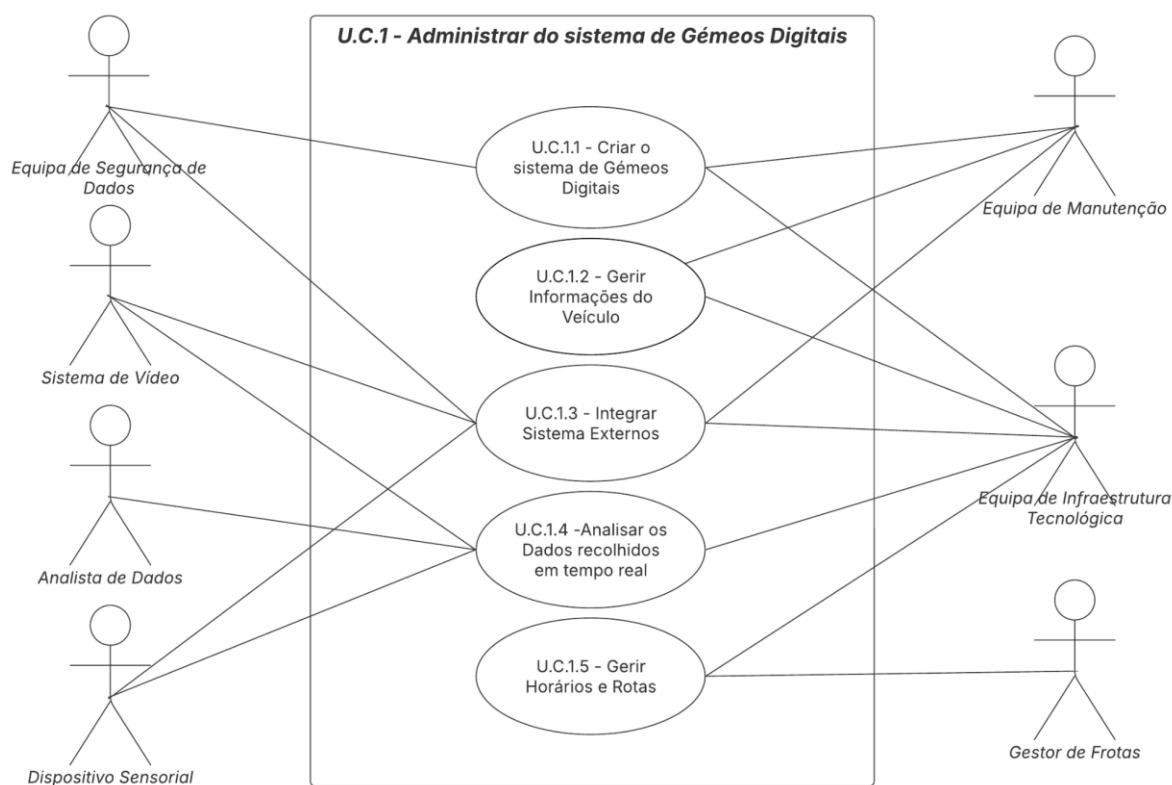
Atores	Descrição
Dispositivo Sensorial	Responsável por captar e fornecer informações em tempo real, como a temperatura interna dos veículos, permitindo o monitoramento contínuo.
Sistema de Vídeo	Responsável por captar e fornecer informações em tempo real, como as condições da via, permitindo o monitoramento contínuo.

Equipa de Infraestrutura Tecnológica	Responsável pela configuração, monitorização e manutenção do sistema informático, garantindo o seu funcionamento contínuo e sem falhas.
Equipa de Manutenção	Responsável por garantir o perfeito estado de funcionamento dos veículos e dos sistemas operacionais, realizando manutenção preventiva e reparações quando necessário.
Sistema de Localização	Aplicação integrada no Gémeo Digital que permite acompanhar a localização das viaturas em tempo real, fornecendo dados atualizados sobre o posicionamento dos veículos.
Gestor de Frotas	Tem como função otimizar os percursos e horários das viaturas, tendo em conta as necessidades dos passageiros e a eficiência operacional, visando sempre a melhor experiência para os utilizadores.
Equipa de Segurança de Dados	Responsável por proteger os dados, garantindo a sua integridade, confidencialidade e segurança durante a recolha, transmissão e armazenamento, implementando medidas como criptografia e monitorização contínua contra acessos não autorizados.
Analista de Dados	Responsável por analisar os dados recolhidos em tempo real, gerar insights e realizar simulações para apoiar a tomada de decisões, utilizando ferramentas analíticas e técnicas de visualização para otimizar operações e prever tendências.

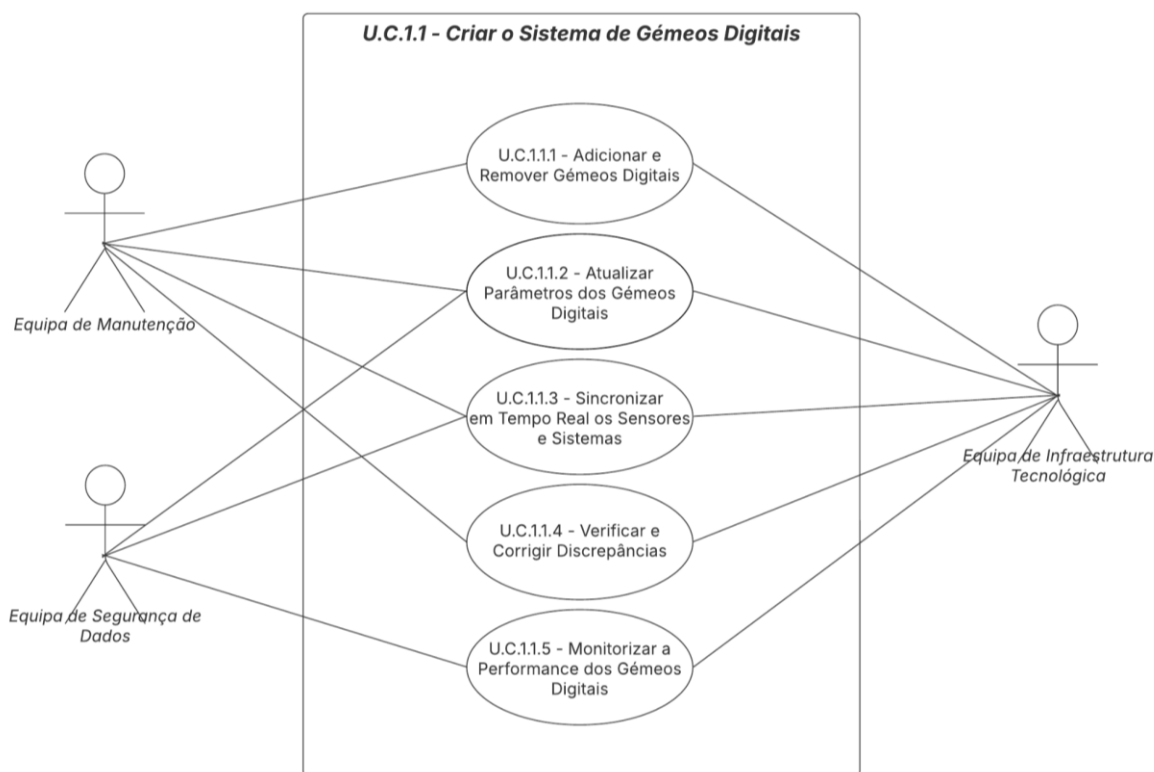
Caso de Uso TwinTUB



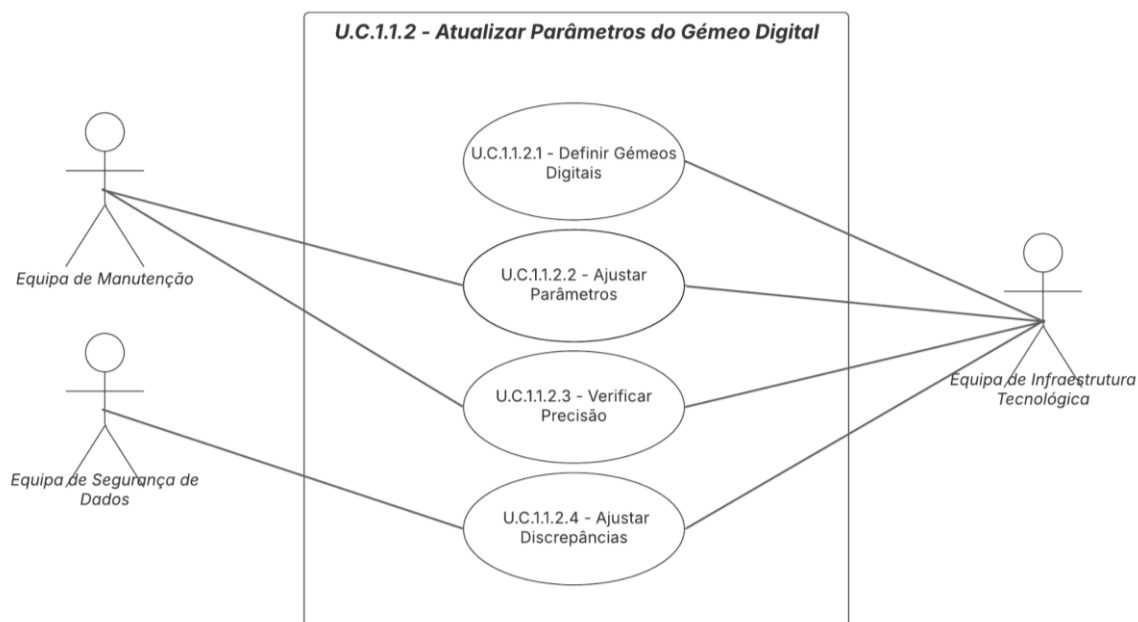
1. Administrar o sistema de Gémeos Digitais



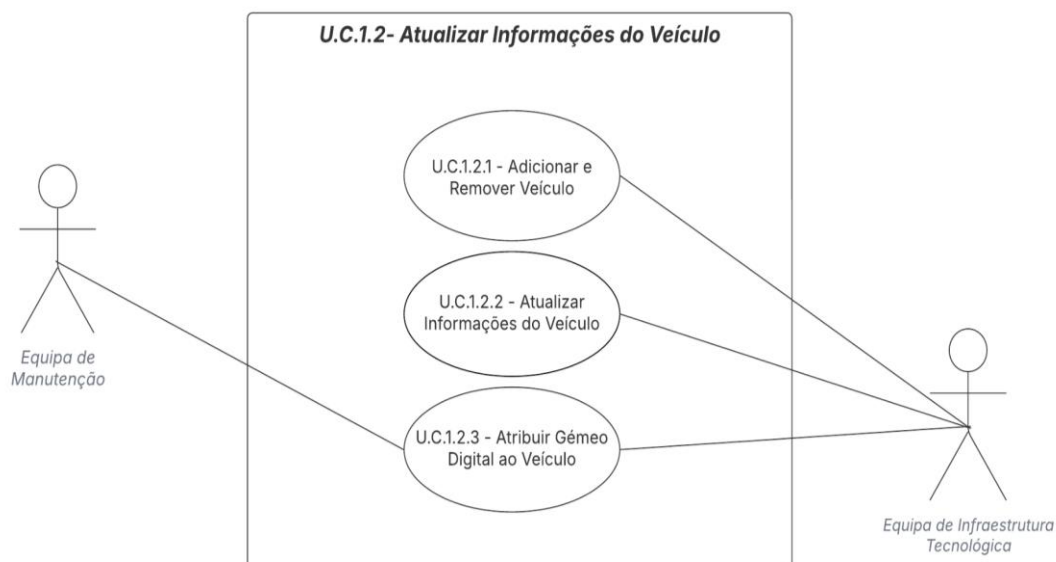
U.C.1.1 Criar o sistema de Gémeos Digitais



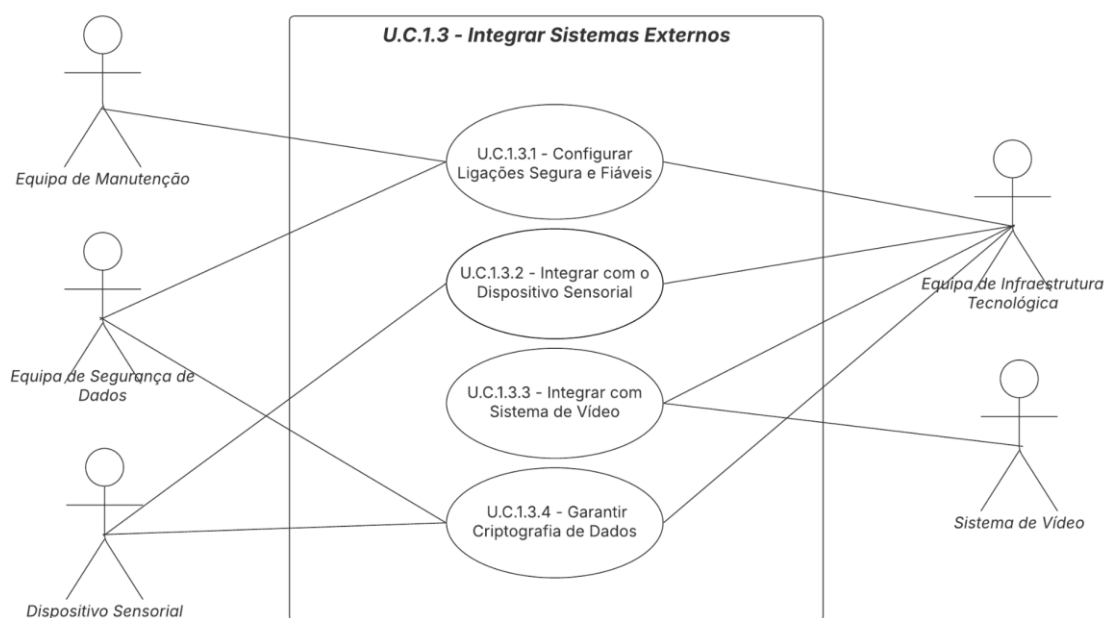
U.C.1.1.2 Atualizar Parâmetros do Gémeo Digital



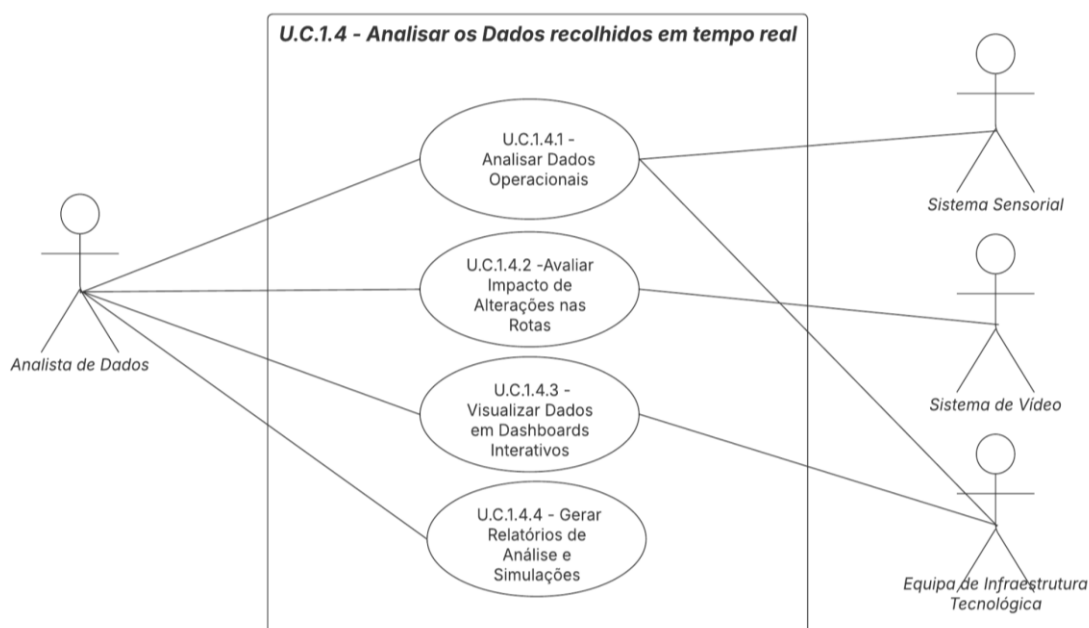
U.C.1.2 Atualizar Informações do Veículo



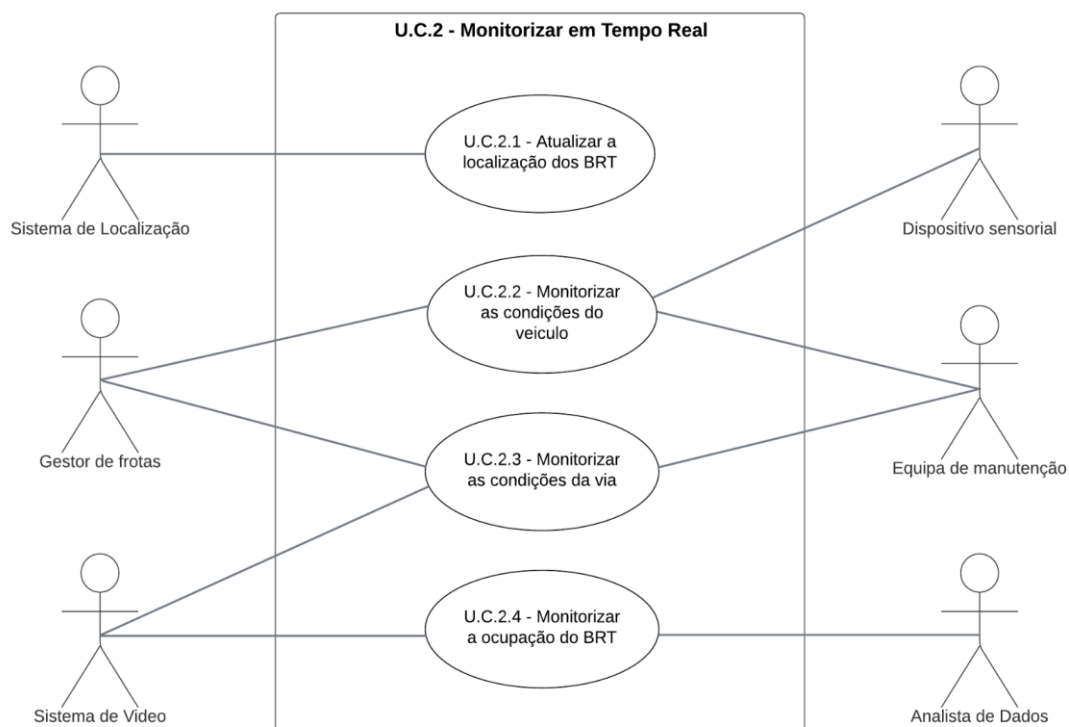
U.C.1.3 Integrar Sistemas Externos



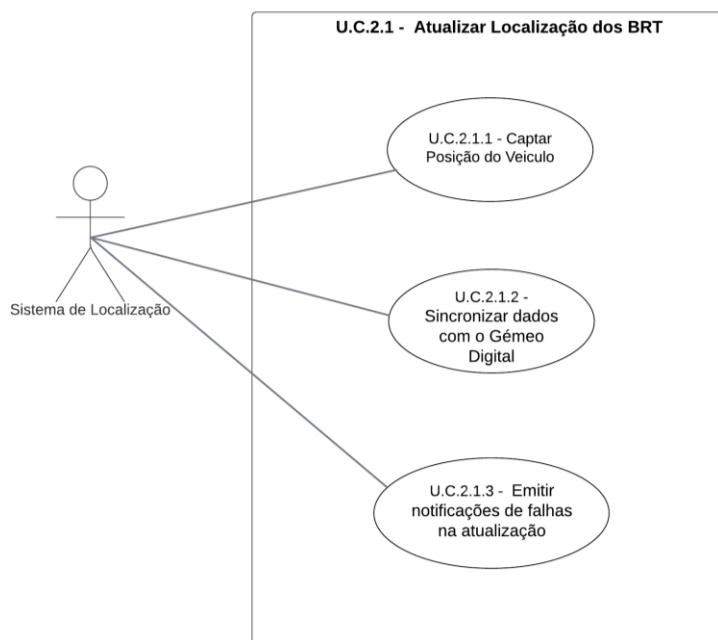
U.C.1.4 Analisar Dados recolhidos em tempo real



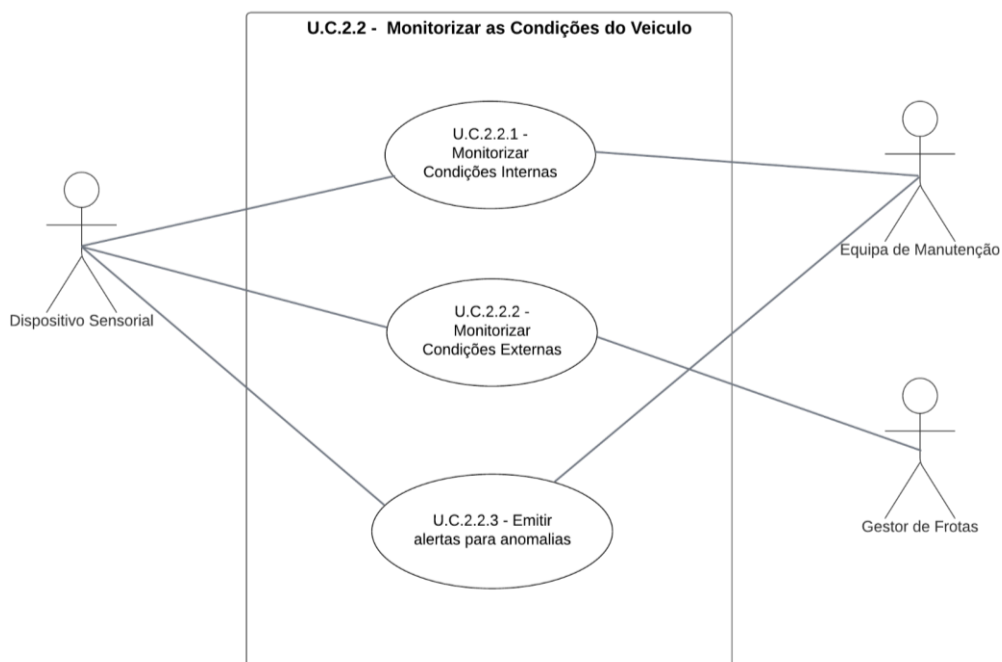
2. Monitorizar em Tempo Real



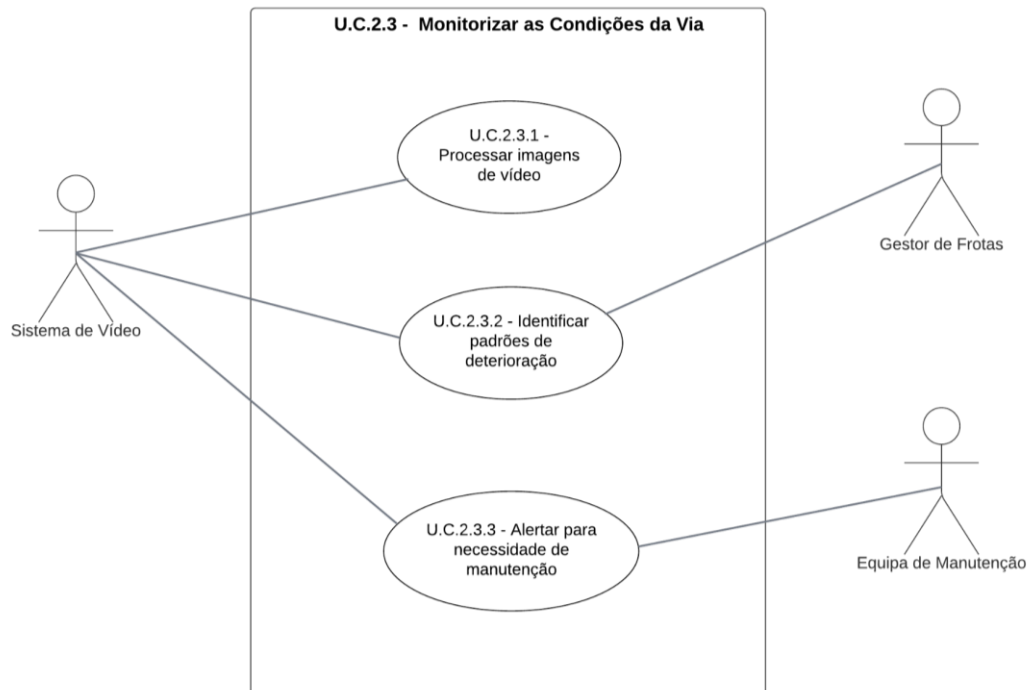
U.C.2.1 Atualizar a localização dos BRT



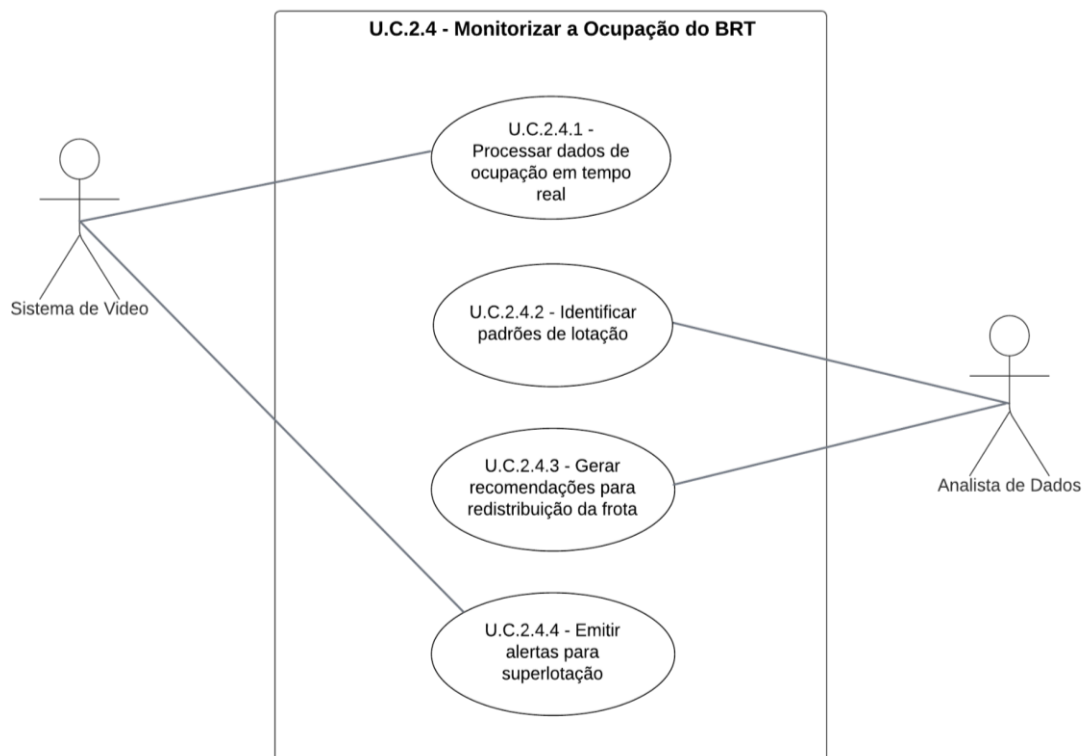
U.C.2.2 Monitorizar as Condições do Veículo



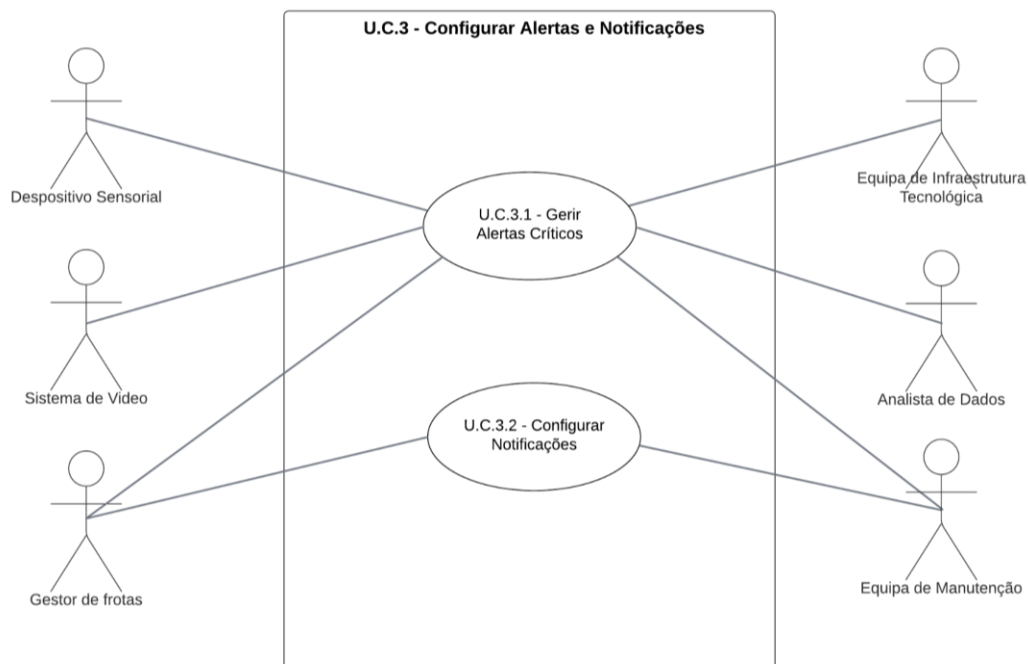
U.C.2.3 Monitorizar as Condições da Via



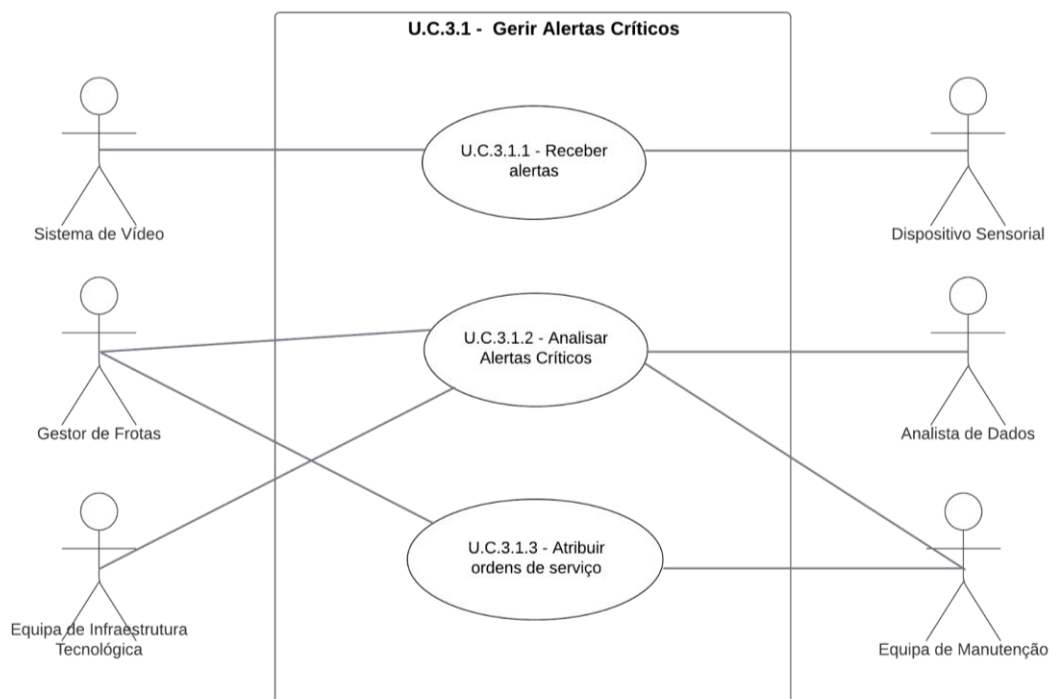
U.C.2.4 Monitorizar Ocupação do BRT



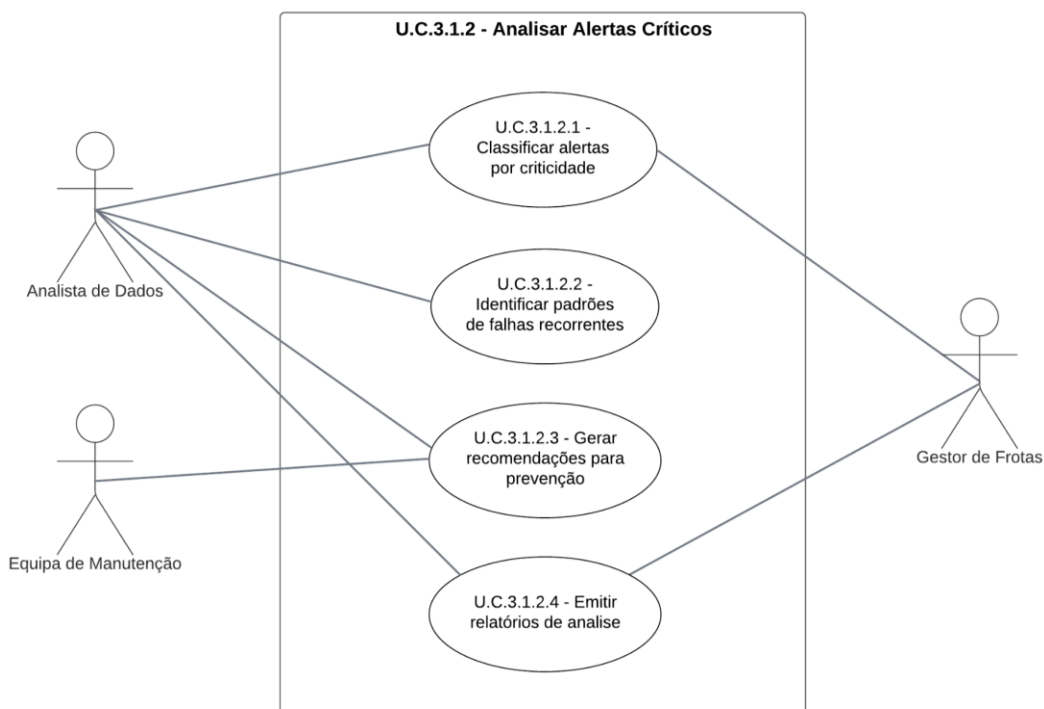
3. Configurar Alerta e Notificações



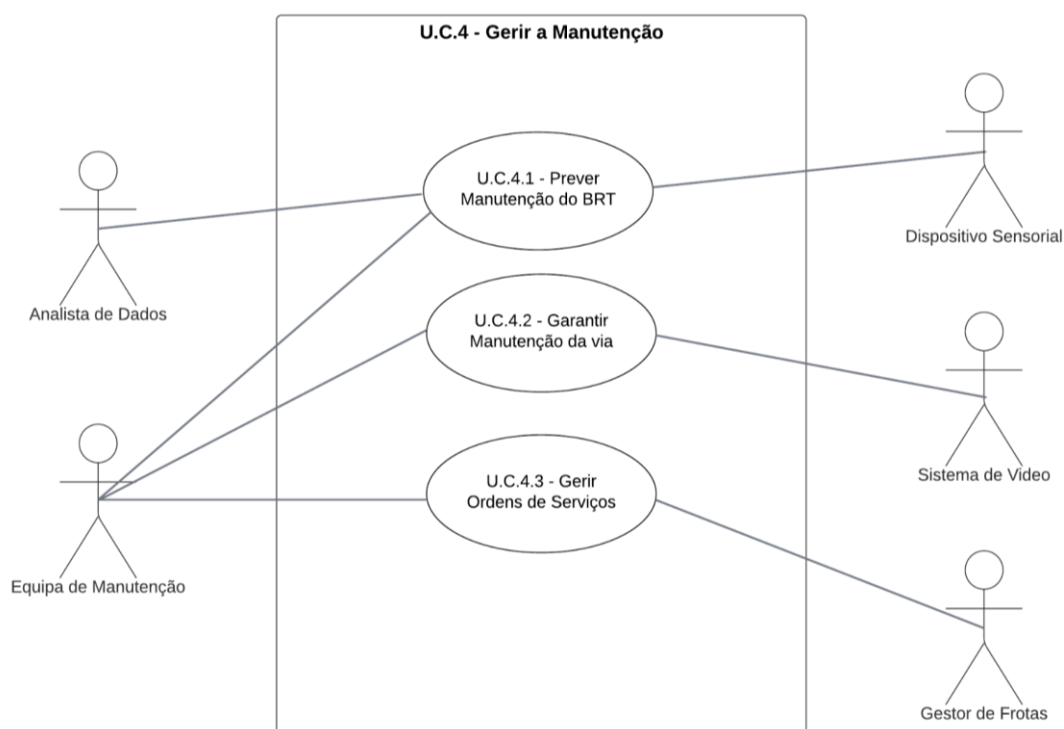
U.C.3.1 Gerir Alertas Críticos



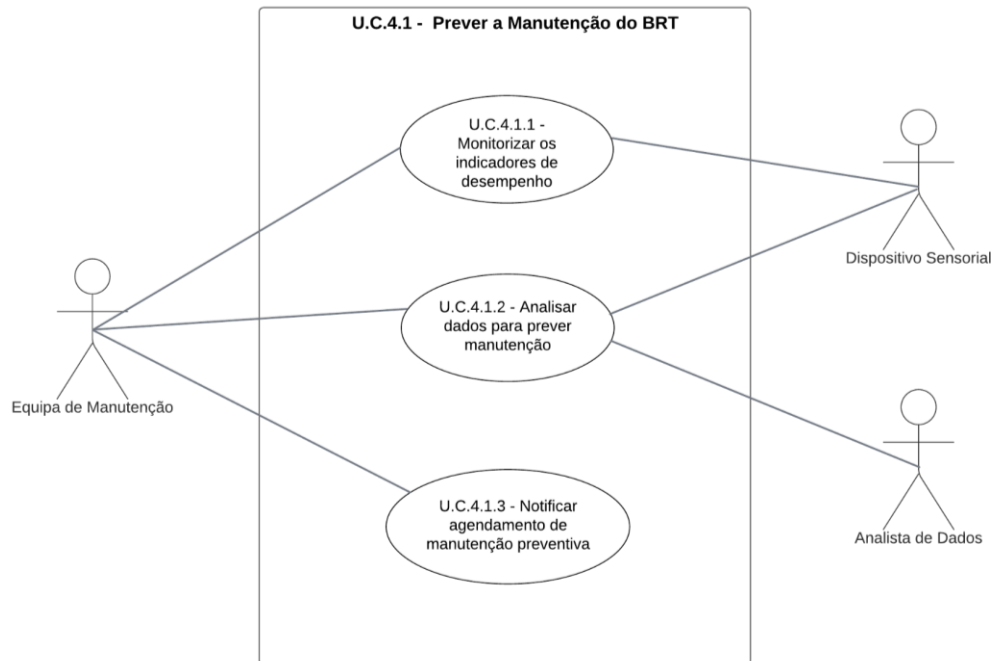
U.C.3.1.2 Analisar Alertas Críticos



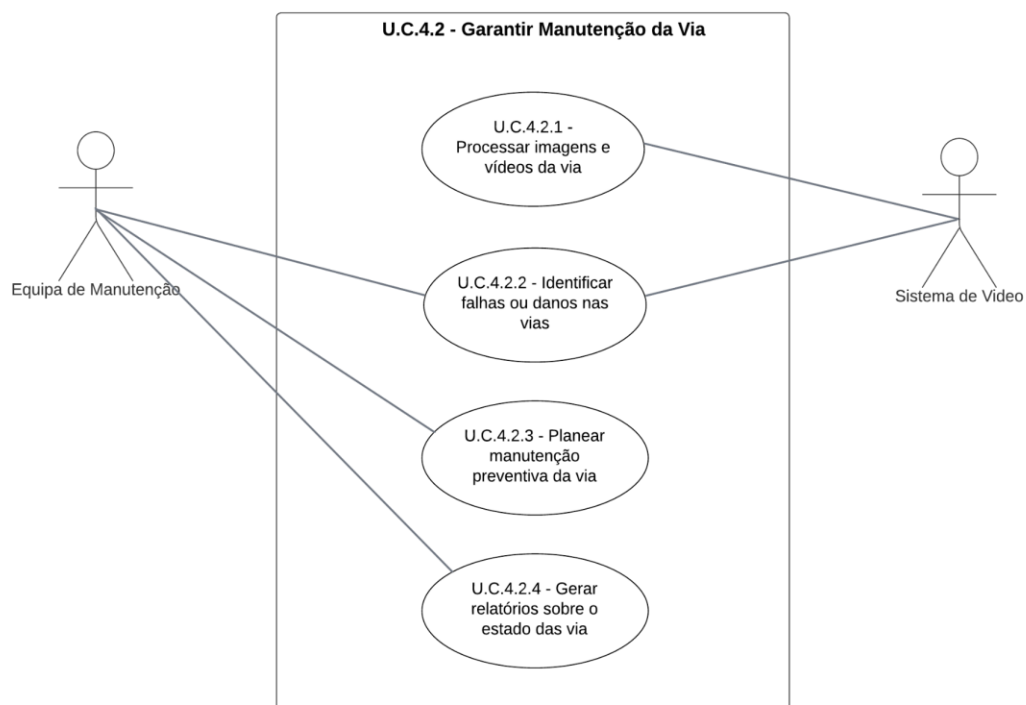
4. Gerir a Manutenção



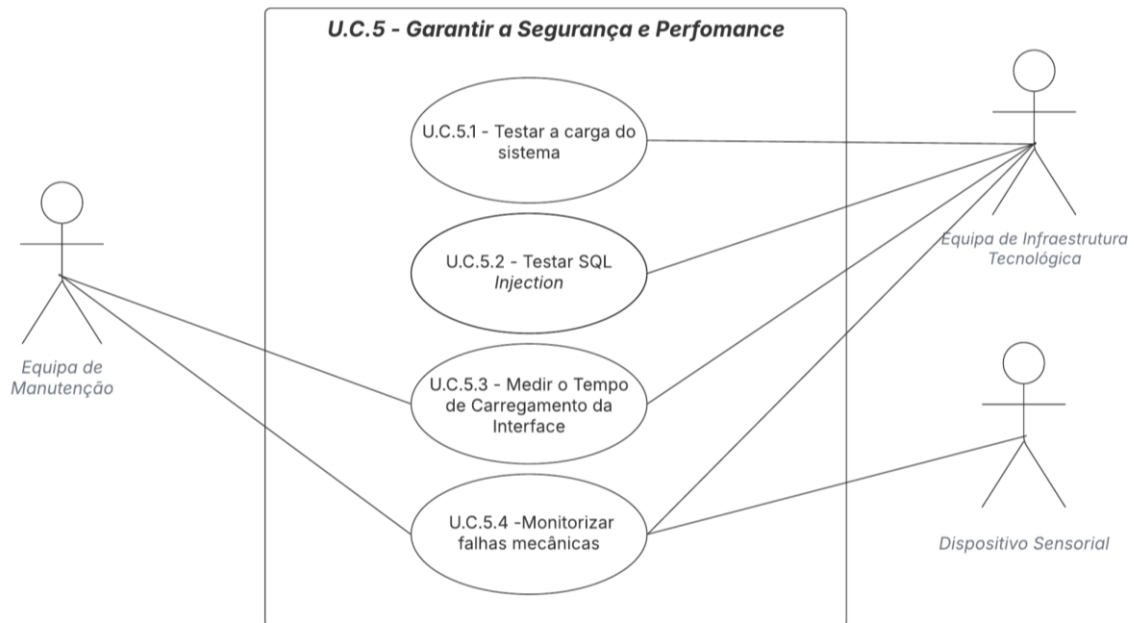
U.C.4.1 Prever a Manutenção do BRT



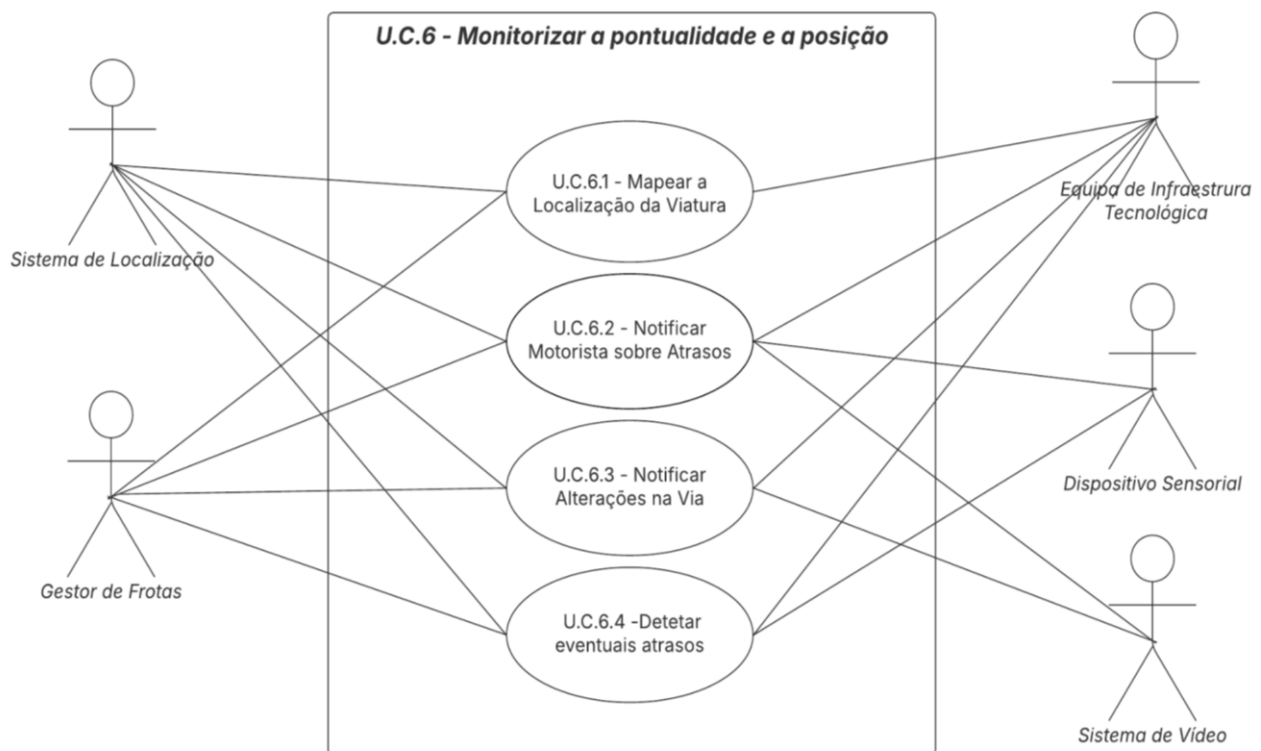
U.C.4.2 Garantir a Manutenção da Via



5. Garantir a Segurança e Performance



6. Monitorizar a pontualidade e a posição



11. Especificação dos Casos de Uso

U.C.1

1. **Nome do caso de uso:** Administrar o sistema de Gémeos Digitais
2. **Descrição do caso de uso:** Este caso de uso permite gerir Gémeos Digitais, recolher e analisar dados em tempo real para otimizar a segurança e a eficiência.
3. **Atores que interagem com o caso de uso:** Equipa de Segurança de Dados, Sistema de Vídeo, Analista de Dados, Dispositivo Sensorial, Equipa de Manutenção, Equipa de Infraestrutura Tecnológica e Gestor de Frotas.
4. **Pré-condições:**
 - Os dispositivos sensoriais e sistemas externos estão configurados e operacionais.
 - As equipas têm acesso aos dados em tempo real e podem gerir informações atualizadas.
 - A infraestrutura tecnológica está funcional e configurada para a monitorização e análise contínua.
5. **Pós-condições:**

Conclusão bem-sucedida:

 - As intervenções realizadas para gerir as informações e o sistema foram bem-sucedidas.
 - Os ajustes nos horários e rotas foram realizados com base na análise de dados.
 - As informações recolhidas foram integradas e analisadas de forma eficaz.

Condição de falha:

 - Os registos do sistema são atualizados com os motivos de falhas.
6. **Fluxo de eventos :**

Fluxo Normal:

 - O caso de uso começa quando os dispositivos sensoriais e sistemas externos são conectados ao sistema;
 - As equipas fazem login utilizando credenciais válidas;
 - Os dispositivos sensoriais começam a transmitir dados em tempo real para o sistema;
 - O sistema recolhe, processa e organiza os dados transmitidos;
 - As equipas acedem e analisam os dados;
 - Com os dados recebidos são realizados ajustes;
 - O caso de uso termina com sucesso quando o sistema está operacional e os dados são analisados corretamente.

Fluxo Alternativo:

Interrupção no sistema de processamento:

- O sistema deteta uma anomalia que impede o processamento de dados;
- Um alerta é enviado para a Equipa de Infraestrutura Tecnológica, que analisa e resolve o problema;
- Após a correção o sistema processa os dados e retorna ao estado normal.

7. Requisitos Especiais:

- Garantir a compatibilidade do sistema com os dispositivos e câmaras de diferentes fabricantes;
- Implementar mecanismos de segurança fortes para proteger os dados em tempo real;
- Garantir redundância para evitar interrupções nas operações críticas.

U.C.1.1**1. Nome do caso de uso:** Criar o sistema de Gémeos Digitais**2. Descrição do caso de uso:** Este caso de uso descreve os passos e processos para criar o sistema de Gémeos Digitais.**3. Atores que interagem com o caso de uso:** Equipa de Manutenção, Equipa de Segurança de Dados e Equipa de Infraestrutura Tecnológica.**4. Pré-condições:**

- O ambiente de desenvolvimento do sistema de Gémeos Digitais está configurado;
- Tanto os Gémeos Digitais como os requisitos foram definidos pelas equipas de projeto;
- Existem modelos predefinidos para a criação do sistema.

5. Pós-condições:**Conclusão bem-sucedida:**

- O sistema encontra-se configurado e com capacidade para recolher e organizar dados em tempo real;
- As discrepâncias identificadas entre os dados reais e os Gémeos Digitais foram corrigidas.

Condição de falha:

- O sistema não foi criado com sucesso e foram fornecidas recomendações para corrigir falhas.

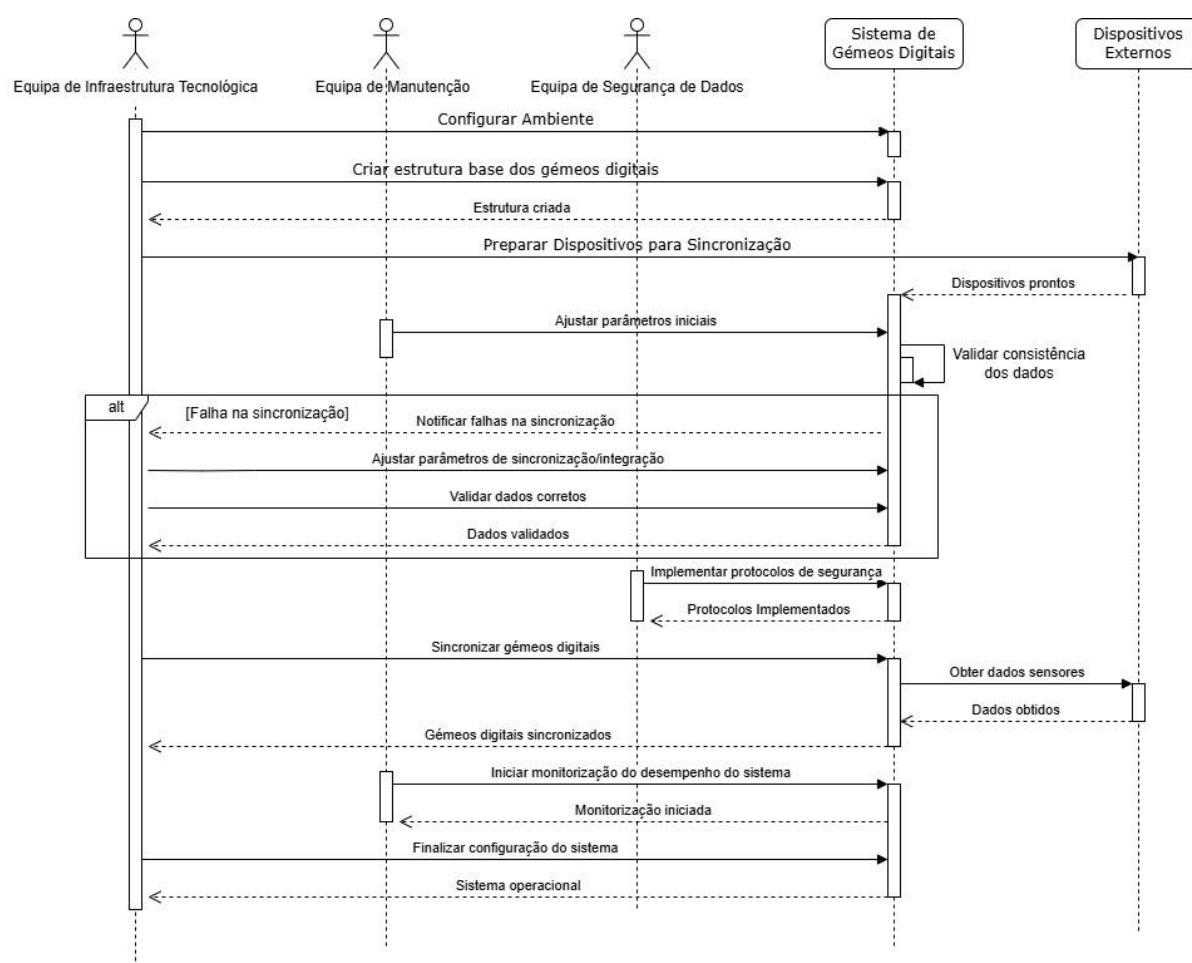
6. Fluxo de eventos:**Fluxo Normal:**

- O caso de uso começa com a configuração do ambiente de desenvolvimento dos Gémeos Digitais;
- A Equipa de Infraestrutura Tecnológica inicia a criação da estrutura básica dos Gémeos Digitais;
- Os dispositivos e sistemas externos são preparados para sincronização.
- A Equipa de Manutenção realiza os ajustes nos parâmetros iniciais dos Gémeos Digitais;
- O sistema executa uma validação para garantir a consistência dos dados integrados;
- O caso de uso termina com sucesso quando os Gémeos Digitais estão configurados e sincronizados corretamente.

Fluxo Alternativo:

Falha na sincronização inicial de dados:

- Durante a sincronização o sistema deteta falhas na integração de um ou mais sensores e dispositivos externos:
- Um alerta é enviado à Equipa de Infraestrutura Tecnológica para identificar e corrigir o problema;
- A equipa ajusta os parâmetros de integração e valida os dados corretos.



7. Requisitos Especiais:

- O sistema deve ser capaz de processar grandes volumes de dados em tempo real sem comprometer a performance.

U.C.1.1.2

1. **Nome do caso de uso:** Atualizar Parâmetros do Gémeo Digital
2. **Descrição do caso de uso:** Este caso de uso descreve os passos para ajustar e validar parâmetros dos Gémeos Digitais para refletir condições reais e otimizar operações.
3. **Atores que interagem com o caso de uso:** Equipa de Manutenção, Equipa de Segurança de Dados e Equipa de Infraestrutura Tecnológica.
4. **Pré-condições:**
 - O sistema de Gémeos Digitais está em pleno funcionamento.
 - As ferramentas de análise para validação estão disponíveis.
5. **Pós-condições:**

Conclusão bem-sucedida:

 - Os ajustes nos parâmetros foram realizados com sucesso, garantindo concordância com condições reais.
 - A precisão e integridade do sistema foram mantidas após a atualização.

Condição de falha:

 - As falhas foram identificadas e encaminhadas com recomendações para correção.
6. **Fluxo de eventos**

Fluxo Normal:

 - O caso de uso inicia com a análise de necessidades de ajustes nos parâmetros pela Equipa de Infraestrutura Tecnológica.
 - Os dados dos dispositivos sensoriais são integrados ao sistema para identificar desajustes.
 - A Equipa de Segurança de Dados verifica se os ajustes não comprometem a integridade das operações.
 - A atualização dos parâmetros é realizada no sistema pelos Analistas de Dados.
 - O sistema executa uma validação para verificar a consistência dos ajustes.
 - O caso de uso finaliza com os parâmetros atualizados e validados em relatórios.

Fluxo Alternativo:

Em caso de falhas durante o ajuste de parâmetros:

 - Um alerta é emitido para as equipas responsáveis.

- A Equipa de Infraestrutura Tecnológica investiga e resolve os problemas identificados.

7. **Requisitos Especiais:**

- O sistema deve permitir a análise e validação em tempo real sem impactar o desempenho.

U.C.1.2

1. **Nome do caso de uso:** Atualizar Informações do Veículo

2. **Descrição do caso de uso:** Este caso de uso permite atualizar e validar as informações dos veículos para garantir precisão e alinhamento com as operações.

3. **Atores que interagem com o caso de uso:** Equipa de Manutenção e Equipa de Infraestrutura Tecnológica.

4. **Pré-condições:**

- O sistema de gestão de veículos está operacional.
- As equipas têm acesso às permissões e ferramentas necessárias.
- Todos os dispositivos de recolha de dados estão configurados e conectados.

5. **Pós-condições:**

Conclusão bem-sucedida:

- As informações dos veículos foram atualizadas corretamente no sistema.
- Os registos obsoletos foram eliminados e os novos dados foram integrados.

Condição de falha:

- As alterações não foram realizadas devido a falhas técnicas.

6. **Fluxo de eventos**

Fluxo Normal:

- O caso de uso começa com a Equipa de Manutenção a aceder ao sistema e verificar as informações existentes.
- A Equipa de Infraestrutura Tecnológica prepara o ambiente para a atualização dos dados.
- Informações como o estado do veículo e disponibilidade são introduzidas no sistema.
- Dados redundantes ou incorretos são eliminados.
- Um processo de validação é realizado para garantir a consistência das atualizações.
- O caso de uso termina com os dados atualizados e validados.

Fluxo Alternativo:

Se ocorrerem falhas na atualização:

- Um alerta é enviado para a Equipa de Infraestrutura Tecnológica para resolução do problema.
- Os dados são revisados e o processo é reiniciado.

7. Requisitos Especiais:

- O sistema deve garantir a integridade dos dados durante as atualizações.

U.C.1.3

1. Nome do caso de uso: Integrar Sistemas Externos

2. Descrição do caso de uso: Este caso de uso aborda a integração de sistemas externos para garantir uma comunicação eficiente, segura e confiável entre diferentes componentes operacionais.

3. Atores que interagem com o caso de uso: Equipa de Manutenção, Equipa de Segurança de Dados, Dispositivo Sensorial, Equipa de Infraestrutura Tecnológica e Sistema de Vídeo.

4. Pré-condições:

- Todos os sistemas externos estão disponíveis para integração.
- A Infraestrutura Tecnológica está funcional e configurada para suportar o processo de integração.

5. Pós-condições:

Conclusão bem-sucedida:

- Os sistemas externos foram integrados com sucesso.
- Os dados estão disponíveis e prontos para utilização nos sistemas internos.

Condição de falha:

- O processo de integração falhou devido a erros técnicos ou de compatibilidade.

6. Fluxo de eventos

Fluxo Normal:

- A Equipa de Infraestrutura Tecnológica verifica a conectividade dos sistemas externos.
- A Equipa de Segurança de Dados configura as medidas de criptografia e segurança.
- A integração com o sistema de vídeo e dispositivos sensoriais é realizada.
- O processo é finalizado após uma validação geral da comunicação entre sistemas.

Fluxo Alternativo:

Se ocorrerem falhas na integração:

- Um alerta é enviado para a Equipa de Segurança de Dados e Infraestrutura Tecnológica.
- As configurações são revisadas para corrigir os problemas encontrados.
- O processo de integração é reiniciado após as correções.

7. Requisitos Especiais:

- Os sistemas devem garantir a criptografia de dados durante a transferência.
- A comunicação entre sistemas deve ser continuamente monitorada para evitar falhas.

U.C.1.4

1. Nome do caso de uso: Analisar Dados recolhidos em tempo real

2. Descrição do caso de uso: Este caso de uso permite recolher, interpretar e apresentar dados em tempo real para apoiar decisões rápidas e eficazes.

3. Atores que interagem com o caso de uso: Analista de Dados, Sistema Sensorial, Sistema de Vídeo e Equipa de Infraestrutura Tecnológica.

4. Pré-condições:

- Todos os dispositivos de recolha de dados estão ativos e conectados ao sistema.
- O sistema possui dashboards interativos e ferramentas de análise disponíveis.
- A Infraestrutura Tecnológica suporta o processamento de grandes volumes de dados em tempo real.

5. Pós-condições:

Conclusão bem-sucedida:

- Os dados recolhidos foram analisados e interpretados com precisão.
- Relatórios detalhados foram gerados e compartilhados para apoiar a tomada de decisão.
- *Dashboards* interativos foram atualizados, refletindo informações em tempo real.

Condição de falha:

- O sistema apresenta problemas de recolha ou análise de dados.

6. Fluxo de eventos

Fluxo Normal:

- O caso de uso inicia com a recolha de dados operacionais pelo Sistema Sensorial e Sistema de Vídeo.

- A Equipa de Infraestrutura Tecnológica garante o fluxo contínuo de dados para o sistema de análise.
- O Analista de Dados acede aos *dashboards* interativos para monitorizar os dados.
- Análises e simulações são realizadas para avaliar anomalias ou tendências nos dados.
- O caso de uso termina com os dados analisados e as informações apresentadas.

Fluxo Alternativo:

Caso ocorram falhas na recolha de dados:

- Um alerta é enviado à Equipa de Infraestrutura Tecnológica.
- A equipa investiga e corrige as falhas antes de retomar a análise.

Caso anomalias sejam detetadas durante a análise:

- O Analista de Dados gera simulações específicas para investigar o impacto.
- As simulações ajudam a validar as decisões a serem tomadas.

7. Requisitos Especiais:

- Os mecanismos de segurança devem proteger a integridade dos dados.
- Os *dashboards* interativos devem ser acessíveis a partir de diferentes dispositivos.

U.C.1.5

1. **Nome do caso de uso:** Gerir Horários e Rotas.
2. **Descrição do caso de uso:** Este caso de uso descreve os passos para planear, ajustar e gerir horários e rotas de forma eficiente.
3. **Atores que interagem com o caso de uso:** Gestor de Frotas e Sistema de Localização.
4. **Pré-condições:**
 - O sistema de gestão de horários e rotas está operacional.
 - As viaturas estão devidamente equipadas com os dispositivos de localização.
5. **Pós-condições:**
Conclusão bem-sucedida:
 - Os horários e rotas foram otimizados e ajustados para garantir eficiência.
Condição de falha:
 - Foram alertados problemas ou atrasos registados.
6. **Fluxo de eventos**

Fluxo Normal:

- Este caso de uso começa com o Gestor de Frotas a aceder ao sistema para consultar os horários e rotas existentes.
- O Sistema de Localização fornece dados em tempo real sobre as condições das vias e posições das viaturas.
- São realizados ajustes para desempenho operacional.
- Notificações automáticas são enviadas às viaturas com as atualizações realizadas.
- O caso de uso termina com os horários e rotas atualizadas.

Fluxo Alternativo:**Condições climáticas adversas detetadas:**

- O Sistema de Localização identifica condições climáticas severas que afetam um dos trajetos das linhas do sistema BRT.
- Um alerta é gerado e enviado ao Gestor de Frotas indicando as áreas impactadas.
- Os motoristas são notificados das mudanças realizadas para evitar atrasos e garantir a segurança.

7. Requisitos Especiais:

- Notificações automáticas devem ser entregues com precisão.
- O sistema deve permitir ajustes em tempo real com um reduzido atraso.

U.C.2

1. Nome do caso de uso: Monitorizar em Tempo Real**2. Descrição do caso de uso:** Este caso de uso, permite o acompanhamento em tempo real da localização dos BRTs, das condições dos veículos e das vias, garantindo uma operação eficiente e segura.**3. Atores que interagem com o caso de uso:** Gestor de Frotas, Equipa de Manutenção, Dispositivo Sensorial, Sistema de Localização, Sistema de Vídeo.**4. Pré-condições:**

- Os sensores e sistemas de monitorização devem estar funcionais.
- Os veículos devem estar equipados com GPS e sensores operacionais.
- O sistema de vídeo deve estar ativo para capturar imagens das vias.

5. Pós-condições:**Conclusão bem-sucedida:**

- As informações de localização, estado dos veículos e condições das vias são atualizadas no sistema.

Condição de falha:

- O sistema gera alertas para falhas na atualização dos dados.

6. Fluxo de eventos**Fluxo Normal:**

- Os sensores e sistemas de localização captam os dados em tempo real.
- O sistema processa e exibe as informações para os utilizadores.
- Caso haja alguma anomalia, um alerta é enviado.

Fluxo Alternativo:**Se um sensor apresentar alguma falha:**

- O sistema utiliza a última informação disponível e gera um alerta para verificação manual.

7. Requisitos Especiais:

- Os dados devem ser atualizados com intervalos máximos de X segundos para a localização, com uma precisão mínima de X metros, e X segundos para as condições do veículo.
- O sistema de vídeo deve ter qualidade mínima de X e processar imagens em até X segundos.

U.C.2.1

1. **Nome do caso de uso:** Atualizar a localização dos BRT
2. **Descrição do caso de uso:** Este caso de uso permite atualizar e visualizar a localização dos veículos em tempo real, garantindo que o Gémeo Digital esteja sempre sincronizado com a posição real do BRT.
3. **Atores que interagem com o caso de uso:** Sistema de Localização.
4. **Pré-condições:**
 - O sistema de localização deve estar operacional.
 - O veículo deve estar equipado com GPS funcional.
 - O Gémeo Digital deve estar ativo para receber os dados.
5. **Pós-condições:**
Conclusão bem-sucedida:
 - A localização do veículo é atualizada no sistema.

Condição de falha:

-O sistema emite um alerta de erro.

6. Fluxo de eventos

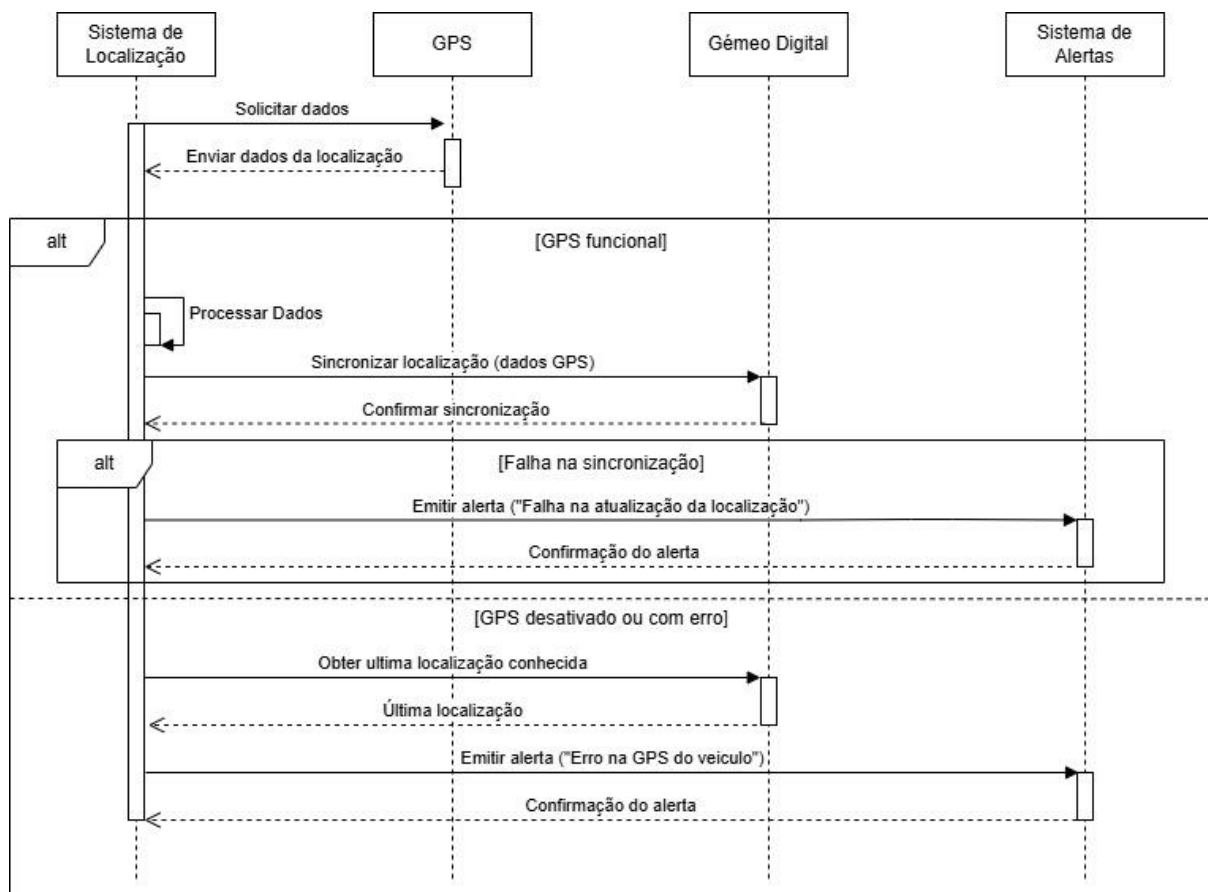
Fluxo Normal:

- O Sistema de Localização recebe dados GPS do veículo.
- Os dados são processados e sincronizados com o Gémeo Digital.
- Se houver falha na atualização, um alerta é gerado.

Fluxo Alternativo:

Se o GPS do veículo estiver desativado ou com erro:

- O sistema tenta obter a última localização conhecida.
- Um alerta é enviado ao gestor para verificar o erro.



7. Requisitos Especiais:

- O sistema deve atualizar a localização em um intervalo máximo de X segundos.
- O GPS deve ter precisão mínima de X metros.

U.C.2.2

1. **Nome do caso de uso:** Monitorizar as Condições do Veículo
2. **Descrição do caso de uso:** Este caso de uso permite acompanhar o estado dos veículos, garantindo que condições internas e externas sejam monitorizadas em tempo real.
3. **Atores que interagem com o caso de uso:** Gestor de Frotas, Equipa de Manutenção, Dispositivo Sensorial.
4. **Pré-condições:**
 - Os sensores do veículo devem estar operacionais.
 - O sistema de monitorização deve estar funcional.
 - O veículo deve estar em circulação ou disponível para inspeção.
5. **Pós-condições:**

Conclusão bem-sucedida:

 - As condições do veículo são atualizadas no sistema.

Condição de falha:

 - O sistema emite alertas para a equipa de manutenção sobre a falha no sistema.
6. **Fluxo de eventos**

Fluxo Normal:

 - Os sensores captam dados do veículo
 - O sistema processa e exibe informações para o Gestor de Frotas.
 - No caso de alguma anomalia se verificar, um alerta é enviado para a Equipa de Manutenção.

Fluxo Alternativo:

Se um sensor apresentar falhas:

 - O sistema ignora temporariamente os dados deste sensor.
 - Um alerta é enviado para a Equipa de Manutenção.
7. **Requisitos Especiais:**
 - Os sensores devem enviar dados a cada X segundos.

U.C.2.3

1. **Nome do caso de uso:** Monitorizar as Condições da Via
2. **Descrição do caso de uso:** Este caso de uso permite analisar continuamente as condições da estrada para prever impactos na operação dos BRTs.

3. **Atores que interagem com o caso de uso:** Gestor de Frotas, Equipa de Manutenção, Sistema de Vídeo.
4. **Pré-condições:**
 - O sistema de vídeo deve estar funcional.
 - A Equipa de Manutenção deve ter acesso às imagens processadas.
5. **Pós-condições:**
Conclusão bem-sucedida:
 - As informações sobre a via são atualizadas.
Condição de falha:
 - O sistema não consegue processar as imagens e emite um alerta.
6. **Fluxo de eventos**
Fluxo Normal:
 - O sistema de Vídeo capta imagens da estrada.
 - Um algoritmo analisa os dados e identifica problemas.
 - Se forem detetados danos, a Equipa de Manutenção e o Gestor de Frotas são notificados.
Fluxo Alternativo:
Se o sistema de vídeo não conseguir captar imagens devido a falhas:
 - Um alerta é enviado à Equipa de Manutenção.
7. **Requisitos Especiais:**
 - O sistema de vídeo deve operar com uma qualidade mínima de X.
 - O processamento das imagens deve ser concluído em menos de X segundos.

U.C.2.4

1. **Nome do caso de uso:** Monitorizar Ocupação do BRT
2. **Descrição do caso de uso:** Este caso de uso permite verificar a ocupação dos veículos em tempo real e analisar tendências de lotação para otimizar a distribuição da frota.
3. **Atores que interagem com o caso de uso:** Sistema de Vídeo, Analista de Dados.
4. **Pré-condições:**
 - O sistema de vídeo deve estar ativo.

- O veículo deve estar em operação e a transportar passageiros.
- O sistema de análise de dados deve estar funcional.

5. **Pós-condições:**

Conclusão bem-sucedida:

- As informações de ocupação são atualizadas no sistema.

Condição de falha:

- O sistema gera um alerta para a possível falha na captação de dados.

6. **Fluxo de eventos**

Fluxo Normal:

- O sistema de Vídeo monitoriza a ocupação do BRT.
- Os dados são enviados para o Analista de Dados, que identifica padrões de uso.
- Caso seja detetada superlotação, um alerta é gerado.
- O sistema sugere redistribuição da frota para otimizar o transporte.

Fluxo Alternativo:

Se o sistema não conseguir processar corretamente os dados de ocupação devido a falha na câmara:

- O sistema usa estimativas baseadas em histórico de ocupação.
- Um alerta é gerado para manutenção das câmaras.

7. **Requisitos Especiais:**

- O sistema deve processar dados de ocupação a cada X segundos.
- A precisão na deteção de ocupação deve ser superior a 90%.

U.C.3

1. **Nome do caso de uso:** Configurar Alertas e Notificações

2. **Descrição do caso de uso:** Este caso de uso, permite a gestão de alertas críticos e notificações para garantir ações rápidas e eficazes na operação dos BRTs.

3. **Atores que interagem com o caso de uso:** Gestor de Frotas, Equipa de Manutenção, Dispositivo Sensorial, Sistema de Vídeo, Analista de Dados, Equipa de Infraestrutura Tecnológica.

4. **Pré-condições:**

- Os sensores e sistema de vídeo devem estar operacionais.

- O sistema de gestão de alertas deve estar funcional.
- Os utilizadores devem estar autenticados para configurar notificações.

5. **Pós-condições:**

Conclusão bem-sucedida:

- Os alertas são analisados e resolvidos conforme a criticidade.

Condição de falha:

- Se os alertas não forem tratados a tempo, o sistema gera outro alerta.

6. **Fluxo de eventos**

Fluxo Normal:

- Os sensores e sistemas de vídeo identificam falhas e enviam alertas.
- O sistema classifica os alertas por nível de criticidade.
- O Gestor de Frotas e a Equipa de Manutenção analisam os alertas e tomam as devidas ações.
- Se necessário, ordens de serviço são geradas para correção.

Fluxo Alternativo:

Se o alerta for de baixa criticidade:

- Ele é registado, mas nenhuma ação imediata é tomada.

7. **Requisitos Especiais:**

- Os alertas devem ser processados em até X segundos.
- As notificações devem ser configuráveis e enviadas.
- A análise dos alertas deve ser concluída em até X tempo.

U.C.3.1

1. **Nome do caso de uso:** Gerir Alertas Críticos.
2. **Descrição do caso de uso:** Este caso de uso deve receber e gerir alertas críticos relacionados com o estado dos veículos e da via, de forma a garantir uma resposta eficiente para evitar falhas operacionais.
3. **Atores que interagem com o caso de uso:** Gestor de Frotas, Equipa de Manutenção, Equipa de Infraestrutura Tecnológica, Dispositivo Sensorial, Sistema de Vídeo, Analista de Dados.

4. **Pré-condições:**

- Os sensores e o sistema de vídeo devem estar operacionais.
- O sistema de gestão de alertas deve estar funcional.
- Os atores responsáveis devem ter acesso ao sistema.

5. **Pós-condições:**

Conclusão bem-sucedida:

- Os alertas são analisados e resolvidos conforme a criticidade.

Condição de falha:

- Se os alertas não forem tratados dentro do tempo necessário, o sistema gera uma notificação de escalonamento.

6. **Fluxo de eventos**

Fluxo Normal:

- Os sensores e o sistema de vídeo identificam falhas e enviam alertas.
- O sistema classifica os alertas por nível de criticidade.
- O Gestor de Frotas e a Equipa de Manutenção analisam os alertas e priorizam ações corretivas.
- A Equipa de Infraestrutura Tecnológica verifica falhas do sistema.
- Caso necessário, são geradas ordens de serviço para correção do problema.

Fluxo Alternativo:

Se o alerta for falso positivo ou de baixa criticidade:

- O alerta é registado, mas nenhuma ação imediata é tomada.
- O sistema armazena a informação para futuras análises.

7. **Requisitos Especiais:**

- Os alertas devem ser processados em menos de X segundos após detetar a falha.
- O sistema deve garantir alta disponibilidade para evitar perda de informações.

U.C.3.1.2

1. **Nome do caso de uso:** Analisar Alertas Críticos
2. **Descrição do caso de uso:** Este caso de uso foca-se na análise dos alertas críticos recebidos, identificando padrões e priorizando ações corretivas para evitar falhas futuras.
3. **Atores que interagem com o caso de uso:** Gestor de Frotas, Equipa de Manutenção, Analista de Dados.

4. **Pré-condições:**

- O sistema de alertas deve estar funcional.
- Os dados históricos de falhas devem estar acessíveis.

5. **Pós-condições:**

Conclusão bem-sucedida:

- Os alertas são classificados, analisados e tratados corretamente.

Condição de falha:

- Se não houver análise dentro do tempo necessário, o alerta é reenviado.

6. **Fluxo de eventos**

Fluxo Normal:

- O Analista de Dados recebe e categoriza os alertas críticos.
- São analisadas ocorrências anteriores para identificar padrões de falha.
- Com base nos padrões, são geradas recomendações de manutenção preventiva.
- O Gestor de Frotas e a Equipa de Manutenção recebem os relatórios e implementam ações corretivas.

Fluxo Alternativo:

Se não houver dados históricos suficientes:

- O alerta é tratado isoladamente sem identificação de padrões.
- O sistema sugere monitorização contínua para coletar mais dados.

7. **Requisitos Especiais:**

- A análise dos alertas deve ser concluída em até X tempo para evitar atrasos.
- O sistema deve utilizar algoritmos preditivos para análise de padrões.

U.C.3.2

1. **Nome do caso de uso:** Configurar Notificações

2. **Descrição do caso de uso:** Este caso de uso permite que os utilizadores personalizem as notificações que desejam receber, garantindo que apenas informações relevantes sejam enviadas.

3. **Atores que interagem com o caso de uso:** Gestor de Frotas, Equipa de Manutenção.

4. **Pré-condições:**

- O sistema de notificações deve estar funcional.

- O utilizador deve estar autenticado no sistema.

5. **Pós-condições:**

Conclusão bem-sucedida:

- As preferências de notificações são atualizadas corretamente.

Condição de falha:

- Se as configurações não forem guardadas, o sistema mantém as opções anteriores e gera um alerta

6. **Fluxo de eventos**

Fluxo Normal:

- O Gestor de Frotas ou a Equipa de Manutenção acedem ao sistema.
- Escolhem quais os tipos de notificações que desejam receber e ajustam os níveis de prioridade.
- Guardam as configurações, e o sistema aplica as alterações.

Fluxo Alternativo:

Se houver erro ao guardar as configurações:

- O sistema mantém as configurações anteriores e exibe um alerta ao utilizador.
- O utilizador pode tentar novamente ou contactar suporte técnico.

7. **Requisitos Especiais:**

- As notificações devem ser enviadas via múltiplos canais (SMS, e-mail, app).
- Os utilizadores devem poder alterar as preferências a qualquer momento.

U.C.4

1. **Nome do caso de uso:** Gerir a Manutenção
2. **Descrição do caso de uso:** Este caso de uso permite prever e executar a manutenção dos BRTs e das vias, garantindo a segurança e continuidade da operação.
3. **Atores que interagem com o caso de uso:** Gestor de Frotas, Equipa de Manutenção, Dispositivo Sensorial, Sistema de Vídeo, Analista de Dados.
4. **Pré-condições:**
 - Os sensores e sistemas de vídeo devem estar operacionais.
 - O sistema de análise de dados deve estar funcional.
 - A Equipa de Manutenção deve ter acesso às informações dos veículos e vias.

5. **Pós-condições:**

Conclusão bem-sucedida:

- A manutenção preventiva é programada e executada conforme necessário.

Condição de falha:

- Se a previsão falhar, pode ocorrer impacto na operação dos BRTs.

6. **Fluxo de eventos**

Fluxo Normal:

- Os sensores captam dados sobre o desempenho dos veículos.
- O sistema analisa os dados e identifica a necessidade de manutenção.
- A Equipa de Manutenção recebe notificações e agenda as ações corretivas.
- O Sistema de Vídeo captura imagens das vias e deteta irregularidades.
- Caso necessário, intervenções são programadas para reparos.

Fluxo Alternativo:

Se os sensores não fornecerem dados suficientes:

- Um alerta é enviado.

7. **Requisitos Especiais:**

- O tempo máximo para gerar uma ordem de serviço deve ser de X após um alerta crítico.
- As ordens de serviço devem conter um histórico de alterações e responsáveis.
- Os relatórios de manutenção devem ser armazenados por pelo menos X meses.

U.C.4.1

1. **Nome do caso de uso:** Prever a Manutenção do BRT

2. **Descrição do caso de uso:** Este caso de uso recolhe dados de sensores para prever a necessidade de manutenção preventiva e evitar falhas inesperadas nos veículos.

3. **Atores que interagem com o caso de uso:** Equipa de Manutenção, Dispositivo Sensorial, Analista de Dados.

4. **Pré-condições:**

- Os sensores devem estar operacionais e captando dados corretamente.
- O sistema de análise de dados deve estar funcional.
- A Equipa de Manutenção deve ter acesso às informações dos sensores.

5. **Pós-condições:**

Conclusão bem-sucedida:

-A manutenção preventiva é programada e evita falhas inesperadas.

Condição de falha:

-Se a previsão não for realizada corretamente, há risco de falha mecânica e o veículo pode apresentar problemas operacionais.

6. **Fluxo de eventos**

Fluxo Normal:

-Os sensores coletam dados sobre o desempenho dos veículos (temperatura, consumo de combustível, etc.).

-O sistema processa os dados e identifica padrões indicando a necessidade de manutenção.

-A Equipa de Manutenção recebe notificações automáticas para agendar as ações preventivas.

Fluxo Alternativo:

Se os sensores não fornecerem dados suficientes:

-O sistema envia um alerta à Equipa de Manutenção.

7. **Requisitos Especiais:**

-O sistema deve analisar os dados em tempo real para prever falhas.

-Os relatórios de previsão devem ser armazenados por pelo menos 6 meses para análises futuras.

U.C.4.2

1. **Nome do caso de uso:** Garantir a Manutenção da Via.

2. **Descrição do caso de uso:** Este caso de uso assegura que as vias utilizadas pelos BRTs estejam em bom estado, garantindo segurança e eficiência operacional.

3. **Atores que interagem com o caso de uso:** Equipa de Manutenção, Sistema de Vídeo.

4. **Pré-condições:**

-O sistema de vídeo deve estar operacional e a imagens da via.

-A Equipa de Manutenção deve ter acesso ao sistema de monitorização da via.

5. **Pós-condições:**

Conclusão bem-sucedida:

-As vias são inspecionadas regularmente e mantidas em boas condições.

Condição de falha:

-Se os problemas não forem identificados ou corrigidos a tempo, pode ocorrer impacto na segurança dos passageiros e no funcionamento dos BRTs.

6. **Fluxo de eventos**

Fluxo Normal:

- O **Sistema de Vídeo** captura imagens das vias em tempo real.
- O sistema analisa as imagens e deteta irregularidades.
- Com base nas informações, é agendada uma intervenção preventiva para reparos necessários.

Fluxo Alternativo:

Se o sistema não conseguir detetar falhas automaticamente:

- O alerta é enviado à Equipa de Manutenção para uma inspeção manual.
- O diagnóstico manual é inserido no sistema para futura referência.

7. **Requisitos Especiais:**

- O processamento das imagens deve ocorrer com um tempo de resposta inferior a X segundos.
- Os relatórios de manutenção devem ser gerados automaticamente.

U.C.4.3

1. **Nome do caso de uso:** Gerir Ordens de Serviço

2. **Descrição do caso de uso:** Este caso de uso cria, gere e acompanha ordens de serviço para manutenções corretivas dos veículos e vias.

3. **Atores que interagem com o caso de uso:** Gestor de Frotas, Equipa de Manutenção.

4. **Pré-condições:**

- O sistema de ordens de serviço deve estar disponível.
- A Equipa de Manutenção e o Gestor de Frotas devem ter permissões para aceder e gerir ordens.

5. **Pós-condições:**

Conclusão bem-sucedida:

- As ordens de serviço são geridas e as manutenções são realizadas dentro do prazo.

Condição de falha:

- Se as ordens não forem executadas a tempo, pode haver impacto na operação dos BRTs e falhas inesperadas no sistema.

6. **Fluxo de eventos**

Fluxo Normal:

- O sistema recebe alertas críticos e cria automaticamente uma ordem de serviço.
- A Equipa de Manutenção executa a manutenção e atualiza o status da ordem de serviço para "Em progresso".
- Após a conclusão da manutenção, a ordem de serviço é marcada como "Concluída".

Fluxo Alternativo:

Se o alerta não gerar uma ordem de serviço automática:

- O Gestor de Frotas pode criar uma ordem manualmente.
- A Equipa de Manutenção recebe a ordem e dá seguimento ao serviço.

7. **Requisitos Especiais:**

- O tempo máximo para atribuir uma ordem de serviço deve ser de X minutos após a geração do alerta.
- Todas as ordens de serviço devem conter histórico de alterações e responsáveis pelo atendimento.

U.C.5

1. **Nome do caso de uso:** Garantir a Segurança e Performance.

2. **Descrição do caso de uso:** Este caso de uso garante a operação robusta e segura do sistema de Gémeos Digitais por meio de testes avançados de desempenho, segurança e integridade, respondendo ativamente a falhas ou vulnerabilidades.

3. **Atores que interagem com o caso de uso:** Equipa de Manutenção, Equipa de Infraestrutura Tecnológica e Dispositivo Sensorial.

4. **Pré-condições:**

- Um ambiente de testes controlado foi configurado para realizar análises no sistema;
- Ferramentas de monitorização estão disponíveis para as equipas;
- Os dados operacionais do sistema estão acessíveis para comparação e validade;
- A Infraestrutura suporta análises de alta complexidade.

5. **Pós-condições:**

Conclusão bem-sucedida:

- As vulnerabilidades identificadas foram minimizadas com sucesso, garantindo maior segurança;

- O sistema demonstrou capacidade de manter desempenho elevado sob diferentes níveis de carga;
- Todas as métricas de desempenho e segurança respondem ou superam os requisitos operacionais definidos.

Condição de falha:

- Relatórios detalhados são gerados, indicando os pontos de falha e sugestões de melhoria.

6. Fluxo de eventos

Fluxo Normal:

- O caso de uso começa com a ativação de testes programados pela Equipa de Infraestrutura Tecnológica;
- São realizados testes de carga no sistema, avaliando o desempenho;
- A segurança do sistema é validada, incluindo ataques como *SQL Injection*;
- A Equipa de Manutenção observa o comportamento de componentes mecânicos durante os testes de desempenho;
- Os resultados dos testes são compilados e comparados com os requisitos operacionais definidos, de forma a ajustar o necessário;
- O caso de uso termina com o sistema pronto para operar normalmente.

Fluxo Alternativo:

Desempenho insuficiente identificado nos testes:

- Durante os testes, o sistema apresenta tempos de resposta demasiado altos;
- Um alerta é enviado à Equipa de Infraestrutura Tecnológica para realizar os ajustes no dimensionamento do sistema;
- Após os ajustes, os testes de carga são reiniciados para validar as melhorias.

7. Requisitos Especiais:

- Garantir um histórico de registos para análise comparativa;
- Estabelecer protocolos claros de resposta a falhas durante os testes.

U.C.5.1

1. **Nome do caso de uso:** Testar a carga do sistema.
2. **Descrição do caso de uso:** Este caso de uso verifica a capacidade do sistema em lidar com diferentes níveis de carga, garantindo com que o desempenho se mantenha estável sob condições variadas.
3. **Atores que interagem com o caso de uso:** Equipa de Infraestrutura Tecnológica.

4. **Pré-condições:**

- O sistema deve estar operacional e disponível para testes.
- Os requisitos de desempenho devem estar claramente definidos.

5. **Pós-condições:**

Conclusão bem-sucedida:

- O sistema suporta os níveis de carga esperados.

Condição de falha:

- O sistema apresenta degradação de desempenho ou falhas.
- São recomendados ajustes na infraestrutura.

6. **Fluxo de eventos**

Fluxo Normal:

- A Equipa de Infraestrutura Tecnológica inicia os testes de carga.
- O sistema é submetido a um aumento progressivo de carga.
- São analisadas métricas como tempo de resposta e consumo de recursos.
- O caso de uso termina com a avaliação dos resultados.

Fluxo Alternativo:

- Se o desempenho for inferior ao esperado, novos testes são agendados.
- São realizados ajustes no sistema antes de repetir os testes.

7. **Requisitos Especiais:**

- Utilização de ferramentas automatizadas para simulação de carga.

U.C.5.2

1. **Nome do caso de uso:** Testar *SQL Injection*.

2. **Descrição do caso de uso:** Este caso de uso avalia a vulnerabilidade do sistema a ataques de *SQL Injection*, garantindo que os mecanismos de segurança impedem acessos não autorizados.

3. **Atores que interagem com o caso de uso:** Equipa de Infraestrutura Tecnológica.

4. **Pré-condições:**

- O sistema deve ter medidas de segurança ativas contra injeção de SQL.

5. **Pós-condições:**

Conclusão bem-sucedida:

- O sistema bloqueia todas as tentativas de injeção de SQL.

Condição de falha:

- Alguma tentativa de ataque é bem-sucedida, expondo falhas de segurança.

6. **Fluxo de eventos**

Fluxo Normal:

- A Equipa de Infraestrutura Tecnológica executa testes de *SQL Injection*.
- São tentadas inserções de código SQL malicioso nos campos do sistema.
- O sistema valida e bloqueia as tentativas de injeção de código.
- O caso de uso termina com a revisão dos mecanismos de proteção.

Fluxo Alternativo:

- Se alguma vulnerabilidade for encontrada, medidas corretivas são aplicadas antes da próxima testagem.

7. **Requisitos Especiais:**

- Utilização de ferramentas especializadas para testes de segurança.

U.C.5.3

1. **Nome do caso de uso:** Medir o Tempo de Carregamento da Interface.

2. **Descrição do caso de uso:** Este caso de uso mede o tempo de resposta da interface, garantindo que os tempos de carregamento estão dentro dos parâmetros aceitáveis.

3. **Atores que interagem com o caso de uso:** Equipa de Infraestrutura Tecnológica e Equipa de Manutenção.

4. **Pré-condições:**

- Os tempos de carregamento aceitáveis devem estar definidos.
- As ferramentas de monitorização devem estar disponíveis.

5. **Pós-condições:**

Conclusão bem-sucedida:

- O sistema atende aos tempos de resposta esperados.

Condição de falha:

- Os tempos de carregamento estão acima dos limites definidos.

6. **Fluxo de eventos**

Fluxo Normal:

- A Equipa de Infraestrutura Tecnológica inicia a medição do tempo de carregamento.
- São analisados diferentes cenários de uso e acesso ao sistema.
- A Equipa de Manutenção avalia o impacto do desempenho na usabilidade geral.
- O tempo de resposta da interface é comparado com os padrões aceitáveis.
- O caso de uso termina com a análise das métricas de desempenho.

Fluxo Alternativo:

- Se os tempos forem inadequados, ajustes na infraestrutura são aplicados antes de uma nova medição.

7. **Requisitos Especiais:**

- Ferramentas de monitorização devem registar os tempos de resposta automaticamente.

U.C.5.4

1. **Nome do caso de uso:** Monitorizar falhas mecânicas.
2. **Descrição do caso de uso:** Este caso de uso monitoriza o estado mecânico do sistema, identificando falhas em tempo real para evitar interrupções inesperadas.
3. **Atores que interagem com o caso de uso:** Equipa de Infraestrutura Tecnológica, Equipa de Manutenção e Dispositivo Sensorial.
4. **Pré-condições:**
 - Os sensores de monitorização mecânica devem estar operacionais;
 - A infraestrutura deve permitir a comunicação entre sensores e as equipas responsáveis.
5. **Pós-condições:**

Conclusão bem-sucedida:

 - As falhas mecânicas foram identificadas e corrigidas antes de impactar a operação.

Condição de falha:

 - As falhas mecânicas graves ocorreram sem tempo suficiente para correção preventiva.
6. **Fluxo de eventos**

Fluxo Normal:

- O Dispositivo Sensorial recolhe dados sobre o estado mecânico dos componentes;
- A Equipa de Infraestrutura Tecnológica analisa os dados recebidos;
- A Equipa de Manutenção recebe alertas sobre possíveis falhas e toma ações corretivas;
- O caso de uso termina com a garantia de funcionamento adequado do sistema.

Fluxo Alternativo:

- Se uma falha for identificada, a manutenção preditiva é acionada antes da interrupção do serviço.

7. Requisitos Especiais:

- Integração de sensores avançados para detetar anomalias mecânicas.
- Sistema de alertas automatizado para notificações em tempo real.

U.C.6

1. **Nome do caso de uso:** Monitorizar a Pontualidade e a Posição.

2. **Descrição do caso de uso:** Este caso de uso permite monitorizar a localização e pontualidade das viaturas, notificando atrasos ou alterações para uma gestão eficiente.

3. **Atores que interagem com o caso de uso:** Sistema de Localização, Gestor de Frotas, Equipa de Infraestrutura Tecnológica, Dispositivo Sensorial e Sistema de Vídeo.

4. **Pré-condições:**

- As viaturas estão devidamente equipadas com os sensores, dispositivos e sistemas de localização.
- As equipas têm acesso às ferramentas de monitorização e análise.
- A Infraestrutura Tecnológica deve permitir a comunicação em tempo real.

5. **Pós-condições:**

Conclusão bem-sucedida:

- A localização das viaturas é registada corretamente.
- Os motoristas foram informados sobre os atrasos e alterações na via, permitindo os ajustes necessários.
- A pontualidade das operações foi monitorizada e os relatórios gerados com sucesso.

Condição de falha:

- Os registos do sistema foram registados com os problemas identificados, como falhas na recolha de dados ou atrasos imprevistos.

6. **Fluxo de eventos**

Fluxo Normal:

- O caso de uso começa com a ativação do sistema de monitorização da posição das viaturas.
- Os sensores e sistemas de localização enviam dados em tempo real.
- O sistema compara os tempos de chegada esperados com os tempos reais.
- Se forem identificados atrasos ou alterações na rota, os motoristas e gestores são notificados.
- O caso de uso termina com os dados registados e analisados.

Fluxo Alternativo:

Falha na comunicação com o sistema de localização:

- O sistema deteta uma falha na receção de dados.
- Tenta recuperar a informação por meio de fontes secundárias, como o Sistema de Vídeo.
- Se a falha persistir, um alerta é enviado à Equipa de Infraestrutura Tecnológica.

Caso sejam detetadas alterações na via:

- Os motoristas são notificados com alternativas sugeridas.
- O Gestor de Frotas avalia o impacto na pontualidade e faz os ajustes necessários.

7. **Requisitos Especiais:**

- O sistema deve ser capaz de operar em tempo real com baixa demora.
- A comunicação com os motoristas deve ser garantida em tempo real.
- Relatórios para cada ciclo de monitorização devem ser gerados.

U.C.6.1

1. **Nome do caso de uso:** Mapear a localização da viatura.
2. **Descrição do caso de uso:** Este caso de uso permite acompanhar a localização da viatura em tempo real e garantir a sua atualização contínua no sistema.
3. **Atores que interagem com o caso de uso:** Sistema de Localização, Equipa de Infraestrutura Tecnológica e Gestor de Frotas.
4. **Pré-condições:**
 - O sistema de localização deve estar ligado e funcional.

- A Infraestrutura Tecnológica deve estar a processar corretamente os dados de posição.

5. **Pós-condições:**

Conclusão bem-sucedida:

- A viatura foi localizada corretamente e a sua posição foi registada no sistema.

Condição de falha:

- O sistema não conseguiu obter a localização da viatura devido a falhas na rede ou nos sensores.

6. **Fluxo de eventos**

Fluxo Normal:

- O sistema de localização ativa o acompanhamento da viatura.
- A posição da viatura é capturada e armazenada.
- A Equipa de Infraestrutura Tecnológica verifica a precisão dos dados.
- O caso de uso termina quando a localização é atualizada corretamente.

Fluxo Alternativo:

Falha na obtenção da localização:

- Se o sistema não conseguir obter a posição, tenta uma nova leitura.
- Se a falha persistir, a equipa de infraestrutura tecnológica é acionada para diagnóstico.

7. **Requisitos Especiais:**

- O sistema deve garantir uma atualização mínima da localização a cada X segundos.

U.C.6.2

1. **Nome do caso de uso:** Notificar Motorista sobre Atrasos.
2. **Descrição do caso de uso:** Este caso de uso permite que o motorista seja notificado sobre atrasos no percurso, com base nos dados analisados pelo sistema.
3. **Atores que interagem com o caso de uso:** Sistema de Localização, Equipa de Infraestrutura Tecnológica, Gestor de Frotas, Dispositivo Sensorial e Sistema de Vídeo.
4. **Pré-condições:**
 - O sistema de localização deve estar a monitorizar os tempos de deslocação.
 - Os sensores e câmaras devem captar dados sobre outras viaturas.

5. **Pós-condições:**

Conclusão bem-sucedida:

- O motorista foi informado sobre atrasos e pôde ajustar a sua condução.

Condição de falha:

- O sistema não conseguiu calcular os atrasos corretamente.
- O motorista não é notificado sobre atrasos.

6. **Fluxo de eventos**

Fluxo Normal:

- O sistema de localização recebe os dados sobre tempos de deslocação.
- Sensores e câmaras analisam o trânsito e verificam possíveis atrasos.
- O motorista é notificado sobre o atraso e recebe recomendações.
- O caso de uso termina com a notificação registada no sistema.

Fluxo Alternativo:

Erro na deteção de atrasos:

- Se os sensores não captam corretamente os dados, o gestor de frotas recebe um alerta

7. **Requisitos Especiais:**

- Garantir uma margem de erro mínima nas estimativas de atraso.

U.C.6.3

1. **Nome do caso de uso:** Notificar Alterações na Via.

2. **Descrição do caso de uso:** Este caso de uso informa os motoristas e gestores sobre alterações na via, como obras ou acidentes.

3. **Atores que interagem com o caso de uso:** Sistema de Localização, Equipa de Infraestrutura Tecnológica, Sistema de Vídeo e Gestor de Frota.

4. **Pré-condições:**

- As Câmaras de Vídeo devem estar operacionais para detetar incidentes na via.

5. **Pós-condições:**

Conclusão bem-sucedida:

- As informações sobre as condições da via foram enviadas corretamente.

Condição de falha:

- O sistema não conseguiu identificar mudanças na via a tempo.

6. Fluxo de eventos

Fluxo Normal:

- As câmaras de Vídeo captam informações sobre mudanças na via;
- O sistema processa os dados e identifica impactos no percurso.
- O motorista e o Gestor de Frotas são notificados.

Fluxo Alternativo:

Deteção de evento incorreto:

- Se houver uma falha na deteção, a equipa de Infraestrutura Tecnológica investiga o problema.

7. Requisitos Especiais:

- Implementar validação dupla das informações antes do envio.

U.C.6.4

1. **Nome do caso de uso:** Detetar eventuais atrasos.
2. **Descrição do caso de uso:** Este caso de uso analisa os tempos de deslocação e prevê possíveis atrasos antes que ocorram.
3. **Atores que interagem com o caso de uso:** Sistema de Localização, Equipa de Infraestrutura Tecnológica, Sistema de Vídeo, Dispositivo Sensorial e Gestor de Frotas.
4. **Pré-condições:**
 - Os tempos de deslocação anteriores devem estar armazenados para comparação.
5. **Pós-condições:**

Conclusão bem-sucedida:

 - Atrasos foram previstos e notificações foram enviadas.

Condição de falha:

- Os atrasos não foram detetados corretamente.

6. Fluxo de eventos

Fluxo Normal:

- O sistema de localização recolhe os dados sobre a posição e velocidade das viaturas.
- Os dispositivos sensoriais e o sistema de vídeo captam informações sobre as condições da via.
- O sistema compara os tempos de deslocação atuais com os tempos previstos.
- Se houver desvios significativos, o sistema identifica um possível atraso.
- O gestor de frotas e o motorista são notificados sobre o possível atraso.
- O caso de uso termina com o registo da análise e envio das notificações.

Fluxo Alternativo:**Falha na previsão de atrasos:**

- O sistema calibra novamente os parâmetros e volta a tentar.

7. Requisitos Especiais:

- O sistema deve utilizar IA para melhorar a previsão de atrasos.

12. Casos de Teste

T.C.1

1. **Nome do caso de teste:** Criar um Gémeo Digital
2. **Caso de Uso Associado:** U.C.1.1 Criar o sistema de Gémeos Digitais
3. **Descrição:** Testa se o sistema permite a criação de um novo Gémeo Digital e a sua correta inclusão no ambiente virtual.
4. **Pré-condições:** O utilizador deve ter permissões administrativas para criar novos Gémeos Digitais.
5. **Resultado Esperado:** O sistema cria o Gémeo Digital e exibe uma mensagem de sucesso.
6. **Pós-condição:** O novo Gémeo Digital é listado no sistema e está pronto para sincronização.

T.C.2

1. **Nome do caso de teste:** Atualizar Parâmetros do Gémeo Digital
2. **Caso de Uso Associado:** U.C.1.1.2 Atualizar Parâmetros do Gémeo Digital
3. **Descrição:** Verifica se o utilizador pode modificar os parâmetros do Gémeo Digital para garantir uma representação fiel da realidade.
4. **Pré-condições:** O Gémeo Digital deve estar previamente criado.
5. **Resultado Esperado:** Os parâmetros são atualizados e uma mensagem de confirmação é exibida.
6. **Pós-condição:** O Gémeo Digital reflete as novas configurações sem falhas.

T.C.3

1. **Nome do caso de teste:** Atualizar Informações do Veículo
2. **Caso de Uso Associado:** U.C.1.2 Atualizar Informações do Veículo
3. **Descrição:** Avalia se o sistema permite a atualização de dados do veículo de forma precisa.
4. **Pré-condições:** O veículo deve estar registado no sistema.
5. **Resultado Esperado:** O sistema exibe uma mensagem de sucesso e as informações do veículo são atualizadas.
6. **Pós-condição:** As novas informações do veículo são refletidas corretamente no sistema.

T.C.4

1. **Nome do caso de teste:** Integrar Sensores Externos
2. **Caso de Uso Associado:** U.C.1.3 Integrar Sistemas Externos
3. **Descrição:** Testa a capacidade do sistema de integrar sensores externos para recolha de dados.
4. **Pré-condições:** Os sensores devem estar ligados e disponíveis para integração.
5. **Resultado Esperado:** O sistema recebe dados dos sensores e reflete-os corretamente.
6. **Pós-condição:** A integração com os sensores está ativa e funcional.

T.C.5

1. **Nome do caso de teste:** Atualizar a Localização dos BRT

2. **Caso de Uso Associado:** U.C.2.1 Atualizar a localização dos BRT
3. **Descrição:** Verifica se a localização dos BRTs é atualizada corretamente no sistema.
4. **Pré-condições:** O BRT deve estar equipado com GPS funcional.
5. **Resultado Esperado:** O Gémeo Digital exibe a posição correta do BRT.
6. **Pós-condição:** A localização dos BRTs é atualizada corretamente no sistema.

T.C.6

1. **Nome do caso de teste:** Monitorizar Ocupação do BRT
2. **Caso de Uso Associado:** U.C.2.4 Monitorizar Ocupação do BRT
3. **Descrição:** Testa se o sistema é capaz de monitorizar a ocupação dos veículos em tempo real.
4. **Pré-condições:** O sistema deve estar a recolher dados de ocupação através de câmaras ou sensores.
5. **Resultado Esperado:** O sistema exibe corretamente os níveis de ocupação e emite alertas quando necessário.
6. **Pós-condição:** Os dados de ocupação são registados corretamente e usados para otimização da frota.

T.C.7

1. **Nome do caso de teste:** Gerir Alertas Críticos
2. **Caso de Uso Associado:** U.C.3.1 Gerir Alertas Críticos

3. **Descrição:** Verifica se o sistema gere corretamente alertas críticos gerados pelos veículos e infraestruturas.
4. **Pré-condições:** O sistema deve estar a monitorizar as condições dos veículos e das vias.
5. **Resultado Esperado:** O alerta é gerado, a equipa responsável é notificada e a falha é registada.
6. **Pós-condição:** O sistema regista corretamente os alertas críticos e envia notificações aos utilizadores.

T.C.8

1. **Nome do caso de teste:** Configurar e Personalizar Notificações
2. **Caso de Uso Associado:** U.C.3.2 Configurar Notificações
3. **Descrição:** Testa se o utilizador pode configurar as notificações do sistema conforme as suas preferências.
4. **Pré-condições:** O utilizador deve estar autenticado com credenciais válidas para configurar notificações.
5. **Resultado Esperado:** O sistema guarda as preferências e notifica corretamente conforme as definições.
6. **Pós-condição:** As notificações personalizadas são registadas e aplicadas corretamente.

T.C.9

1. **Nome do caso de teste:** Gerir a Manutenção
2. **Caso de Uso Associado:** U.C.4 Gerir a Manutenção

3. **Descrição:** Testa se o sistema permite gerir manutenções preventivas e corretivas de veículos e infraestruturas.
4. **Pré-condições:** O sistema deve ter veículos e infraestruturas registadas.
5. **Resultado Esperado:** O sistema regista manutenções agendadas e permite monitorizar a execução.
6. **Pós-condição:** Os registos de manutenção são atualizados corretamente e refletem o estado do veículo ou infraestrutura.

T.C.10

1. **Nome do caso de teste:** Testar a carga do sistema
2. **Caso de Uso Associado:** U.C.5.1 Testar a carga do sistema
3. **Descrição:** Avalia a estabilidade do sistema sob carga elevada
4. **Pré-condições:** O sistema deve estar a funcionar num ambiente controlado.
5. **Resultado Esperado:** O sistema mantém a estabilidade e não apresenta falhas.
6. **Pós-condição:** O sistema suporta a carga prevista sem comprometer a usabilidade.

T.C.11

1. **Nome do caso de teste:** Medir Tempo de Carregamento da Interface
2. **Caso de Uso Associado:** U.C.5.3 Medir o Tempo de carregamento da interface
3. **Descrição:** Testa se o tempo de carregamento da interface está dentro dos parâmetros aceitáveis.
4. **Pré-condições:** O sistema deve estar implementado e acessível.

5. **Resultado Esperado:** A interface carrega em menos de x segundos.
6. **Pós-condição:** O sistema mantém tempos de resposta aceitáveis.

T.C.12

1. **Nome do caso de teste:** Simular uma Falha no Sistema e Verificar se Gera um Alerta
2. **Caso de Uso Associado:** U.C.5.4 Monitorizar falhas no sistema
3. **Descrição:** Avalia se o sistema é capaz de detetar falhas operacionais e gerar alertas automáticos para os utilizadores responsáveis.
4. **Pré-condições:** O sistema deve estar em funcionamento e com os sensores ativos.
5. **Resultado Esperado:** O sistema deteta a falha, gera um alerta e envia notificações.
6. **Pós-condição:** O alerta permanece registado no histórico de falhas do sistema.

T.C.13

1. **Nome do caso de teste:** Notificar Motorista sobre Atrasos
2. **Caso de Uso Associado:** U.C.6.2 Notificar Motorista sobre Atrasos
3. **Descrição:** Verifica se o sistema consegue detetar atrasos nos veículos e notificar os motoristas de forma automática.
4. **Pré-condições:** O sistema deve estar a monitorizar os horários dos veículos.
5. **Resultado Esperado:** O motorista recebe uma notificação sobre o atraso.
6. **Pós-condição:** O sistema mantém um registo de notificações enviadas.

T.C.14

1. **Nome do caso de teste:** Detetar Complicações na Via e Emitir Alertas Automáticos
2. **Caso de Uso Associado:** U.C.6.4 Detetar eventuais complicações
3. **Descrição:** Testa se o sistema é capaz de identificar automaticamente complicações na via, como congestionamentos, acidentes ou obstáculos, e alertar os gestores.
4. **Pré-condições:** O sistema deve estar integrado com sensores e câmaras para análise da via.
5. **Resultado Esperado:** O sistema deteta a complicação e envia alertas em tempo real.
6. **Pós-condição:** O sistema mantém um registo das complicações detetadas.

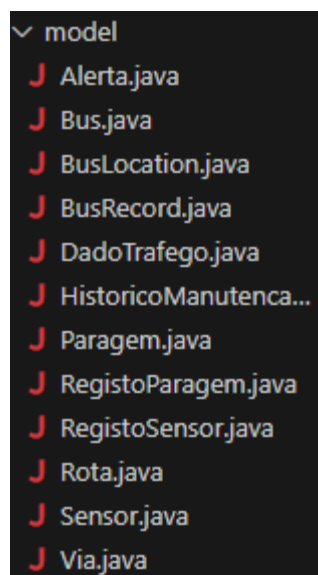
13. Build

13.1-Introdução

A elaboração deste sistema de monitorização de autocarros foi desenvolvida através de uma metodologia fundamentada em Spring Boot, que depois de analisarmos as outras aplicações, chegamos à conclusão de que esta proporciona solidez e escalabilidade ao sistema. O processo de construção abrange a incorporação de dependências fundamentais para a interação entre os elementos do sistema, como bancos de dados, serviços de cronograma e gestão de dados em tempo real. Adicionalmente, o sistema foi desenvolvido para assegurar a atualização constante das informações dos autocarros e a integração com os gêmeos digitais, permitindo o gerenciamento e a supervisão da frota de maneira eficiente e com elevada disponibilidade.

13.2-Model

Pasta onde estão localizados os ficheiros referentes ao model.



13.2.1-Bus.java

Este código define a classe Bus como uma entidade JPA, mapeando dados para uma tabela chamada "Autocarros". Contém atributos como id, licensePlate, modelo, totalCapacity, estado, engineType e batteryType, mapeados com a anotação @Column.

```
package com.busmonitoring.model;

import jakarta.persistence.*;
import lombok.Data;

@Data
@Entity
@Table(name = "Autocarros")
public class Bus {
    @Id
    @Column(name = "id_autocarro")
    private Integer id;

    @Column(name = "id_matricula", nullable = false)
    private String licensePlate;

    @Column(nullable = false)
    private String modelo;

    @Column(name = "capacidade_total")
    private Integer totalCapacity;

    @Column(nullable = false)
    private String estado;

    @Column(name = "tipo_motor")
    private String engineType;

    @Column(name = "tipo_bateria")
    private String batteryType;
}
```

13.2.2-BusLocation.java

Este código define a classe BusLocation como uma entidade JPA, mapeando dados para a tabela "Localizacoes_Autocarros". Contém atributos como id, latitude, longitude e timestamp, sendo latitude e longitude do tipo Double e timestamp do tipo LocalDateTime.

```
package com.busmonitoring.model;

import jakarta.persistence.*;
import lombok.Data;
import java.time.LocalDateTime;

@Data
@Entity
@Table(name = "Localizacoes_Autocarros")
public class BusLocation {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    @Column(name = "id_localizacao")
    private Integer id;

    @ManyToOne
    @JoinColumn(name = "id_autocarro", referencedColumnName = "id_autocarro")
    private Bus bus;

    private Double latitude;
    private Double longitude;

    @Column(name = "data_hora")
    private LocalDateTime timestamp;
}
```

13.2.3-BusRecord.java

Este código define a classe BusRecord como uma entidade JPA, mapeando dados para a tabela "Registos_Autocarros". A classe contém atributos como id, velocidade, energyConsumption e internalTemperature.

```
package com.busmonitoring.model;

import jakarta.persistence.*;
import lombok.Data;
import java.time.LocalDateTime;

@Data
@Entity
@Table(name = "Registos_Autocarros")
public class BusRecord {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    @Column(name = "id_registro")
    private Integer id;

    @ManyToOne
    @JoinColumn(name = "id_autocarro", referencedColumnName = "id_autocarro")
    private Bus bus;

    private Double velocidade;

    @Column(name = "consumo_energetico")
    private Double energyConsumption;

    @Column(name = "temperatura_interna")
    private Double internalTemperature;

    @Column(name = "pressao_pneus")
    private Double tirePressure;

    @Column(name = "temperatura_motor")
    private Double engineTemperature;

    @Column(name = "lugares_ocupados")
    private Integer occupiedSeats;

    @Column(name = "estado_travoes")
    private String brakeStatus;

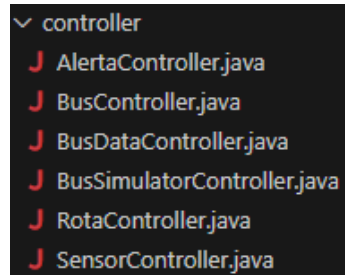
    @Column(name = "estado_suspensao")
    private String suspensionStatus;

    @Column(name = "estado_kms")
    private String kilometerStatus;

    @Column(name = "data_hora")
    private LocalDateTime timestamp;
```

13.3-Controller

Pasta onde estão localizados os ficheiros referentes ao controller.



13.3.1-BusController.java

Este código define a classe BusController como um controlador Spring, mapeando as operações relacionadas aos autocarros. Contém dependências injetadas com a anotação @Autowired, como BusRepository e BusLocationRepository, que permitem interagir com os dados de autocarros e localizações.

```
package com.busmonitoring.controller;

import com.busmonitoring.model.Bus;
import com.busmonitoring.model.BusLocation;
import com.busmonitoring.model.BusRecord;
import com.busmonitoring.repository.BusLocationRepository;
import com.busmonitoring.repository.BusRecordRepository;
import com.busmonitoring.repository.BusRepository;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.ResponseBody;

import java.util.HashMap;
import java.util.List;
import java.util.Map;

@Controller
public class BusController {

    @Autowired
    private BusRepository busRepository;

    @Autowired
    private BusLocationRepository locationRepository;
```

```
@Autowired
private BusRecordRepository recordRepository;

@GetMapping("/")
public String listBuses(Model model) {
    List<Bus> buses = busRepository.findAll();
    model.addAttribute("buses", buses);
    return "index";
}

@GetMapping("/autocarro/{id}")
public String showBus(@PathVariable Integer id, Model model) {
    Bus bus = busRepository.findById(id).orElseThrow();
    BusLocation location = locationRepository.findFirstByBusOrderByTimestampDesc(id);
    List<BusRecord> records = recordRepository.findByBusOrderByTimestampDesc(id);

    model.addAttribute("bus", bus);
    model.addAttribute("location", location);
    model.addAttribute("record", records.isEmpty() ? null : records.get(0));

    return "autocarro";
}

@GetMapping("/api/bus/{id}/data")
@ResponseBody
```

```
public Map<String, Object> getBusData(@PathVariable Integer id) {
    Map<String, Object> data = new HashMap<>();

    BusLocation location = locationRepository.findFirstByBusOrderByTimestampDesc(id);
    List<BusRecord> records = recordRepository.findByBusOrderByTimestampDesc(id);
    BusRecord latestRecord = records.isEmpty() ? null : records.get(0);

    data.put("location", location);
    data.put("record", latestRecord);

    return data;
}

@GetMapping("/bus/{id}/records")
public String showBusRecords(@PathVariable Integer id, Model model) {
    Bus bus = busRepository.findById(id).orElseThrow();
    List<BusRecord> records = recordRepository.findByBusOrderByTimestampDesc(id);

    model.addAttribute("bus", bus);
    model.addAttribute("records", records);

    return "records";
}
}
```


13.3.2-BusDataController.java

Este código define a classe BusDataController como um controlador REST, responsável por lidar com a atualização dos dados de autocarros. Utiliza a anotação @RestController e a rota /api/bus-data para gerir requisições POST.

```
package com.busmonitoring.controller;

import com.busmonitoring.model.BusLocation;
import com.busmonitoring.model.BusRecord;
import com.busmonitoring.service.BusDataService;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

@RestController
@RequestMapping("/api/bus-data")
@CrossOrigin(origins = "*")
public class BusDataController {

    @Autowired
    private BusDataService busDataService;

    @PostMapping("/{busId}")
    public ResponseEntity<String> updateBusData(
        @PathVariable Integer busId,
        @RequestBody BusDataRequest request) {
        try {
            busDataService.updateBusData(busId, request.getLocation(), request.getRecord());
            return ResponseEntity.ok("Data updated successfully");
        } catch (Exception e) {
            return ResponseEntity.badRequest().body("Error updating data: " + e.getMessage());
        }
    }
}
```

```
}  
  
public static class BusDataRequest {  
    private BusLocation location;  
    private BusRecord record;  
  
    // Getters and setters  
    public BusLocation getLocation() {  
        return location;  
    }  
  
    public void setLocation(BusLocation location) {  
        this.location = location;  
    }  
  
    public BusRecord getRecord() {  
        return record;  
    }  
  
    public void setRecord(BusRecord record) {  
        this.record = record;  
    }  
}  
}
```

13.3.3-BusSimulatorController.java

Este código define a classe BusSimulatorController como um controlador REST para gerenciar a simulação de movimentos de autocarros. Utiliza a anotação @RestController e a rota /api/simulator. A classe depende do serviço BusSimulatorService, injetado com a anotação @Autowired.

```
package com.busmonitoring.controller;

import com.busmonitoring.service.BusSimulatorService;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

@RestController
@RequestMapping("/api/simulator")
public class BusSimulatorController {

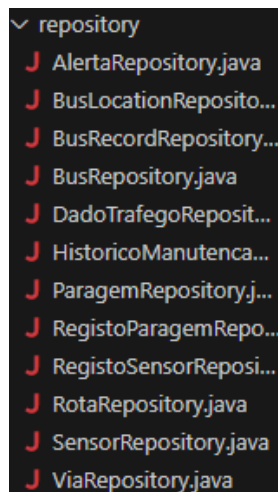
    @Autowired
    private BusSimulatorService simulatorService;

    @PostMapping("/start")
    public ResponseEntity<String> startSimulation() {
        simulatorService.simulateBusMovement();
        return ResponseEntity.ok("Simulation started");
    }

    @PostMapping("/stop")
    public ResponseEntity<String> stopSimulation() {
        // Future implementation for stopping simulation
        return ResponseEntity.ok("Simulation stopped");
    }
}
```

13.4-Repository

Pasta onde estão localizados os ficheiros referentes ao Repository.



- repository
 - AlertaRepository.java
 - BusLocationRepository.java
 - BusRecordRepository.java
 - BusRepository.java
 - DadoTrafegoRepository.java
 - HistoricoManutencaoRepository.java
 - ParagemRepository.java
 - RegistoParagemRepository.java
 - RegistoSensorRepository.java
 - RotaRepository.java
 - SensorRepository.java
 - ViaRepository.java

13.4.1-BusLocationRepository.java

Esta classe define um repositório para a entidade BusLocation, com uma consulta personalizada que retorna a última localização de um autocarro com base no seu ID. A consulta SQL ordena os resultados pela data e hora em ordem decrescente. O método `findFirstByBusOrderByTimestampDesc` executa essa consulta e retorna o registo mais recente.

```
1 package com.busmonitoring.repository;
2
3 import com.busmonitoring.model.BusLocation;
4 import org.springframework.data.jpa.repository.JpaRepository;
5 import org.springframework.data.jpa.repository.Query;
6 import org.springframework.data.repository.query.Param;
7
8 public interface BusLocationRepository extends JpaRepository<BusLocation, Integer> {
9     @Query("SELECT bl FROM BusLocation bl WHERE bl.bus.id = :busId ORDER BY bl.timestamp DESC LIMIT 1")
10     BusLocation findFirstByBusOrderByTimestampDesc(@Param("busId") Integer busId);
11 }
```

13.4.2-BusRecordRepository.java

Esta classe define um repositório para a entidade BusRecord, com uma consulta personalizada que retorna os registos de um autocarro com base no seu ID (busId). A consulta SQL ordena os resultados pela data e hora de registo em ordem decrescente.

```
1 package com.busmonitoring.repository;
2
3 import com.busmonitoring.model.BusRecord;
4 import org.springframework.data.jpa.repository.JpaRepository;
5 import org.springframework.data.jpa.repository.Query;
6 import org.springframework.data.repository.query.Param;
7
8 import java.util.List;
9
10 public interface BusRecordRepository extends JpaRepository<BusRecord, Integer> {
11     @Query("SELECT br FROM BusRecord br WHERE br.bus.id = :busId ORDER BY br.timestamp DESC")
12     List<BusRecord> findByBusOrderByTimestampDesc(@Param("busId") Integer busId);
13 }
```

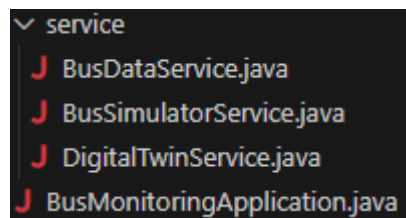
13.4.3-BusRepository.java

Esta classe define um repositório para a entidade Bus, permitindo realizar operações de persistência sobre a tabela associada. Ele estende JpaRepository, proporcionando métodos prontos para manipulação dos dados de Bus com identificadores do tipo Integer.

```
1 package com.busmonitoring.repository;
2
3 import com.busmonitoring.model.Bus;
4 import org.springframework.data.jpa.repository.JpaRepository;
5
6 public interface BusRepository extends JpaRepository<Bus, Integer> {
7 }
```

13.5-Service

Pasta onde estão localizados os ficheiros referentes ao Service.



13.5.1-BusDataService.java

Esta classe atualiza os dados de localização e registo de um autocarro, além de sincronizar com um gêmeo digital. O método updateBusData recebe o ID do autocarro, a localização e o registo, atualizando-os com o carimbo de data e hora atual. Se fornecidos, os dados de localização e registo são salvos, e o gêmeo digital do autocarro é atualizado com os novos dados.

```
1 package com.busmonitoring.service;
2
3 import com.busmonitoring.model.Bus;
4 import com.busmonitoring.model.BusLocation;
5 import com.busmonitoring.model.BusRecord;
6 import com.busmonitoring.repository.BusLocationRepository;
7 import com.busmonitoring.repository.BusRecordRepository;
8 import com.busmonitoring.repository.BusRepository;
9 import org.springframework.beans.factory.annotation.Autowired;
10 import org.springframework.stereotype.Service;
11 import org.springframework.transaction.annotation.Transactional;
12
13 import java.time.LocalDateTime;
14
15 @Service
16 public class BusDataService {
17
18     @Autowired
19     private BusRepository busRepository;
20
21     @Autowired
22     private BusLocationRepository locationRepository;
23
24     @Autowired
25     private BusRecordRepository recordRepository;
26
27     @Autowired
28     private DigitalTwinService digitalTwinService;
29
30     @Transactional
```

```
31     public void updateBusData(Integer busId, BusLocation location, BusRecord record) {
32         Bus bus = busRepository.findById(busId)
33             .orElseThrow(() -> new RuntimeException("Bus not found with id: " + busId));
34
35         // Set current timestamp
36         LocalDateTime now = LocalDateTime.now();
37
38         // Update location if provided
39         if (location != null) {
40             location.setId(null); // Ensure ID is null for auto-generation
41             location.setBus(bus);
42             location.setTimestamp(now);
43             location = locationRepository.save(location);
44         }
45
46         // Update record if provided
47         if (record != null) {
48             record.setId(null); // Ensure ID is null for auto-generation
49             record.setBus(bus);
50             record.setTimestamp(now);
51             record = recordRepository.save(record);
52         }
53
54         // Update Digital Twin
55         digitalTwinService.updateBusDigitalTwin(bus, location, record);
56     }
57 }
```

13.5.2-BusSimulatorService.java

Esta classe define um serviço BusSimulatorService que simula o movimento de autocarros e gera dados de localização e registos do sistema em intervalos regulares. Ele atualiza a localização dos autocarros em um padrão circular, baseado em coordenadas centrais de Braga, e regista valores tais como a velocidade, consumo de energia, temperatura interna, pressão dos pneus e outros parâmetros. Esses dados são então salvos nos repositórios correspondentes.

```
1  package com.busmonitoring.service;
2
3  import com.busmonitoring.model.Bus;
4  import com.busmonitoring.model.BusLocation;
5  import com.busmonitoring.model.BusRecord;
6  import com.busmonitoring.repository.BusLocationRepository;
7  import com.busmonitoring.repository.BusRecordRepository;
8  import com.busmonitoring.repository.BusRepository;
9  import org.springframework.beans.factory.annotation.Autowired;
10 import org.springframework.scheduling.annotation.Scheduled;
11 import org.springframework.stereotype.Service;
12
13 import java.time.LocalDateTime;
14 import java.util.Random;
15
16 @Service
17 public class BusSimulatorService {
18
19     @Autowired
20     private BusRepository busRepository;
21
22     @Autowired
23     private BusLocationRepository locationRepository;
24
25     @Autowired
26     private BusRecordRepository recordRepository;
27
28     private final Random random = new Random();
29
30     // Center of Braga coordinates
31     private static final double CENTER_LAT = 41.5454;
32     private static final double CENTER_LNG = -8.4265;
```



```

31 private static final double CENTER_LAT = 41.5454;
32 private static final double CENTER_LNG = -8.4265;
33 private static final double RADIUS = 0.01; // Approximately 1km radius
34
35 @Scheduled(fixedRate = 5000) // Update every 5 seconds
36 public void simulateBusMovement() {
37     busRepository.findAll().forEach(this::updateBusData);
38 }
39
40 private void updateBusData(Bus bus) {
41     // Generate new location
42     BusLocation location = new BusLocation();
43     location.setBus(bus);
44     location.setTimestamp(LocalDateTime.now());
45
46     // Simulate movement in a circular pattern
47     double angle = (System.currentTimeMillis() / 10000.0) % (2 * Math.PI);
48     location.setLatitude(CENTER_LAT + RADIUS * Math.cos(angle));
49     location.setLongitude(CENTER_LNG + RADIUS * Math.sin(angle));
50
51     locationRepository.save(location);
52
53     // Generate new system status
54     BusRecord record = new BusRecord();
55     record.setBus(bus);
56     record.setTimestamp(LocalDateTime.now());
57
58     // Simulate realistic values with some randomization
59     record.setVelocidade(30 + random.nextDouble() * 20); // 30-50 km/h
60     record.setEnergyConsumption(15 + random.nextDouble() * 10); // 15-25 kWh
61     record.setInternalTemperature(20 + random.nextDouble() * 5); // 20-25°C
62     record.setTirePressure(2.2 + random.nextDouble() * 0.4); // 2.2-2.6 bar
63     record.setEngineTemperature(70 + random.nextDouble() * 20); // 70-90°C
64     record.setOccupiedSeats(random.nextInt(bus.getTotalCapacity())); // 0 to max capacity
65
66     // Status values
67     record.setBrakeStatus(random.nextDouble() > 0.1 ? "Normal" : "Check Required");
68     record.setSuspensionStatus(random.nextDouble() > 0.1 ? "Normal" : "Check Required");
69     record.setKilometerStatus(String.format("%.1f km", 50000 + random.nextDouble() * 1000));
70
71     recordRepository.save(record);
72 }
73 }

```

13.6-Config

Pasta onde estão localizados os ficheiros referentes à config.

```

v config
J AzureDigitalTwinsConfig.java
J WebConfig.java

```


13.6.1- WebConfig.java

Esta classe implementa a interface `WebMvcConfigurer` para configurar manipuladores de recursos no Spring. Ele regista um manipulador de recursos que mapeia URLs, garantindo que os recursos sejam carregados de forma adequada.

```
1 package com.busmonitoring.config;
2
3 import org.springframework.context.annotation.Configuration;
4 import org.springframework.web.servlet.config.annotation.ResourceHandlerRegistry;
5 import org.springframework.web.servlet.config.annotation.WebMvcConfigurer;
6 import org.slf4j.Logger;
7 import org.slf4j.LoggerFactory;
8
9 @Configuration
10 public class WebConfig implements WebMvcConfigurer {
11     private static final Logger logger = LoggerFactory.getLogger(WebConfig.class);
12
13     @Override
14     public void addResourceHandlers(ResourceHandlerRegistry registry) {
15         logger.info("Configuring resource handlers...");
16         registry.addResourceHandler("/models/**")
17             .addResourceLocations("classpath:/static/models/")
18             .setCachePeriod(0);
19         logger.info("Resource handlers configured successfully");
20     }
21 }
```

13.7-BusMonitoringApplication.java

Este código define a classe para a aplicação Spring Boot. Além disso, permite o agendamento de tarefas, e a possibilidade de execução de métodos com agendamento regular tais como a simulação do movimento dos autocarros. O método `main` inicia a aplicação Spring Boot.

```
1 package com.busmonitoring;
2
3 import org.springframework.boot.SpringApplication;
4 import org.springframework.boot.autoconfigure.SpringBootApplication;
5 import org.springframework.scheduling.annotation.EnableScheduling;
6
7 @SpringBootApplication
8 @EnableScheduling
9 public class BusMonitoringApplication {
10     public static void main(String[] args) {
11         SpringApplication.run(BusMonitoringApplication.class, args);
12     }
13 }
```

13.8- Interface



Available Buses

1

Model: 1
Capacity: 1
Engine: 1

Monitor Bus



Bus ID: 1

1

High Internal Temp: 22.5°C

Low Tire Pressure: 1.0 bar

Low Tire Pressure: 1.0 bar

Low Tire Pressure: 1.0 bar

Reset View Auto Rotate

Mapa Satélite



45.5 km/h
Current Speed

25.3 kWh
Energy Consumption

30/1
Passengers

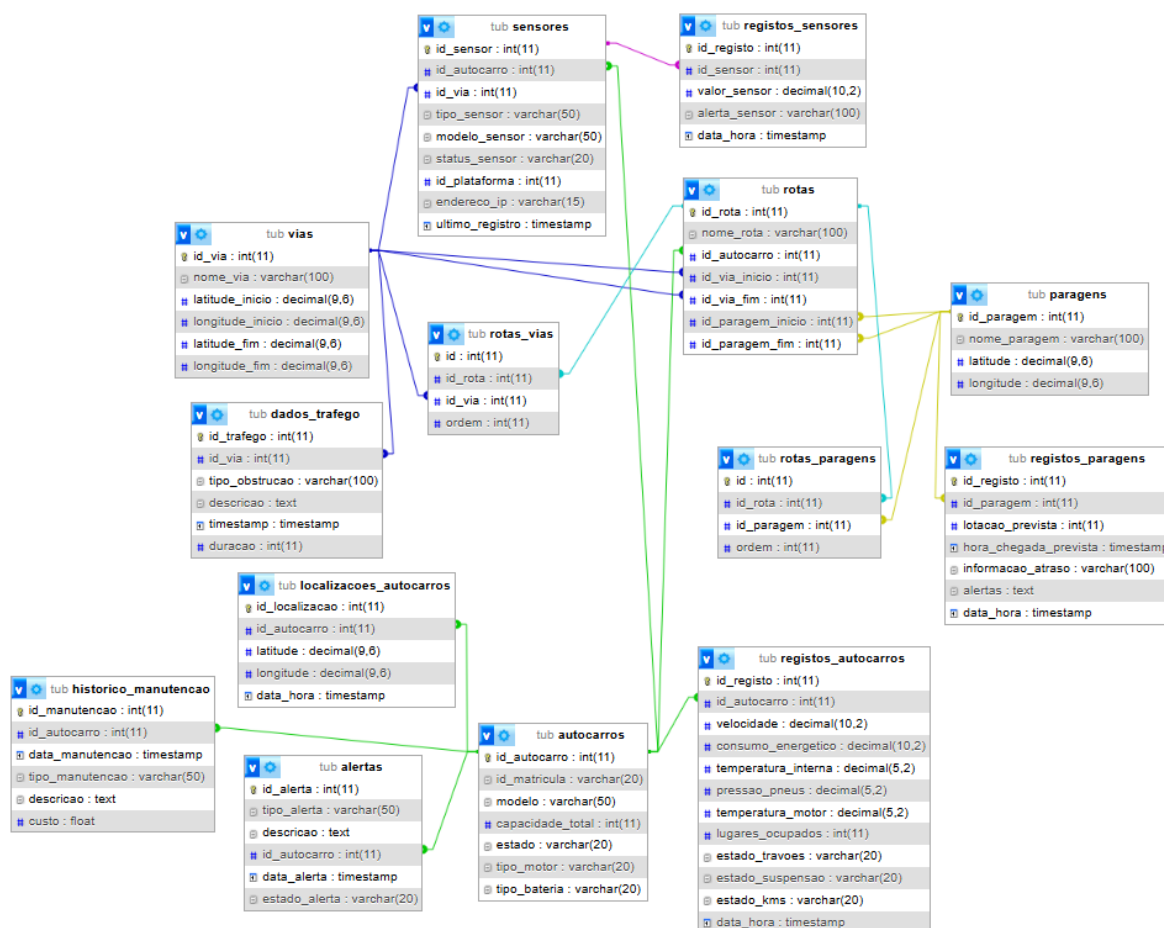
22.5°C
Internal Temperature

System Status

All Records

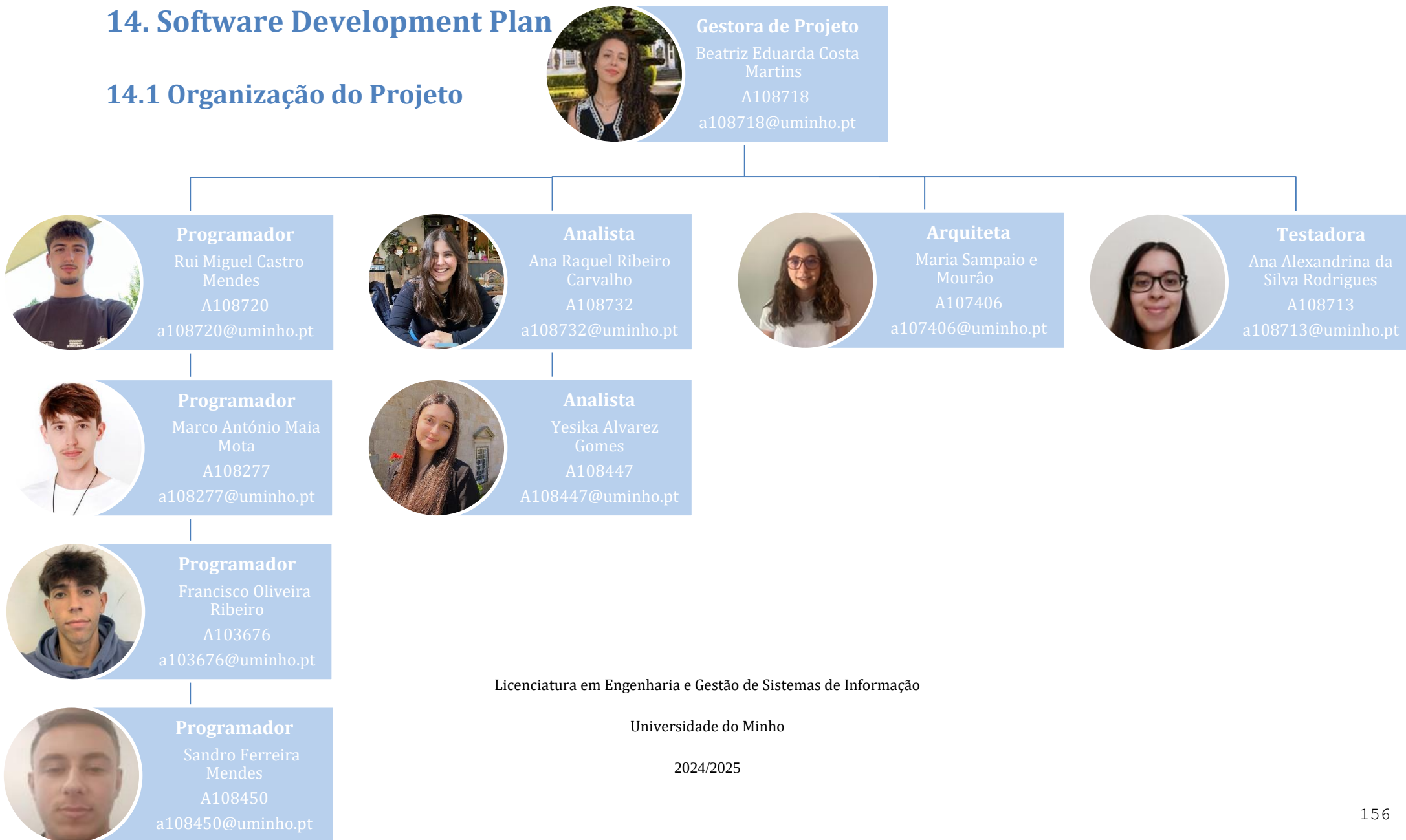
Tire Pressure	1 bar	Engine Temperature	75.2°C
Brake Status	Normal	Suspension Status	Normal
Kilometer Status	51234 km	Engine Type	1

13.9 – Modelo Base de Dados



14. Software Development Plan

14.1 Organização do Projeto



Papeis	Atribuição	Responsabilidade
Gestor de Projeto	Beatriz Martins	Assumir a liderança do planeamento; Gerir e coordenar a comunicação com as partes interessadas; Assegurar o foco e cumprimento dos objetivos definidos; Capacidade de negociação.
Arquiteto	Maria Mourão	Definir a arquitetura do software; Tomada de decisões técnicas para a implementação global do sistema.
Analista	Ana Carvalho Yesika Gomes	Responder às necessidades e pedidos do cliente; Possuir habilidades de comunicação.
Testador	Ana Rodrigues	Identificar, planear e executar os testes necessários; Registrar e analisar os resultados.
Programador	Rui Mendes Marco Mota Sandro Mendes Francisco Ribeiro	Desenvolver a interface do utilizador; Implementar e testar unidades; Integrar os componentes que fazem parte da solução.

14.2 Milestones e Objetivos

Iteração	Fase	Objetivos	Início (Fim Previsto)	Duração Estimada
M1	Planeamento	-Organização do trabalho e da equipa, definir o papel de cada membro; -Desenvolver o Plano do Projeto; -Avaliar Riscos e desenvolver <i>Risk List</i>; -Realizar o levantamento de necessidades do cliente; -Desenvolver as <i>Key Milestones</i>; -Desenvolver os <i>High-Level Objectives</i>; -Identificar problemas e desenvolver <i>Issues</i>; -Identificar e desenvolver os critérios de avaliação; -Criar um plano de iteração.	14/02/2025 (21/02/2025)	7 dias
M2	<i>Inception</i>	-Desenvolver/continuar plano de iteração -Desenvolver a ordem de trabalho -Definir ideia a ser desenvolvida -Pesquisar e estudar sobre o tema, aplicações e papéis -Identificar e Definir Requisitos -Iniciar os System-Wide-Requirements -Definir e iniciar os Casos de Uso -Preparação do ambiente -Desenvolver o Project Defined Process -Desenvolver Tools -Configurar ferramentas -Desenvolvimento da arquitetura -Preparar reunião com cliente	22/02/2025 (21/03/2025)	28 dias
		- Desenvolver/continuar plano de iteração -Finalizar os <i>System-Wide-Requirements</i> -Construção dos Casos de Uso -Definir cenários completos de interação dos atores com o sistema -Completar visão geral dos casos de		

		uso -Finalizar o <i>Architecture Notebook</i> -Finalização do desenvolvimento da arquitetura -Avançar com o modelo e implementação da Base de Dados -Definir Entidades, atributos e relações detalhadamente -Refinar o <i>Project Defined Process</i>		
		- Desenvolver/continuar plano de iteração -Rever e atualizar planeamento do projeto -Rever artefactos desenvolvidos até ao momento -Finalização do desenvolvimento dos Casos de Uso -Descrição detalhada dos Casos de Uso -Iniciar os diagramas de sequência -Iniciar o <i>Build</i> -Iniciar protótipo inicial das Digital Twin -Implementação da camada <i>Model</i> -Criar classes e respetivas camadas -Implementação da <i>Main</i>		
		- Desenvolver/continuar plano de iteração -Rever e atualizar plano do projeto -Rever M1 e registar possíveis mudanças -Rever todos os artefactos do M2 -Terminar diagramas de sequência -Terminar detalhamento dos Casos de Uso -Fazer os Test Cases -Terminar o glossário -Implementação (Acrescentar as tarefas) -Terminar o artefacto <i>Build</i> -Correr testes ao código -Planear M3 -Formatar Relatório		
M3	<i>Elaboration</i>	-Desenvolver/continuar plano de Iteração -Work Order (planeamento das semanas) -Estudar tarefas a ser desenvolvidas pelas analistas	22/03/2025 (02/05/2025)	42 dias

		-Estudar tarefas a ser desenvolvidas pela tester -Estudar tarefas a ser desenvolvidas pela arquiteta -Adicionar dados tub à base de dados -Organizar as funcionalidades -Implementar funcionalidades (inicio) -Continuar desenvolvimento das digital twins -Reunião Semanal		
		-Desenvolver/continuar plano de iteração -<i>Issues List</i> -Project Measurements/Status Assessment -Iteration Assessment -Review Record -Work Order -Identificar e Refinar Requisitos -Detalhar Cenários de Casos de Uso -Detalhar <i>System-Wide Requirements</i> -Refinar a Arquitetura -<i>Design the Solution</i> -Desenvolver o Design -Implementar Testes de Desenvolvimento -Desenvolver o Artefacto <i>Developer Test</i> -Implementar Solução -Executar Testes de Desenvolvimento -Desenvolver o <i>Test Log</i> -Integrar e Criar <i>Build</i> -Testar a Solução -Implementar Testes de Desenvolvimento -Desenvolver o <i>Test Script</i> -Executar Testes -Revisões do M1 e M2 (se necessário) -Rever Relatório		

		-Formatar Relatório.		
M5	Construction		03/05/2025 (30/05/2025)	28 dias
		-Implementar a solução; -Realizar os testes a solução; -Completar o desenvolvimento do sistema; -Criar documentação de apoio; -Realizar os últimos testes.		
M5	Apresentação Comercial	-Apresentação final ao cliente.	02/06/2025	-
M5	Apresentação Técnica	-Apresentação final à equipa docente.	04/06/2025	-

14.3 Lições aprendidas

Durante o desenvolvimento desta Iteração, foram aprendidas várias lições, com um foco especial na organização, preparação e comunicação da equipa. Estes pontos são fundamentais para o sucesso de uma iteração e para a eficiência no desenvolvimento de tarefas, ajudando a melhorar o processo nas próximas fases do projeto.

- **Importância de um plano organizado:** Um plano bem estruturado desde o início é essencial para garantir a execução eficiente das tarefas, evitando confusão, atrasos e sobrecarga de trabalho
- **Definição de prioridades:** Definir claramente as prioridades das tarefas permite uma gestão eficiente do tempo e recursos, evitando bloqueios e garantindo a execução das atividades mais críticas primeiro.
- **Estudo e preparação antecipados:** A preparação antecipada das tarefas melhora a eficácia, reduz erros e aumenta a confiança e a produtividade da equipa durante a execução.

- **Comunicação eficaz:** A comunicação eficaz entre a equipa facilita a troca de ideias, a resolução rápida de problemas e o alinhamento de expectativas, garantindo o bom andamento das tarefas.

15. Risk List

No desenvolvimento do Digital Twin é essencial identificar os potenciais riscos que possam comprometer a qualidade e desempenho do sistema em questão. Dito isto organizados os todos os riscos encontrados em uma lista organizados pelo seu grau de ocorrência. Para além dos riscos, também pensamos em ações de mitigação, ou seja, em propostas para reduzir a gravidade e a probabilidade do risco apresentado acontecer.

Id: Atribuição de um código de modo a tornar a interpretação mais simplificada;

Descrição: Descrição sobre o risco em causa;

Tipo: Identifica o género que o tipo em causa é, podendo ser técnico, segurança...

Probabilidade: Refere a probabilidade de acontecimento do risco numa escala 0 a 5;

Impacto: Refere o impacto que o risco pode ter no desenvolvimento do projeto numa escala de 0 a 5;

Magnitude: Indica a magnitude do risco no programa;

Ações de mitigação: Soluções propostas para resolver o risco em causa;

Impacto após Mitigação: Refere o impacto que o risco pode ter após as ações tomadas;

Probabilidade após mitigação: Refere a probabilidade de acontecimento do risco depois das soluções propostas numa escala 0 a 5;

Magnitude após mitigação: Indica a magnitude do risco no programa após as soluções propostas;

Estado: Estado em que o risco em questão se encontra, ativo ou passivo.

Legenda da seguinte tabela:

- P: Probabilidade [1 - 5];
- I: Impacto [1-5];
- M: Magnitude [PxI];
- I ap. Mi: Impacto após Mitigação;
- M ap. Mi: Magnitude após Mitigação;
- P ap. Mi: Probabilidade após Mitigação;
- A: Ativo;
- P: Passivo.

Id	Descrição	Tipo	P	I	M	Ações de Mitigação	I ap. Mi	P. ap. Mi	M. ap. Mi	Estado
R1	Gestão inadequada das credenciais de acesso	Segurança	4	5	20	Controlo de Acessos Baseado em Privilégios → Garantir que os utilizadores têm apenas os acessos necessários para as suas funções. Revisão Periódica de Permissões → Auditar regularmente as permissões de acesso e remover as desnecessárias. Restrições de IP e Geolocalização → Limitar o acesso a partir de locais e dispositivos não autorizados para reduzir o risco de ataques externos.	2	4	8	A

R2	O modelo virtual pode não representar corretamente o caso real	Técnico	4	5	20	Análises frequentes dos dados → Realizar análises contínuas dos dados e informações fornecidas da TUB para garantir que o modelo virtual esteja alinhado com os objetivos previstos.	3	4	12	A
R3	Chaves primárias e estrangeiras mal definidas na base de dados	Técnico	4	5	20	Definir Chaves Primárias em Todas as Tabelas → Garantir que cada tabela tem um identificador único para evitar registos duplicados. Revisão e Auditoria da estruturação de Dados → Realizar revisões periódicas do esquema da base de dados para evitar falhas de integridade.	2	3	6	P

R4	Desenvolvimento de modelos de caso de uso mal definidos e de difícil interpretação	Técnico	4	5	20	Filtragem de informação → Filtrar a informação necessária e criação de modelos intuitivos e claros.	3	4	12	P
R5	Interface do programa de difícil interpretação e uso	Técnico	4	5	20	Testes de usabilidade e feedback de utilizadores → Realizar testes com utilizadores e ajustar a interface com base nas dificuldades identificadas. Revisão do design com boas práticas de UI → Adotar padrões intuitivos, como navegação clara, ícones compreensíveis e disposição lógica dos elementos.	2	3	6	A

R6	Má interpretação e escolha da informação a ser apresentada no <i>Digital Twin</i> de cada componente rodoviário	Técnico	4	5	20	Testes iterativos com cenários reais → Implementar protótipos e realizar simulações para avaliar como os dados são interpretados e ajustá-los conforme necessário.	2	3	6	A
R7	Discrepâncias entre a arquitetura e os casos de uso	Técnico	4	5	20	Revisões frequentes de alinhamento → Realizar revisões regulares entre os responsáveis pela definição dos casos de uso e a equipa de arquitetura para garantir que os requisitos funcionais e não funcionais estão corretamente refletidos no desenho arquitetural.	2	4	8	P

R8	Discrepâncias entre a arquitetura e a programação	Técnico	4	5	20	Definição clara de padrões e <i>guidelines</i> → Estabelecer normas e boas práticas de desenvolvimento, garantindo que os programadores implementem as soluções em conformidade com a arquitetura definida.				A
R9	Dificuldades na integração com os sistemas existentes da TUB	Técnico	4	4	16	Definição de padrão → Criação de um padrão claro de comunicação para garantir que as APIs estejam corretamente inseridas.	3	3	9	A
R10	Vulnerabilidade e a ataques cibernéticos	Segurança	4	4	16	Criação de restrições → Implementação de restrições de acesso e incluir o tópico da segurança nas reuniões.		3	6	A

R11	Dificuldades na adaptação dos programas requeridos (Java 22, NIOP)	Técnico	4	4	16	Reuniões frequentes → Planear reuniões frequentes de modo a perceber as dificuldades dos programadores e alocar mais recursos conforme necessário.	3	4	12	P
R12	Insuficiência de testes	Técnico	4	4	16	Adoção de uma estratégia de testes abrangente → Implementar testes unitários, de integração e de aceitação, incluindo simulações realistas para garantir a fiabilidade do sistema. Validação contínua com dados reais → Comparar os dados do <i>Digital Twin</i> com medições do mundo real para identificar discrepâncias e corrigi-las antes da implementação completa.	2	3	6	A

R13	Paragens frequentes por problemas técnicos.	Técnico	4	4	16	Monitorização e manutenção preventiva → Implementar sistemas de monitorização contínua e um plano de manutenção para detetar e corrigir problemas antes que causem falhas.	2	3	6	A
R14	Falta de Normalização ou Normalização Excessiva no Digital Twin dos componentes rodoviários	Técnico	4	4	16	Definição de um modelo de dados equilibrado → Aplicar um nível adequado de normalização, garantindo um compromisso entre eficiência de armazenamento e desempenho das consultas.	2	3	6	A

R15	Falta de Suporte para o Aumento da Carga	Técnico	4	4	16	Testes de carga e stress → Realizar testes periódicos para simular cenários de alto tráfego e identificar potenciais erros antes que se tornem problemas reais.	2	3	6	A
R16	Gestão Ineficiente do Armazenamento dos Dados e problemas na cloud	Técnico	4	4	16	Políticas de armazenamento e arquivamento → Implementar regras para compressão, retenção e arquivamento de dados antigos, garantindo que apenas informações relevantes fiquem ativas. Monitorização e otimização da base de dados → Utilizar ferramentas para monitorizar o uso do armazenamento, remover redundâncias e otimizar índices e consultas.	2	3	6	A

						Criação de cópias de segurança → Fazer gravações frequentes dos dados de modo a salvaguardar todos os dados.				
R17	Má comunicação entre a base de dados e classes	Técnico	4	4	16	Prevenir Erros em queries → Testar queries Antes da Implementação. Antes de executar no código Java, testa a query diretamente na base de dados (exemplo: DBeaver, MySQL Workbench).	2	3	6	A
R18	Modelagem de Classes Ineficiente	Técnico	4	4	16	Estratégias de Modelagem e Organização de Classes → Utilizar design patterns adequados (ex: Factory, Repository, DAO). Criar um diagrama UML de classes antes de codificar.	2	3	6	P

R19	Falta de Conhecimento Técnico sobre Digital Twins	Técnico	4	4	16	Consolidação das competências do tester e da colaboração com os Desenvolvedores → Formação específica sobre conceitos e funcionamento de Digital Twins. Trabalho colaborativo entre testers e desenvolvedores para entender melhor o sistema.	2	3	6	
R20	Dependência Tecnológica (Java 22, NIOP)	Técnico	3	4	12	Estratégias para Aceleração da Adoção de Novas Tecnologias → Pesquisa prévia sobre o funcionamento em NIOP e Java 22. Criação de documentação interna com exemplos práticos.	2	3	6	A

16. Conclusão

Este relatório sintetiza o trabalho desenvolvido pela equipa no âmbito do segundo momento de avaliação da Unidade Curricular de Desenvolvimento de Aplicações Informáticas(DAI), tendo como foco a apresentação dos artefactos planeados e a evolução do projeto alinhado com a metodologia do OpenUP.

Ao longo desta fase, foram atribuídas tarefas específicas a cada elemento da equipa, promovendo a colaboração ativa e o cumprimento dos objetivos propostos. Foram realizadas diversas atividades, incluindo a análise de requisitos, o desenvolvimento dos Casos de Uso, e a definição da arquitetura do sistema. Destaca-se também o contacto com o cliente, que possibilitou a recolha de feedback relevante para o alinhamento da solução às suas necessidades.

Neste momento, foi possível concretizar e apresentar os principais artefactos: visão do produto, modelo de caso de uso, casos de teste, requisitos do sistema, glossário, o processo definido para o projeto, arquitetura do sistema, plano de desenvolvimento, lista de riscos e lista de ferramentas. Adicionalmente, foram realizados os primeiros testes ao protótipo desenvolvido, permitindo validar a direção tomada até à data.

Quanto ao nível de prontidão tecnológica do protótipo, destaca-se que o modelo 3D do Digital Twin do autocarro e das paragens está concluído, encontrando-se integrado com a base de dados. Além disso, foi desenvolvido e testado o código *back-end* em java fornecido, que permite a comunicação eficaz entre os componentes do sistema. No entanto, as únicas ligações descritas na arquitetura no diagrama dos diferentes componentes dos sistemas que ainda não foram demonstradas são as que envolvem o NIOP, especificamente a ligação do NIOP aos sensores e ao java, uma vez que ainda não tivemos acesso ao NIOP.

Desta forma, considera-se que todos os artefactos foram desenvolvidos com sucesso, contribuindo para uma solução coesa e ajustada às necessidades do cliente, destacando-se a evolução do protótipo do nosso sistema de monitorização para os TUB.

16.1 Ferramentas de IA

Artefacto	Descrição
Vision	Utilizada na reformulação de texto, sugestão de possíveis funcionalidades e benefícios do produto tendo em conta as necessidades do cliente e as ideias estabelecidas pela equipa.
System-Wide-Requirements	Utilizada na sugestão de requisitos e a sua priorização, assim como a sua estruturação.
Project Defined Process	Utilizada na reformulação de texto e correção de erros linguísticos.
Tools	Utilizada na análise e seleção das ferramentas tecnológicas a adotar e como as utilizar, e na descrição das mesmas
Glossary	Utilizada na elaboração de definições claras e concisas, tradução técnica e sugestão de termos relevantes.
Use-Case-Model	Auxiliou na identificação dos principais atores e descrição dos fluxos, a partir das funcionalidades definidas pela equipa, garantindo clareza na definição das iterações.
Test Case	Auxiliou na definição de cenários de teste e estruturação dos mesmos, com base nos requisitos do sistema e casos de uso.
Build	Utilizada para geração de código e correção de erros, assim como na estruturação da ordem de realização das tarefas de implementação e na criação dos modelos 3D, facilitando o desenvolvimento e tornando o processo mais eficiente.

Foi ainda utilizada na reformulação e correção linguística das restantes partes do relatório, assegurando clareza e coerência na comunicação escrita.

Importa salientar que esta tecnologia foi empregue apenas como ferramenta de apoio, sendo todo o desenvolvimento das ideias, decisões e conteúdos realizado com base no trabalho da equipa.